Neural Networks & Deep Learning

The Complete Guide — from a single neuron to CNNs, RNNs, attention and generative models

Inside: 23 chapters in 4 parts, following the training modules in order • DL fundamentals, perceptron, XOR, MLP • ANN training: activations, losses, backpropagation with fully worked numbers, optimisers, initialisation, normalisation, regularisation • Keras MNIST hands-on • CNN theory with shape & parameter calculations • LeNet, AlexNet, ZFNet, VGG • CIFAR-10 hands-on + Weekly Assessment 02 • Inception, ResNet, DenseNet, MobileNet, SENet, EfficientNet, ViT, ConvNeXt • transfer learning • R-CNN family, YOLO, SSD, RetinaNet, DETR • U-Net, DeepLab, Mask R-CNN, SAM • CNN visualisation (Grad-CAM) • end-to-end CV project • RNN theory, BPTT, vanishing gradients • LSTM, GRU, bidirectional • seq2seq, beam search • Bahdanau/Luong/self-attention • LSTM time-series forecasting • GANs (theory + DCGAN code), WGAN, cGAN, Pix2Pix, CycleGAN, StyleGAN • autoencoders & VAEs • comparison with diffusion & LLMs + Weekly Assessment 03 • ~80 interview Q&A • cheat sheet

Prepared for Mansavi Aggarwal

Contents

1. Neural Networks & Deep Learning Fundamentals **8**

1.1 What is Deep Learning? . . . 8
1.2 From biological to artificial neurons . . . 9
1.3 The Perceptron (Rosenblatt, 1958) . . . 10
1.4 Multi-Layer Perceptron (MLP / ANN) . . . 10
1.5 Tensors — the data structure of deep learning . . . 11
1.6 Frameworks . . . 11

2. ANN Architecture & Training Process **12**

2.1 Forward propagation . . . 12
2.2 Activation functions . . . 12
2.3 Output layers and loss functions . . . 13
2.4 Gradient descent . . . 13
2.5 Backpropagation . . . 14
2.6 Optimisers . . . 15
2.7 Weight initialisation . . . 16
2.8 Vanishing and exploding gradients . . . 16
2.9 Normalisation layers . . . 16
2.10 Regularisation . . . 17
2.11 Hyper-parameters and the training recipe . . . 17

3. Hands-on: ANN with Keras (MNIST Demo) **19**

3.1 The MNIST dataset . . . 19
3.2 Complete Keras code . . . 19
3.3 What training looks like (real run) . . . 21
3.4 Functional API and subclassing . . . 21
3.5 The same model in PyTorch . . . 21
3.6 Experiments to try (and what you should observe) . . . 22

4. Convolutional Neural Networks — Theory **24**

4.1 Why CNNs? . . . 24
4.2 The convolution operation . . . 24
4.3 Stride, padding and output size . . . 25
4.4 Channels, filters and parameter counts . . . 26
4.5 Pooling . . . 26
4.6 Receptive field and feature hierarchy . . . 27
4.7 1×1 convolutions . . . 27
4.8 A complete CNN . . . 27

5. Classic CNN Architectures — LeNet, AlexNet, VGG (+ ZFNet) 29

5.1 LeNet-5 (LeCun et al., 1998) . . . 29
5.2 AlexNet (Krizhevsky, Sutskever, Hinton, 2012) . . . 30
5.3 ZFNet (Zeiler & Fergus, 2013) . . . 30
5.4 VGG-16 / VGG-19 (Simonyan & Zisserman, 2014) . . . 30
5.5 Comparison . . . 31

6. Hands-on: Image Classification with CNN + Weekly Assessment 02 32

6.1 CNN for MNIST (≈99% accuracy) . . . 32
6.2 CIFAR-10 project (32×32 colour images, 10 classes) . . . 32
6.3 Common mistakes . . . 33
6.4 Weekly Assessment 02 (with answers) . . . 33

7. Advanced CNN Architectures 35

7.1 GoogLeNet / Inception (Szegedy et al., 2014) . . . 35
7.2 ResNet (He et al., 2015) . . . 35
7.3 DenseNet (Huang et al., 2017) . . . 36
7.4 MobileNet (Howard et al., 2017) — efficient CNNs . . . 36
7.5 SENet — channel attention (Hu et al., 2017) . . . 37
7.6 EfficientNet (Tan & Le, 2019) . . . 37
7.7 Beyond CNNs: Vision Transformers and ConvNeXt . . . 37
7.8 Comparison and choosing . . . 37

8. Transfer Learning with Pre-trained Models 38

8.1 Concept . . . 38
8.2 Strategies . . . 38
8.3 Keras implementation . . . 38
8.4 PyTorch implementation . . . 39
8.5 Best practices . . . 39

9. Object Detection — R-CNN family, YOLO, SSD 40

9.1 The task . . . 40
9.2 Core concepts: IoU, NMS, anchors . . . 40
9.3 Evaluation: precision, recall, AP, mAP . . . 41
9.4 Two-stage detectors: the R-CNN family . . . 41
9.5 YOLO — You Only Look Once . . . 42
9.6 SSD — Single Shot MultiBox Detector (Liu et al., 2016) . . . 43
9.7 RetinaNet and focal loss . . . 43
9.8 Transformer detectors . . . 43
9.9 Comparison and choosing . . . 43

9.10 Hands-on . . . 44

10. Image Segmentation — U-Net, Mask R-CNN 45

10.1 Fully Convolutional Networks (FCN, Long et al., 2015) . . . 45
10.2 U-Net (Ronneberger et al., 2015) . . . 46
10.3 DeepLab family . . . 47
10.4 Mask R-CNN (He et al., 2017) . . . 47
10.5 Losses and metrics . . . 47
10.6 Segment Anything (SAM) . . . 48

11. CNN Visualisation Techniques 49

11.1 Why visualise? . . . 49
11.2 Visualising filters and feature maps . . . 49
11.3 Occlusion sensitivity . . . 49
11.4 Saliency maps (vanilla gradients) . . . 50
11.5 Grad-CAM (Selvaraju et al., 2017) . . . 50
11.6 Other techniques . . . 50

12. Hands-on: End-to-End Computer Vision Project 51

12.1 Project: defect detection on manufactured parts (classification) . . . 51
12.2 Project structure . . . 51
12.3 Key code . . . 51
12.4 Evaluation & error analysis checklist . . . 52

13. Recurrent Neural Networks (RNN) — Theory 54

13.1 Sequential data . . . 54
13.2 The recurrent neuron . . . 54
13.3 RNN architectures . . . 55
13.4 Training: Backpropagation Through Time (BPTT) . . . 55
13.5 Keras implementation . . . 55
13.6 Applications and limitations . . . 56

14. The Vanishing (and Exploding) Gradient Problem in RNNs 57

14.1 The math . . . 57
14.2 Consequences . . . 57
14.3 Remedies . . . 57

15. LSTM & GRU: Implementation and Applications; Bidirectional RNN and LSTM 59

15.1 LSTM — Long Short-Term Memory (Hochreiter & Schmidhuber, 1997) . . . 59
15.2 GRU — Gated Recurrent Unit (Cho et al., 2014) . . . 60

15.3 Stacked and Bidirectional RNNs . . . 61
15.4 Implementations . . . 61
15.5 Applications . . . 62

16. Sequence-to-Sequence Models 63

16.1 Architecture . . . 63
16.2 Training with teacher forcing . . . 63
16.3 Inference and decoding strategies . . . 64
16.4 Keras implementation (character/word-level translation) . . . 64
16.5 The bottleneck problem . . . 65

17. Attention Mechanism — Theory and Implementation 66

17.1 Intuition . . . 66
17.2 Bahdanau (additive) attention, 2014 . . . 66
17.3 Luong (multiplicative) attention, 2015 . . . 67
17.4 Self-attention and scaled dot-product attention . . . 67
17.5 Implementations . . . 68
17.6 From attention to Transformers . . . 69

18. Hands-on: Time Series Forecasting with LSTM 71

18.1 Understanding the series . . . 71
18.2 Framing as supervised learning . . . 71
18.3 Models . . . 72
18.4 Multivariate and multi-step forecasting . . . 72
18.5 Baselines and evaluation . . . 73

19. Generative Models Overview & GAN Fundamentals 75

19.1 Discriminative vs generative . . . 75
19.2 The GAN idea (Goodfellow et al., 2014) . . . 75
19.3 Training algorithm . . . 76
19.4 DCGAN architecture guidelines (Radford et al., 2015) . . . 76
19.5 Why GANs are hard to train . . . 77
19.6 Evaluating generative models . . . 78

20. Advanced GAN Variants & Variational Autoencoders (Conceptual) 79

20.1 Advanced GAN variants . . . 79
20.2 Autoencoders . . . 80
20.3 Variational Autoencoder (Kingma & Welling, 2013) . . . 80
20.4 Autoencoder vs VAE vs GAN . . . 81

21. Hands-on Demo & Comparison with Modern GenAI + Weekly Assessment 03

21.1 Hands-on demo plan 82
21.2 Diffusion models in one page 82
21.3 Autoregressive transformers (LLMs) 83
21.4 Comparison 83
21.5 Weekly Assessment 03 (with answers) 84

22. Interview Questions & Answers **85**

23. Cheat Sheet **90**

23.1 Formulas 90
23.2 Which model when? 91

Part I — Neural Network Foundations

Neurons, perceptrons, MLPs, activation & loss functions, backpropagation, optimisers, initialisation, normalisation, regularisation, and a full ANN project on MNIST

CHAPTER 1

Neural Networks & Deep Learning Fundamentals

AI vs ML vs DL • why deep learning now • biological vs artificial neuron • perceptron & its learning rule • XOR problem • multi-layer perceptron • universal approximation • depth • tensors • frameworks

1.1 What is Deep Learning?

Deep learning (DL) is the part of machine learning that uses **neural networks with many layers** to learn **representations** directly from raw data (pixels, audio samples, characters). Classical ML relies on hand-crafted features (edge counts, TF-IDF); deep networks learn the features themselves, layer by layer, from simple patterns to complex concepts.

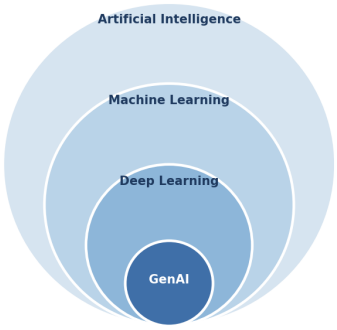


Figure: Deep learning is a subset of ML, which is a subset of AI; generative AI is mostly built on deep learning.

	Classical ML	Deep Learning
Features	hand-engineered by experts	learned automatically (representation learning)
Data needed	small–medium works well	large (or pre-trained models + transfer learning)
Compute	CPU, seconds–minutes	GPU/TPU, minutes–weeks
Best on	tabular / structured data	unstructured: images, audio, text, video
Interpretability	often high	lower (needs XAI techniques)
Examples	XGBoost, SVM, Logistic Regression	CNN, RNN/LSTM, Transformers, GANs, diffusion

Why did deep learning take off after 2012?

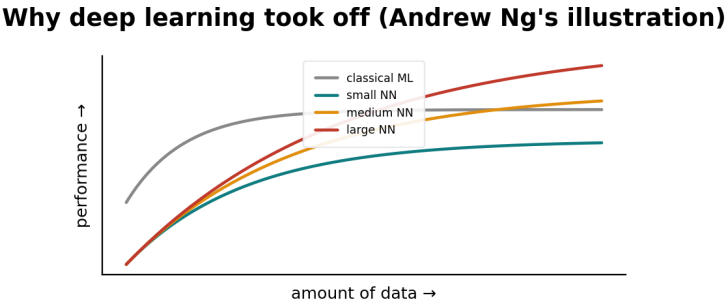


Figure: Classical algorithms plateau with more data; larger neural networks keep improving (conceptual).

- **Data**: internet-scale labelled datasets (ImageNet: 1.2M images, 1000 classes).
- **Compute**: GPUs make the massive matrix multiplications of neural nets 10–100× faster.
- **Algorithms**: ReLU, dropout, batch normalisation, better initialisation, Adam, residual connections — they made deep networks trainable.
- **Software**: TensorFlow, Keras, PyTorch with automatic differentiation.
- **Milestone**: AlexNet won ImageNet 2012 with 16.4% top-5 error vs 25.8% the year before (Chapter 5).

1.2 From biological to artificial neurons

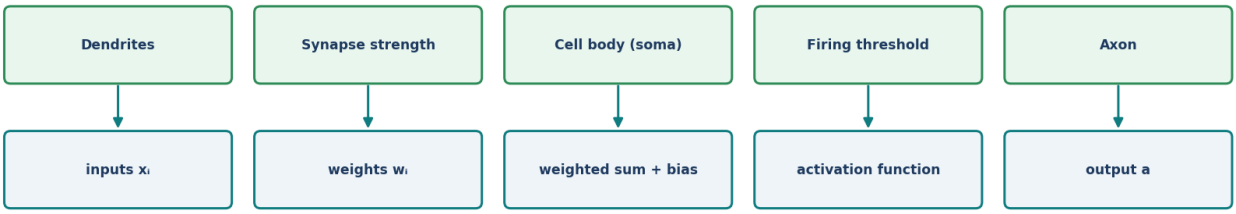


Figure: Analogy between a biological neuron and an artificial one (an analogy only — real neurons are far more complex).

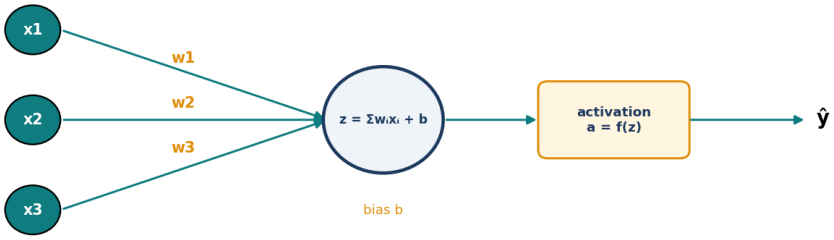


Figure: The artificial neuron: weighted sum plus bias, followed by a non-linear activation.

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b, \qquad a = f(z)$$

Weights express how important each input is; the **bias** shifts the activation threshold (lets the neuron fire even when all inputs are zero); the **activation function** adds non-linearity. Geometrically, $\mathbf{w}^T\mathbf{x} + b = 0$ is a hyperplane: a single neuron is a linear classifier.

1.3 The Perceptron (Rosenblatt, 1958)

$$\hat{y} = \text{step}(\mathbf{w}^T\mathbf{x} + b) \in \{0, 1\} \qquad \mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}, \quad b \leftarrow b + \eta\,(y - \hat{y})$$

step(z) = 1 if z ≥ 0, else 0

Learning rule: if the prediction is correct nothing changes; if it predicts 0 but should be 1, the inputs are added to the weights (pushing the boundary toward that point), and vice versa. The **perceptron convergence theorem** guarantees it finds a separating line in finite steps *if the data are linearly separable*.

Worked example — learning AND (η = 1, w = [0, 0], b = 0): sample (1,1) → z = 0 → ŷ = 1 ✓; sample (1,0) → ŷ = 1 but y = 0 → w = [−1, 0], b = −1; continuing a few epochs gives e.g. w = [2, 1], b = −3: (1,1) → 0 ≥ 0 → 1 ✓, (1,0) → −1 → 0 ✓, (0,1) → −2 → 0 ✓, (0,0) → −3 → 0 ✓.

The XOR problem and why we need hidden layers

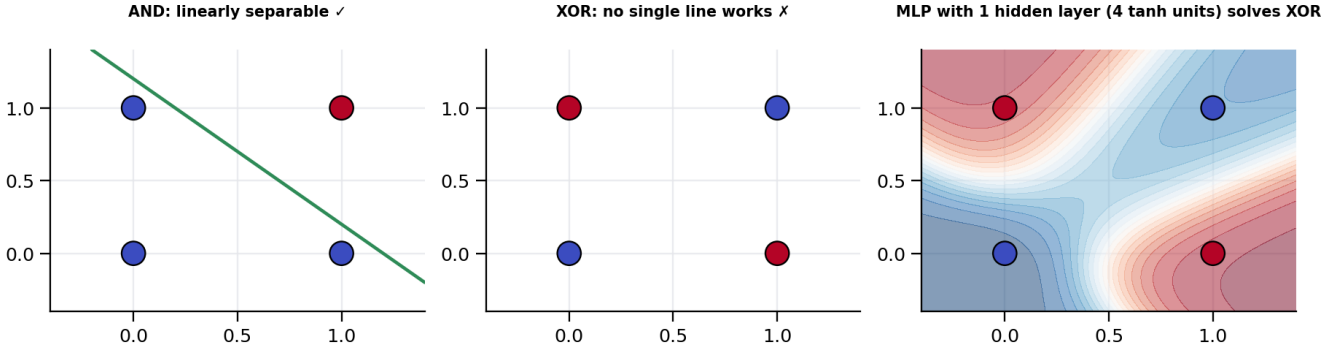


Figure: AND is linearly separable; XOR is not. A small MLP with one hidden layer learns a non-linear boundary (computed with scikit-learn).

Minsky & Papert (1969) showed a single perceptron cannot learn XOR — this contributed to the first 'AI winter'. The fix: stack neurons into **hidden layers** with non-linear activations, and train them with **backpropagation** (popularised by Rumelhart, Hinton & Williams, 1986). Hidden units create new features (e.g. h_1 = OR, h_2 = NAND) in which XOR = AND(h_1, h_2) is linearly separable.

1.4 Multi-Layer Perceptron (MLP / ANN)

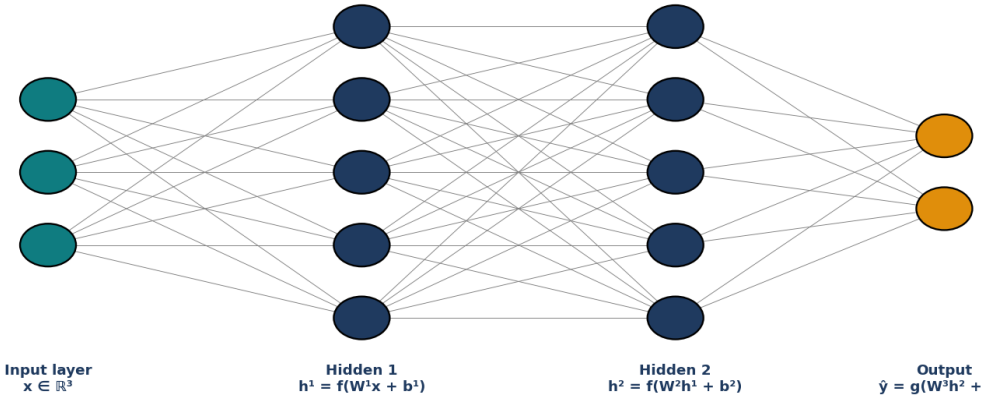


Figure: A fully connected (dense) feed-forward network.

Term	Meaning
Input layer	receives features; no computation
Hidden layers	learn intermediate representations
Output layer	produces predictions (size = number of targets/classes)

Term	Meaning
Dense / fully connected	every neuron connects to every neuron of the previous layer
Feed-forward	information flows one way (no loops) — contrast with RNNs
Depth / width	number of layers / neurons per layer
Parameters	all weights and biases — learned
Hyper-parameters	layers, units, activation, learning rate, batch size, epochs — chosen by you

Universal Approximation Theorem

A feed-forward network with **one hidden layer** of enough neurons and a non-linear activation can approximate any continuous function on a bounded domain to any accuracy (Cybenko 1989; Hornik 1991). It says such a network *exists*, not that training will find it or that it is efficient. In practice **deeper** networks represent complex functions with exponentially fewer neurons by reusing features hierarchically (edges → shapes → objects).

1.5 Tensors — the data structure of deep learning

Rank	Name	Shape example	Data
0	scalar	()	loss value
1	vector	(784,)	flattened MNIST image
2	matrix	(batch, features) = (32, 784)	tabular batch
3	3-D tensor	(batch, timesteps, features) = (32, 50, 8)	sequences / time series
4	4-D tensor	(batch, height, width, channels) = (32, 224, 224, 3)	images (Keras: channels-last; PyTorch: N, C, H, W)
5	5-D tensor	(batch, frames, H, W, C)	video

1.6 Frameworks

Framework	Style	Notes
Keras (TensorFlow / multi-backend Keras 3)	high-level: Sequential, Functional, subclassing; compile → fit → evaluate	fastest to learn; great for prototypes & production (TF Serving, TFLite)
PyTorch	imperative, Pythonic; explicit training loop; autograd	dominant in research and LLMs; Lightning for structure
JAX	functional, XLA compiled, grad / jit / vmap	high-performance research
ONNX	exchange format	deploy across runtimes

CHAPTER 2

ANN Architecture & Training Process

Forward propagation • activation functions • output layers & loss functions • gradient descent • backpropagation (worked numeric example) • computational graphs & autograd • optimisers • learning-rate schedules • weight initialisation • batch & layer normalisation • vanishing/exploding gradients • regularisation • hyper-parameters • debugging training

2.1 Forward propagation

$$\mathbf{z}^{[l]} = W^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \qquad \mathbf{a}^{[l]} = f^{[l]}(\mathbf{z}^{[l]}), \qquad \mathbf{a}^{[0]} = \mathbf{x}, \quad \hat{\mathbf{y}} = \mathbf{a}^{[L]}$$

$W^{[l]}$ has shape (n_l, n_{l-1}). For a batch, stack samples as rows: $Z = A \cdot W^{T} + b$ — one matrix multiply per layer, which GPUs do extremely fast.

Parameter count of a dense layer = inputs × units + units. Network 784 → 128 → 64 → 10: 100,480 + 8,256 + 650 = **109,386** parameters.

2.2 Activation functions

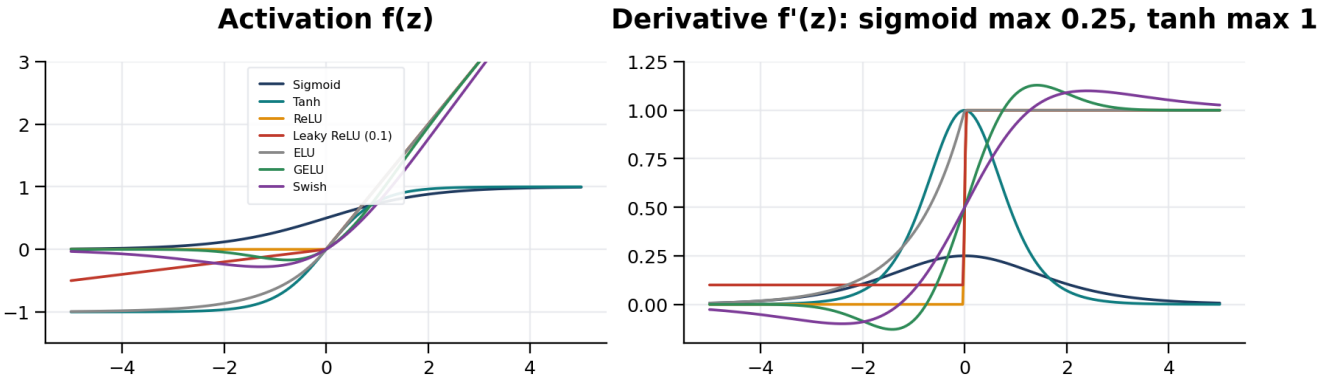


Figure: Common activations and their derivatives.

Function	Formula	Range	Pros	Cons / use
Sigmoid	$1/(1+e^{-z})$	(0, 1)	probability output	saturates, max gradient 0.25 → vanishing; not zero-centred. Binary output layer
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	(−1, 1)	zero-centred	still saturates; RNN/LSTM internals
ReLU	max(0, z)	[0, ∞)	cheap, no saturation for z>0, sparse	'dying ReLU' (stuck at 0). Default for hidden layers
Leaky ReLU / PReLU	max(αz, z)	(−∞, ∞)	fixes dying ReLU	α=0.01 fixed / learned
ELU / SELU	z if z>0 else $\alpha(e^{z}-1)$	(−α, ∞)	smooth, mean near 0	slower; SELU self-normalising
GELU	z·Φ(z)	≈(−0.17, ∞)	smooth; best in transformers	BERT, GPT
Swish / SiLU	z·σ(z)	≈(−0.28, ∞)	smooth, non-monotonic	EfficientNet, LLaMA (SwiGLU)

Function	Formula	Range	Pros	Cons / use
Softmax	$e^{z_k}/\Sigma e^{z_j}$	(0,1), sums to 1	class probabilities	multi-class output layer
Linear	z	$\mathbb{R}$	—	regression output

Key idea: Without non-linear activations, any stack of layers collapses into ONE linear layer: $W_2(W_1x) = (W_2W_1)x$. Non-linearity is what lets depth add expressive power.

2.3 Output layers and loss functions

Task	Output layer	Loss	Keras name
Regression	Dense(1), linear	MSE / MAE / Huber	'mse', 'mae', Huber()
Binary classification	Dense(1), sigmoid	binary cross-entropy	'binary_crossentropy'
Multi-class, one-hot labels	Dense(K), softmax	categorical cross-entropy	'categorical_crossentropy'
Multi-class, integer labels	Dense(K), softmax	sparse categorical CE	'sparse_categorical_crossentropy'
Multi-label	Dense(K), sigmoid	BCE per label	'binary_crossentropy'
Imbalanced detection	sigmoid/softmax	focal loss	BinaryFocalCrossentropy

$$\text{MSE} = \frac{1}{n}\sum(y-\hat{y})^2 \qquad \text{BCE} = -\frac{1}{n}\sum[y\log\hat{y} + (1-y)\log(1-\hat{y})] \qquad \text{CCE} = -\frac{1}{n}\sum_i\sum_k y_{ik}\log\hat{y}_{ik}$$

Why cross-entropy for classification? It is the negative log-likelihood (maximum likelihood estimation), punishes confident wrong answers heavily, and with sigmoid/softmax its gradient is simply $\hat{\mathbf{y}} - \mathbf{y}$ — no saturation problem.

2.4 Gradient descent

$$\theta \leftarrow \theta - \eta\nabla_\theta\mathcal{L}(\theta)$$

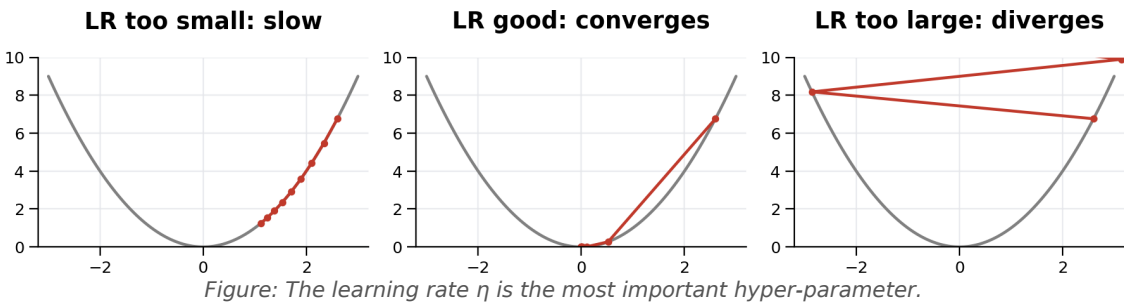


Figure: The learning rate η is the most important hyper-parameter.

Variant	Gradient on	Trade-off
Batch GD	entire training set	stable but slow, memory heavy
Stochastic GD	1 sample	very noisy, can escape shallow minima
Mini-batch GD	batch of 32–512	the standard; uses GPU parallelism; noise acts as regulariser

Epoch = one pass over the training set; **iteration/step** = one parameter update on one batch. Steps per epoch = ⌈N / batch_size⌉ — e.g. 60,000 MNIST images with batch 128 → 469 steps per epoch.

2.5 Backpropagation

Backpropagation computes ∂L/∂θ for **every** parameter efficiently by applying the **chain rule** backwards through the network, reusing intermediate results — cost ≈ 2× a forward pass. Training loop: forward pass → loss → backward pass → optimiser update.

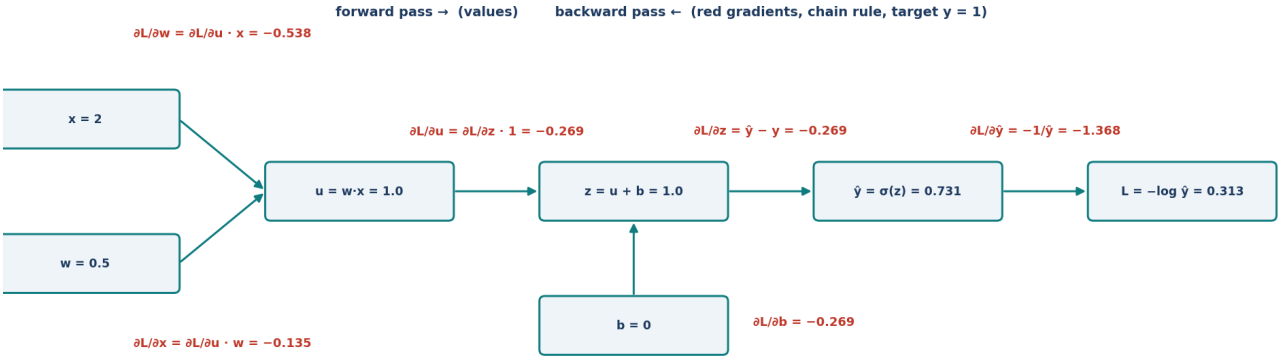


Figure: Computational graph of one sigmoid neuron with cross-entropy loss: forward values and backward gradients.

Worked example 1 — one neuron

x = 2, w = 0.5, b = 0, target y = 1, sigmoid + binary cross-entropy. Forward: z = 1.0, ŷ = σ(1) = 0.731, L = −log ŷ = 0.313. Backward: ∂L/∂z = ŷ − y = -0.269; ∂L/∂w = (ŷ − y)·x = -0.538; ∂L/∂b = -0.269. Update with η = 0.5: w = 0.5 − 0.5·(-0.538) = **0.769**, b = **0.134** → new ŷ = 0.842 (closer to 1).

Worked example 2 — a 2-2-1 network

x = [1, 0.5], W^1 = [[0.2, −0.4], [0.7, 0.1]], b^1 = 0, W^2 = [0.6, −0.3], b^2 = 0.1, sigmoid everywhere, loss L = ½(ŷ − y)², y = 1.

Step	Computation	Value
$z^1 = W^1x$	[0.2·1 − 0.4·0.5, 0.7·1 + 0.1·0.5]	[0.000, 0.750]
$a^1 = \sigma(z^1)$	sigmoid	[0.500, 0.679]
$z^2 = W^2a^1 + b^2$	0.6·0.500 − 0.3·0.679 + 0.1	0.1962
$\hat{y} = \sigma(z^2)$	sigmoid	0.5489
L	½(ŷ − 1)²	0.1017
$\delta^2 = (\hat{y}-y)\cdot\sigma'(z^2)$	(0.5489 − 1)·0.5489·0.4511	-0.1117
$\partial L/\partial W^2 = \delta^2\cdot a^1$		[-0.0558, -0.0759]
$\partial L/\partial b^2 = \delta^2$		-0.1117
$\delta^1 = (W^2\cdot\delta^2) \odot \sigma'(z^1)$	error sent back through W^2	[-0.0168, 0.0073]
$\partial L/\partial W^1 = \delta^1 x^T$	outer product	[[-0.0168, -0.0084], [0.0073, 0.0037]]

$$\delta^{[L]} = \nabla_{\hat{y}}\mathcal{L} \odot f'(z^{[L]}), \qquad \delta^{[l]} = (W^{[l+1]T}\delta^{[l+1]}) \odot f'(z^{[l]}), \qquad \frac{\partial \mathcal{L}}{\partial W^{[l]}} = \delta^{[l]}a^{[l-1]T}, \quad \frac{\partial \mathcal{L}}{\partial b^{[l]}} = \delta^{[l]}$$

the four backpropagation equations

Autograd

Frameworks record every operation in a **computational graph** during the forward pass and apply the chain rule automatically (reverse-mode automatic differentiation). You never write derivatives by hand: *loss.backward()* in PyTorch, *tf.GradientTape* in TensorFlow.

autograd.py

```
import tensorflow as tf
w = tf.Variable(0.5); b = tf.Variable(0.0); x, y = 2.0, 1.0
with tf.GradientTape() as tape:
    y_hat = tf.sigmoid(w * x + b); loss = -tf.math.log(y_hat)
print(tape.gradient(loss, [w, b]))          # [-0.538, -0.269]  — matches the hand calculation

import torch
w = torch.tensor(0.5, requires_grad=True); b = torch.tensor(0.0, requires_grad=True)
loss = -torch.log(torch.sigmoid(w * 2 + b)); loss.backward(); print(w.grad, b.grad)
```

2.6 Optimisers

Optimisers on Rosenbrock (★ minimum), 300 steps **Learning-rate schedules**

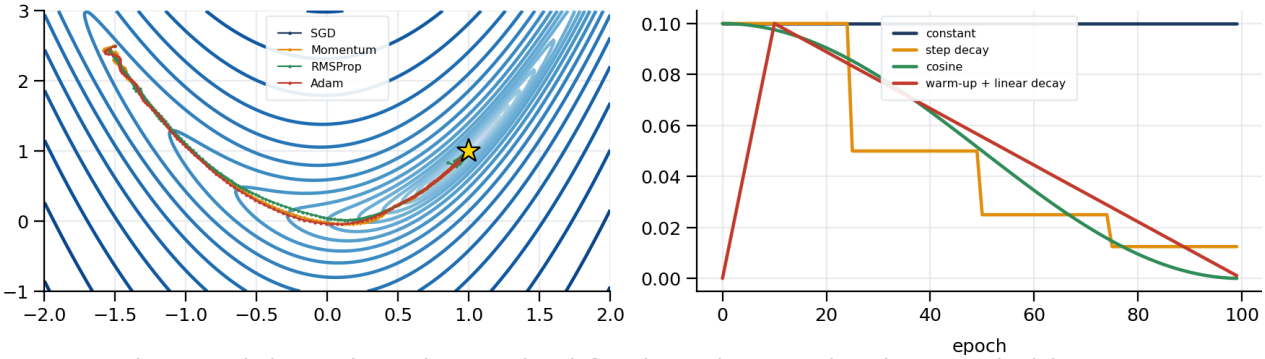


Figure: Optimiser paths on the Rosenbrock function and common learning-rate schedules.

$$\text{Momentum}: \; v \leftarrow \beta v + g, \; \theta \leftarrow \theta - \eta v \qquad \text{RMSProp}: \; s \leftarrow \beta s + (1-\beta) g^2, \; \theta \leftarrow \theta - \frac{\eta g}{\sqrt{s} + \varepsilon}$$

$$\text{Adam}: \; m \leftarrow \beta_1 m + (1-\beta_1) g, \; v \leftarrow \beta_2 v + (1-\beta_2) g^2, \; \hat{m} = \frac{m}{1-\beta_1^t}, \; \hat{v} = \frac{v}{1-\beta_2^t}, \; \theta \leftarrow \theta - \frac{\eta \hat{m}}{\sqrt{\hat{v}} + \varepsilon}$$

Optimiser	Idea	When
SGD	plain gradient step	simple; with momentum often generalises best (vision)
Momentum / Nesterov	accumulate velocity; dampens zig-zag	CNN training with schedules
AdaGrad	per-parameter LR ∝ 1/√(Σg²)	sparse features; LR decays to zero
RMSProp	moving average of g²	RNNs
Adam	momentum + RMSProp + bias correction (β₁=0.9, β₂=0.999, η=1e-3)	default first choice
AdamW	Adam with decoupled weight decay	transformers, modern default

2.7 Weight initialisation

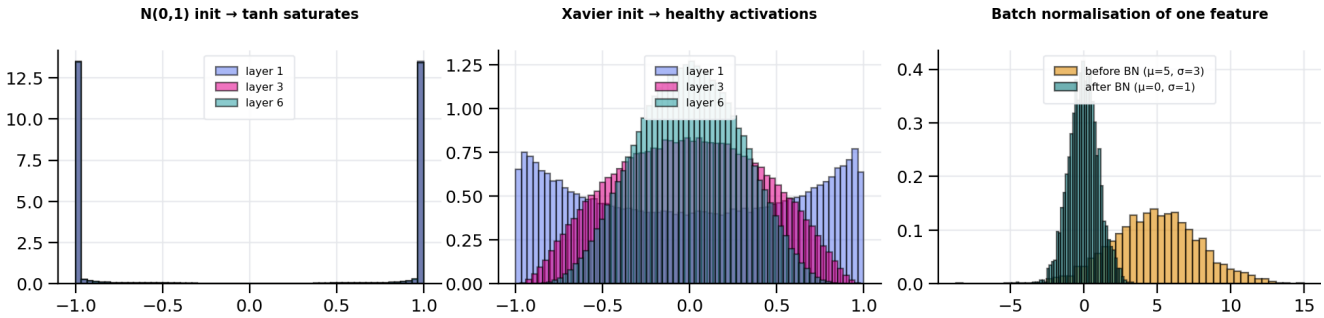


Figure: Left: N(0,1) weights make tanh units saturate at ±1. Middle: Xavier keeps activations spread. Right: batch normalisation (computed).

$$\text{Xavier/Glorot (tanh, sigmoid)}: \ \mathrm{Var}(w) = \frac{2}{n_{in} + n_{out}} \qquad \text{He (ReLU)}: \ \mathrm{Var}(w) = \frac{2}{n_{in}}$$

All-zero initialisation makes every neuron in a layer identical (symmetry is never broken). Too-large weights explode/saturate; too-small vanish. Good initialisation keeps activation and gradient variance roughly constant across layers. Keras defaults: glorot_uniform weights, zero biases; use kernel_initializer='he_normal' with ReLU.

2.8 Vanishing and exploding gradients

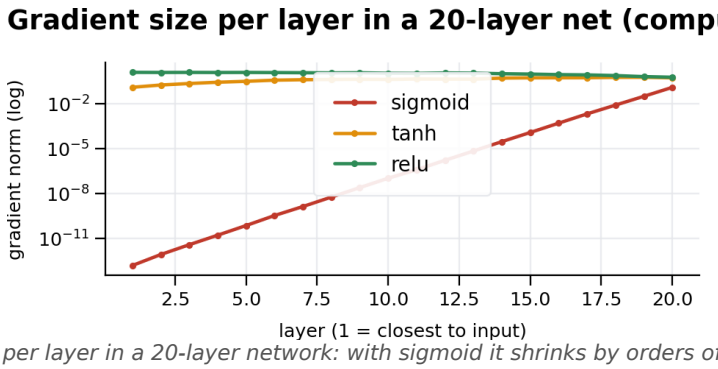


Figure: Gradient size per layer in a 20-layer network: with sigmoid it shrinks by orders of magnitude toward the input (computed with NumPy).

Backprop multiplies many factors W·f'(z). With sigmoid (f' ≤ 0.25) the product shrinks exponentially → early layers barely learn (**vanishing**). With large weights it grows → NaN losses (**exploding**). Fixes: ReLU-family activations, He/Xavier init, batch normalisation, residual connections, gradient clipping, careful learning rates, LSTM/GRU gates for sequences (Chapter 14).

2.9 Normalisation layers

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \hat{x} + \beta$$

Layer	Normalises over	Used in
Batch Norm	each feature/channel across the batch (running mean/var used at inference)	CNNs; allows higher LR, faster training, mild regularisation
Layer Norm	all features of one sample	RNNs, transformers (batch-size independent)
Instance / Group Norm	per sample per channel / groups of channels	style transfer, small batches

2.10 Regularisation

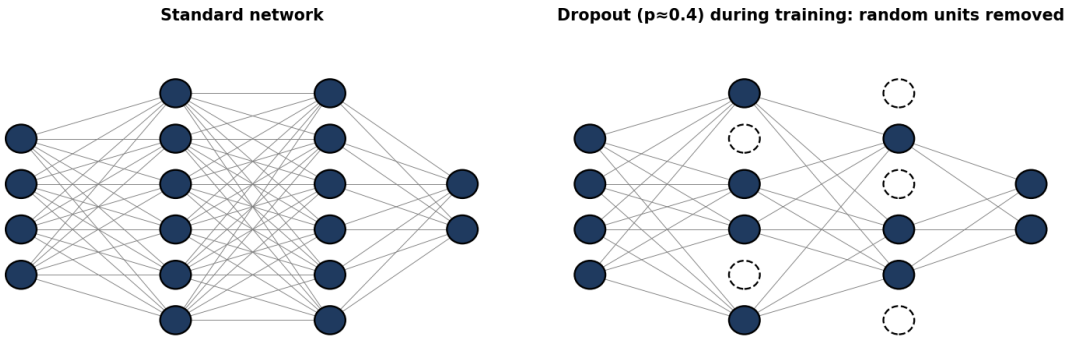


Figure: Dropout randomly removes units during training.

Technique	How it fights overfitting
L2 (weight decay)	adds λΣw² → small weights, smoother function: kernel_regularizer=l2(1e-4)
L1	adds λΣ|w| → sparse weights
Dropout	zero a fraction p of activations per step (scale by 1/(1−p)); prevents co-adaptation; like averaging many sub-networks; off at inference
Early stopping	stop when validation loss stops improving; restore best weights
Data augmentation	random flips, crops, rotations, noise — more effective data
Batch norm	noise from batch statistics
Smaller model / label smoothing / mixup	reduce capacity or over-confidence

Overfitting seen in loss curves (illustrative) **Bias–variance vs model capacity**

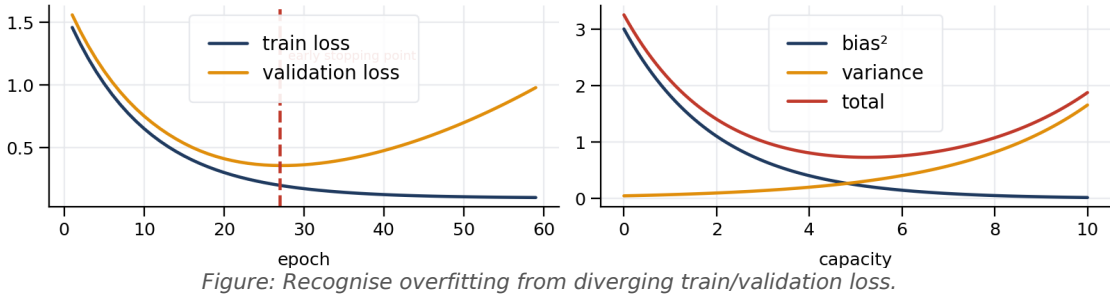


Figure: Recognise overfitting from diverging train/validation loss.

2.11 Hyper-parameters and the training recipe

Hyper-parameter	Typical starting point
learning rate	Adam 1e-3 (fine-tuning 1e-5–1e-4); tune on a log scale
batch size	32–256 (larger → scale LR up)
epochs	until early stopping triggers
hidden layers / units	2–3 layers of 64–512 for tabular/MNIST
activation	ReLU (hidden), task-specific output
dropout	0.2–0.5
weight decay	1e-5 – 1e-3

1. Overfit a tiny batch first (loss should go to ~0) — proves the code can learn.
2. Then train on full data with validation monitoring.

3. Diagnose with learning curves: high bias → bigger model/longer training; high variance → regularise/more data.
4. Tune learning rate first, then architecture, then regularisation.
5. If loss is NaN: lower LR, check data scaling, use gradient clipping, check log(0).

CHAPTER 3

Hands-on: ANN with Keras (MNIST Demo)

Dataset • preprocessing • Sequential model • model.summary & parameter counts • compile • fit with callbacks • learning curves • evaluation & confusion matrix • predictions • save/load • Functional API • PyTorch equivalent • experiments

3.1 The MNIST dataset

70,000 grayscale 28×28 handwritten digits (60,000 train, 10,000 test), 10 classes — the 'hello world' of deep learning. The figure shows similar 8×8 digits from scikit-learn's small dataset, which we also use to produce real training curves below.

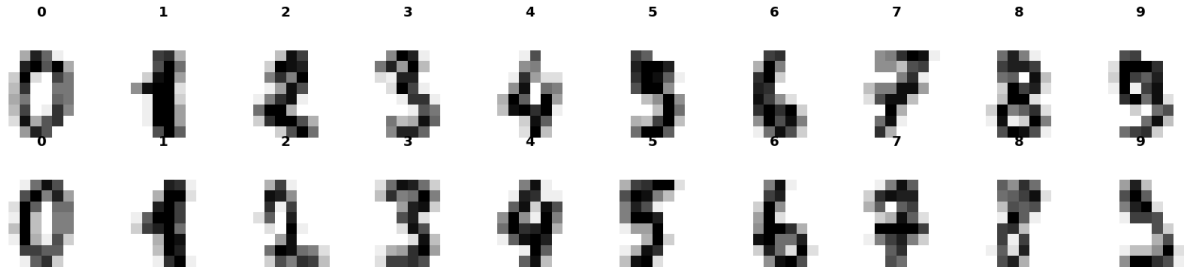


Figure: Sample handwritten digits (scikit-learn digits dataset, 8×8).

3.2 Complete Keras code

mnist_ann_keras.py

```
import numpy as np, matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 1) Load data
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
print(x_train.shape, y_train.shape)          # (60000, 28, 28) (60000,)

# 2) Preprocess: scale to [0,1] and flatten 28×28 → 784
x_train = x_train.reshape(-1, 784).astype("float32") / 255.0
x_test  = x_test.reshape(-1, 784).astype("float32") / 255.0
# labels stay integers 0..9 → use sparse_categorical_crossentropy
# (or y = keras.utils.to_categorical(y, 10) with categorical_crossentropy)

# 3) Build the ANN
model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation="relu", kernel_initializer="he_normal"),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(64, activation="relu", kernel_initializer="he_normal"),
    layers.Dropout(0.3),
    layers.Dense(10, activation="softmax"),
])
model.summary()

# 4) Compile: optimiser + loss + metrics
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])

# 5) Train with callbacks
callbacks = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True),
             keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2),
             keras.callbacks.ModelCheckpoint("best_mnist.keras", save_best_only=True)]
history = model.fit(x_train, y_train, validation_split=0.1, epochs=30, batch_size=128, callbacks=callbacks, verbose=2)

# 6) Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"test accuracy = {test_acc:.4f}")        # typically ≈ 0.98

# 7) Learning curves
plt.plot(history.history["loss"], label="train"); plt.plot(history.history["val_loss"], label="val"); plt.legend();
        plt.show()

# 8) Predict
probs = model.predict(x_test[:5]); print(probs.argmax(axis=1), y_test[:5])

# 9) Save / load
model.save("mnist_ann.keras"); reloaded = keras.models.load_model("mnist_ann.keras")
```

Reading model.summary()

Layer	Output shape	Params	How
Dense(128)	(None, 128)	100,480	784·128 + 128
BatchNormalization	(None, 128)	512	4 per unit: γ, β (trainable) + moving mean, var (non-trainable)
Dropout(0.3)	(None, 128)	0	no weights
Dense(64)	(None, 64)	8,256	128·64 + 64
Dropout	(None, 64)	0	
Dense(10)	(None, 10)	650	64·10 + 10
Total		109,898	of which 256 non-trainable

'None' is the batch dimension (any batch size).

3.3 What training looks like (real run)

To show genuine curves without downloading MNIST, the same kind of network (two hidden layers 128 → 64, ReLU, Adam) was trained with scikit-learn on the 8×8 digits dataset for 60 epochs. Final validation accuracy: **0.978**, training accuracy 0.998.

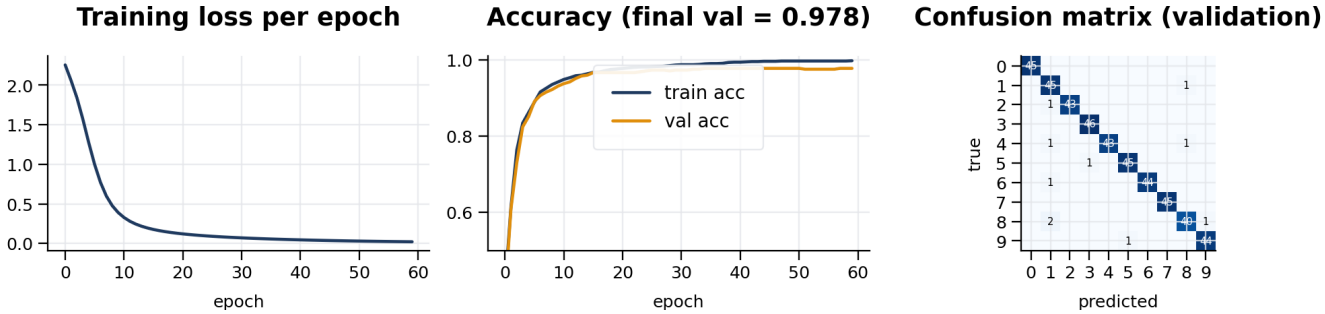


Figure: Loss and accuracy per epoch, and the validation confusion matrix (computed).

How to read the results

- Loss falls quickly then flattens → learning works; if it rises again on validation → overfitting → early stopping/regularisation.
- Train ≈ val accuracy → good generalisation; large gap → variance.
- Confusion matrix shows which digits are confused (commonly 4↔9, 3↔5, 7↔1) → guides augmentation or a CNN.
- An MLP ignores the 2-D structure of images (flattening). A CNN reaches ≈99%+ on MNIST (Chapter 6).

3.4 Functional API and subclassing

functional_api.py

```
inputs = keras.Input(shape=(784,))
x = layers.Dense(128, activation="relu")(inputs)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(10, activation="softmax")(x)
model = keras.Model(inputs, outputs, name="mnist_functional")      # supports multi-input/output, skip connections

class MyANN(keras.Model):                                           # subclassing: full control
    def __init__(self): super().__init__(); self.d1 = layers.Dense(128, activation="relu"); self.out = layers.Dense(10,
        activation="softmax")
    def call(self, x, training=False): return self.out(self.d1(x))
```

3.5 The same model in PyTorch

mnist_ann_pytorch.py

```
import torch, torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
tf_ = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))])
train_dl = DataLoader(datasets.MNIST(".", train=True, download=True, transform=tf_), batch_size=128, shuffle=True)
test_dl = DataLoader(datasets.MNIST(".", train=False, transform=tf_), batch_size=1000)

model = nn.Sequential(nn.Flatten(), nn.Linear(784, 128), nn.ReLU(), nn.Dropout(0.3), nn.Linear(128, 64), nn.ReLU(),
        nn.Linear(64, 10))
loss_fn = nn.CrossEntropyLoss()                 # = log-softmax + NLL → feed raw logits, no softmax layer
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
device = "cuda" if torch.cuda.is_available() else "cpu"; model.to(device)
for epoch in range(10):
    model.train()
    for xb, yb in train_dl:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad(); loss = loss_fn(model(xb), yb); loss.backward(); opt.step()   # forward → backward → update
    model.eval(); correct = 0
    with torch.no_grad():
        for xb, yb in test_dl: correct += (model(xb.to(device)).argmax(1) == yb.to(device)).sum().item()
    print(epoch, correct / 10000)
```

3.6 Experiments to try (and what you should observe)

Change	Expected effect
remove scaling (/255)	slower or unstable training
sigmoid instead of ReLU in 5 hidden layers	much slower learning (vanishing gradients)
LR 1.0 with Adam	loss explodes / accuracy ~10%
no dropout, 1024-unit layers, 100 epochs	train acc → 100%, val stalls/drops (overfitting)
batch size 1 vs 1024	noisy & slow per epoch vs smooth but may generalise worse
SGD without momentum	slower convergence than Adam

Part II — Convolutional Neural Networks

Convolution theory, classic & advanced architectures, transfer learning, object detection, segmentation, visualisation and an end-to-end computer-vision project

CHAPTER 4

Convolutional Neural Networks — Theory

Why not dense layers for images • images as tensors • convolution operation • filters/kernels • stride • padding • output-size formula • channels & parameter counts • activation maps • pooling • receptive field • 1×1 convolutions • feature hierarchy • full CNN pipeline • dimension walkthrough

4.1 Why CNNs?

A 224×224 RGB image has 150,528 inputs. One dense layer of 1,000 neurons would need **150,528,000** weights — huge, slow, overfit-prone, and it ignores that nearby pixels are related and that a cat is a cat wherever it appears. CNNs exploit three ideas:

Idea	Meaning	Benefit
Local connectivity	each neuron looks only at a small patch (its receptive field)	captures local patterns (edges, corners)
Weight sharing	the same filter slides over the whole image	huge parameter reduction; translation **equivariance** (shifted input → shifted feature map)
Pooling / downsampling	summarise neighbourhoods	some translation **invariance**, smaller maps, larger receptive fields

4.2 The convolution operation

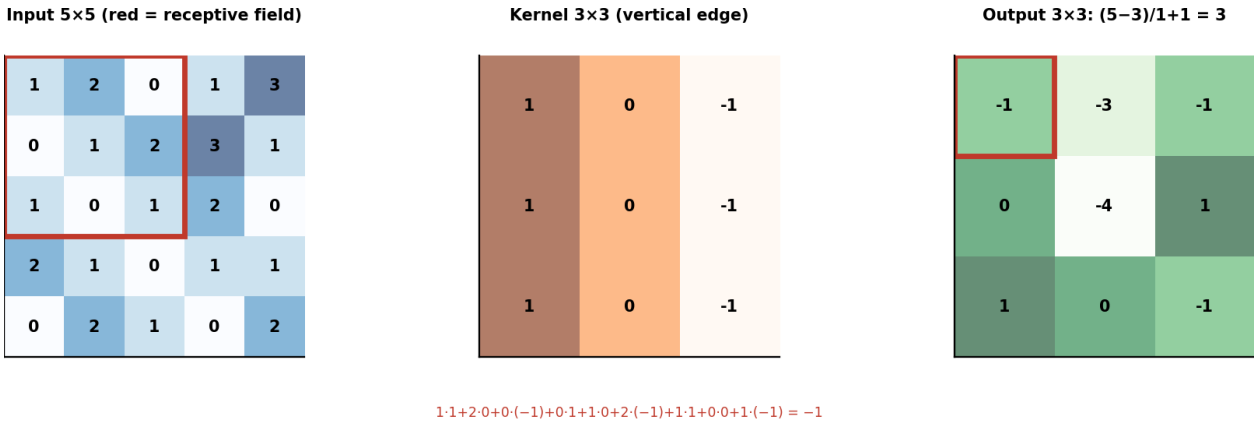


Figure: A 3×3 vertical-edge kernel sliding over a 5×5 input with stride 1 and no padding.

$$S(i,j) = (I * K)(i,j) = \sum_{m=0}^{k-1}\sum_{n=0}^{k-1} I(i+m, j+n)\,K(m,n) + b$$

technically cross-correlation (no kernel flip) — what all DL libraries call 'convolution'

At every position, multiply the kernel element-wise with the patch under it and sum → one number of the output (**feature map / activation map**). A large output means 'this pattern is present here'. The kernel values are **learned** by backprop — the network discovers its own edge, texture

and shape detectors.

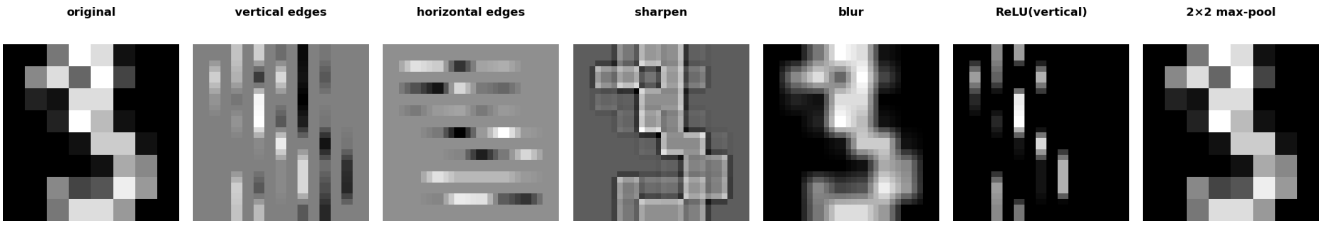


Figure: Hand-made kernels applied to a digit, plus ReLU and max-pooling (computed with SciPy).

4.3 Stride, padding and output size

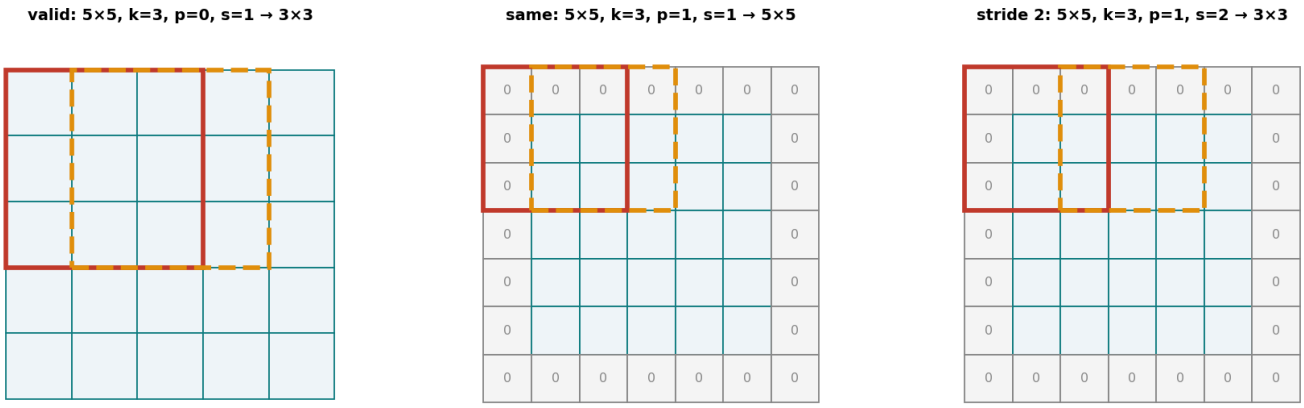


Figure: Valid vs same padding and stride 2 (red/orange = first two kernel positions).

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

W = input size, K = kernel size, P = padding, S = stride

Example	Calculation	Output
32×32, K=5, P=0, S=1	(32 − 5 + 0)/1 + 1	28×28
32×32, K=3, P=1, S=1 ('same')	(32 − 3 + 2)/1 + 1	32×32
224×224, K=7, P=3, S=2 (ResNet stem)	⌊(224 − 7 + 6)/2⌋ + 1	112×112
227×227, K=11, P=0, S=4 (AlexNet conv1)	(227 − 11)/4 + 1	55×55
55×55 maxpool K=3, S=2	⌊(55 − 3)/2⌋ + 1	27×27

- **Padding** ('same') adds zeros around the border so output size = input size and edge pixels are used as often as centre pixels. 'valid' = no padding (output shrinks).
- **Stride** = step of the sliding window; stride 2 halves the spatial size (alternative to pooling).
- **Dilation** spaces out kernel taps → larger receptive field without more parameters (DeepLab, WaveNet).

4.4 Channels, filters and parameter counts

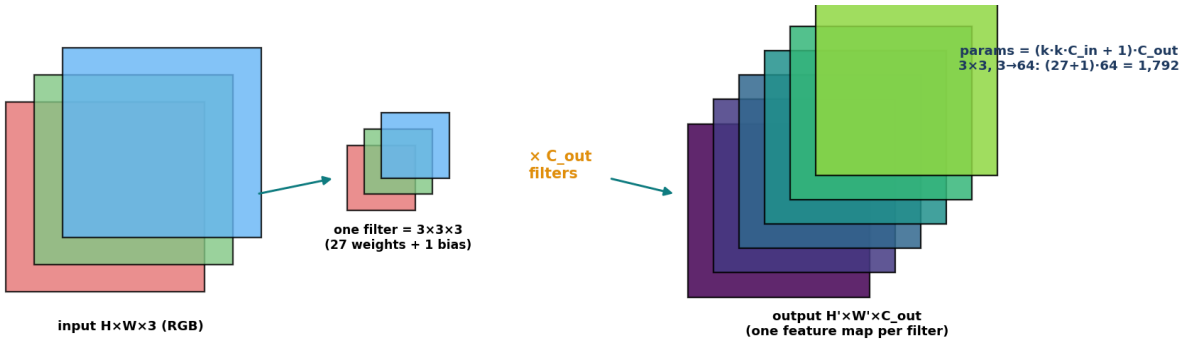


Figure: Each filter spans ALL input channels; the number of filters = number of output channels.

$$\text{params} = (K \cdot K \cdot C_{in} + 1) \cdot C_{out} \qquad \text{output shape} = O \times O \times C_{out}$$

Example: Conv2D(64, 3×3) on an RGB image → (3·3·3 + 1)·64 = **1,792** params; next Conv2D(128, 3×3) on 64 channels → (3·3·64 + 1)·128 = **73,856**. Compare a dense layer on 224×224×3 → 1000 units: 150 million.

4.5 Pooling

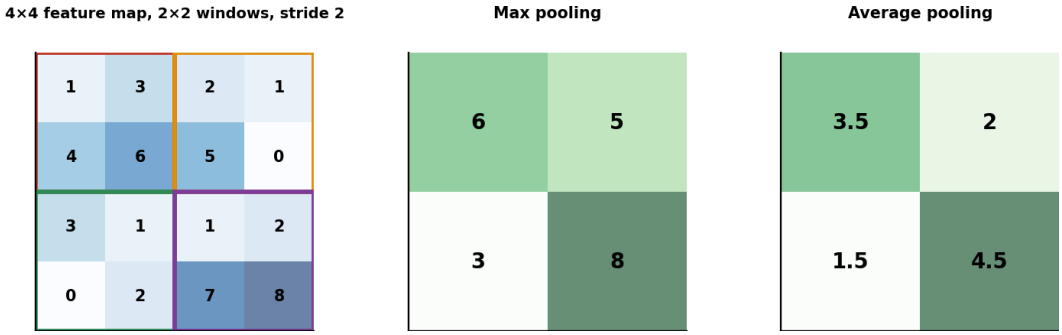


Figure: Max vs average pooling (2×2, stride 2).

Type	What	Use
Max pooling	keeps the strongest activation in each window	most common between conv blocks
Average pooling	averages the window	smoother; LeNet
Global Average Pooling (GAP)	averages each whole feature map to one number → vector of C values	replaces Flatten + big Dense layers (fewer params, less overfitting) — ResNet, GoogLeNet
Strided convolution	conv with S=2	learnable downsampling (modern nets)

Pooling has **no learnable parameters**. Backprop through max pooling sends the gradient only to the position that was the max.

4.6 Receptive field and feature hierarchy

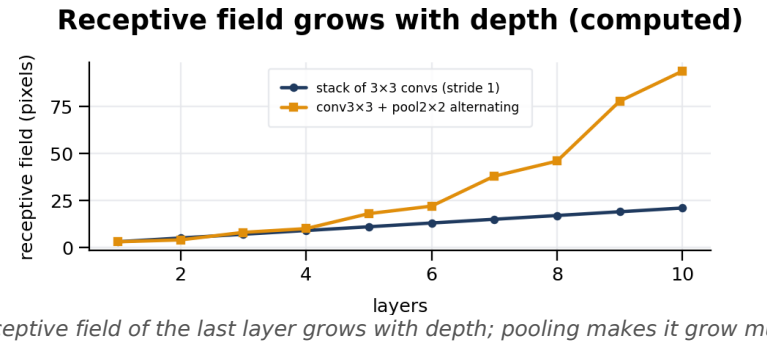


Figure: Receptive field of the last layer grows with depth; pooling makes it grow much faster.

Two stacked 3×3 convs see a 5×5 region; three see 7×7 — with fewer parameters ($3 \cdot 9C^2$ vs $49C^2$) and more non-linearities than one 7×7 conv (the VGG insight). Deep layers therefore 'see' large regions and learn high-level concepts: **layer 1** edges and colours → **layer 2–3** textures, corners → **deeper** parts (eyes, wheels) → **last** whole objects.

4.7 1×1 convolutions

A 1×1 conv is a per-pixel dense layer across channels: it changes depth cheaply (e.g. 256 → 64 channels), adds non-linearity and mixes channel information. Used for bottlenecks in Inception and ResNet and for pointwise convs in MobileNet.

4.8 A complete CNN

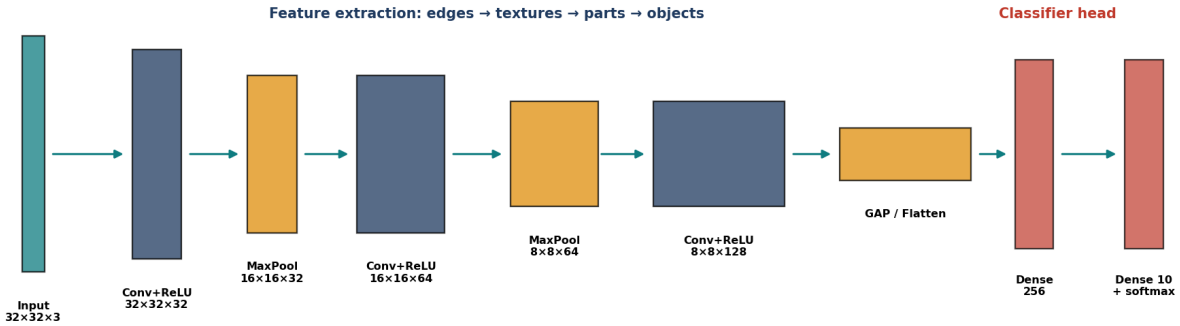


Figure: Typical CNN: [Conv → ReLU → (BN) → Pool] × N → Flatten/GAP → Dense → Softmax.

Layer	Output shape	Params
Input 32×32×3	32×32×3	0
Conv2D(32, 3×3, same) + ReLU	32×32×32	896
MaxPool 2×2	16×16×32	0
Conv2D(64, 3×3, same) + ReLU	16×16×64	18,496
MaxPool 2×2	8×8×64	0
Conv2D(128, 3×3, same) + ReLU	8×8×128	73,856
GlobalAveragePooling2D	128	0
Dense(10, softmax)	10	1,290
Total		94,538

Interview tip: Interview favourite: compute output shapes and parameter counts by hand with $O = (W - K + 2P)/S + 1$ and $(K \cdot K \cdot C_{in} + 1) \cdot C_{out}$. Pooling, Flatten, Dropout have 0 params; BatchNorm has 4 per channel (2 trainable).

CHAPTER 5

Classic CNN Architectures — LeNet, AlexNet, VGG (+ ZFNet)

LeNet-5 layer by layer • AlexNet innovations • ZFNet • VGG-16/19 and the 3×3 principle • parameter analysis • ImageNet history • comparison

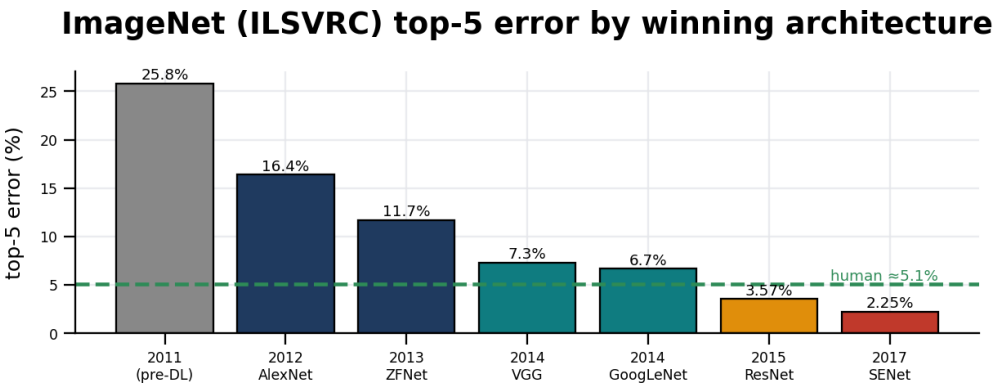


Figure: Winning ImageNet top-5 error by year (published results).

5.1 LeNet-5 (LeCun et al., 1998)

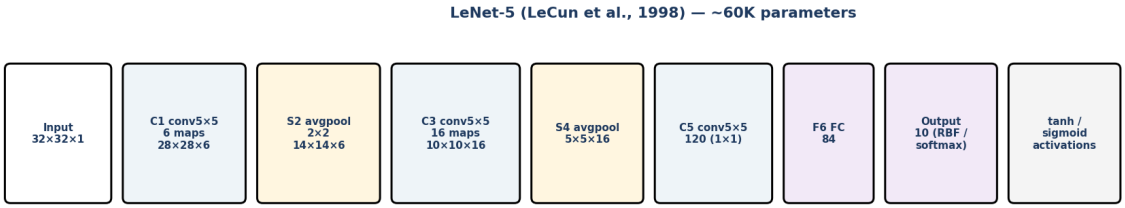


Figure: LeNet-5 architecture.

Layer	Config	Output	Params
Input	grayscale digit	32×32×1	0
C1	6 filters 5×5	28×28×6	156
S2	avg-pool 2×2 s2 (trainable coeff. in original)	14×14×6	12 (orig.)
C3	16 filters 5×5 (sparse connections in original)	10×10×16	2,416 (full)
S4	avg-pool 2×2	5×5×16	32 (orig.)
C5	120 filters 5×5	1×1×120	48,120
F6	dense 84, tanh	84	10,164
Output	10 (RBF in original; softmax today)	10	850

≈60K parameters. Read bank cheques in the 1990s. Introduced the conv → pool → conv → pool → FC pattern still used today; used tanh/sigmoid and average pooling.

5.2 AlexNet (Krizhevsky, Sutskever, Hinton, 2012)

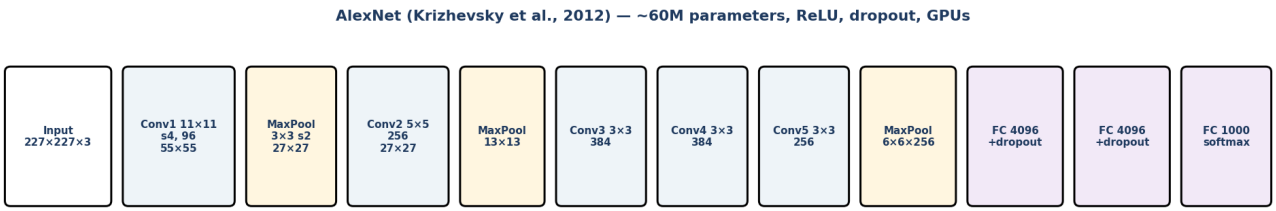


Figure: AlexNet: 5 conv + 3 FC layers.

Innovation	Why it mattered
ReLU activations	~6× faster training than tanh; no saturation
Training on 2 GPUs (GTX 580, 3 GB)	made a 60M-parameter model feasible
Dropout (0.5) in FC layers	controlled overfitting
Data augmentation	random 224/227 crops, flips, PCA colour jitter
Overlapping max pooling (3×3, s2)	slightly less overfitting
Local Response Normalisation	later abandoned (BatchNorm replaced it)
Large first filters 11×11 stride 4	fast downsampling of 227×227 input

Result: 16.4% top-5 error vs 25.8% for the previous best → the start of the deep-learning era. Most parameters (~58M of ~60M) are in the FC layers — a lesson later architectures fixed with GAP.

5.3 ZFNet (Zeiler & Fergus, 2013)

Same family as AlexNet but tuned using **deconvolution visualisations** of what each layer learns: smaller first filters (7×7, stride 2) kept more detail. 11.7% top-5 error. Its main legacy is CNN visualisation (Chapter 11).

5.4 VGG-16 / VGG-19 (Simonyan & Zisserman, 2014)

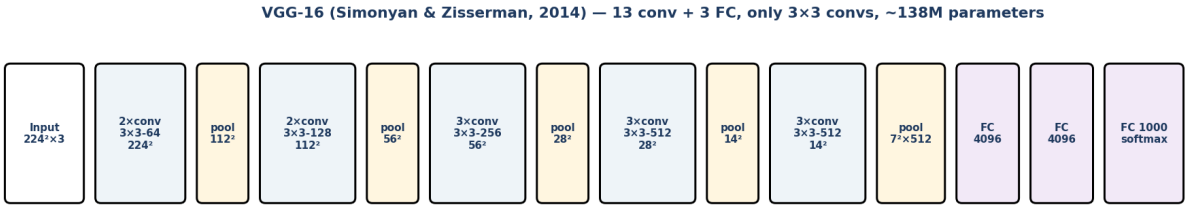


Figure: VGG-16: five blocks of 3×3 convs separated by max-pooling, then three FC layers.

Key idea: use only small 3×3 convs (stride 1, same padding) and 2×2 max-pools, doubling channels after each pool (64 → 128 → 256 → 512 → 512). Simple, uniform, deep. Computed parameter count: conv layers **14,714,688** + FC layers **123,642,856** = **138,357,544** (~138M); ~90% are in the FC layers. Top-5 error 7.3%.

Pros	Cons
simple, uniform design; excellent feature extractor; still popular for transfer learning and perceptual loss (style transfer)	138M params (~528 MB), slow; 15.5 GFLOPs per image; superseded by ResNet/EfficientNet

5.5 Comparison

Model	Year	Depth	Params	Top-5 err.	Key contribution
LeNet-5	1998	5	~60K	— (MNIST)	conv-pool pattern
AlexNet	2012	8	~60M	16.4%	ReLU, dropout, GPUs
ZFNet	2013	8	~60M	11.7%	visualisation-driven tuning
VGG-16	2014	16	138M	7.3%	stacked 3×3 convs, depth
GoogLeNet	2014	22	~6.8M	6.7%	Inception module (Ch. 7)
ResNet-152	2015	152	~60M	3.57%	residual connections (Ch. 7)

CHAPTER 6

Hands-on: Image Classification with CNN + Weekly Assessment 02

CNN on MNIST • CIFAR-10 pipeline with tf.data • augmentation • BatchNorm & dropout • callbacks • evaluation • common mistakes • assessment questions

6.1 CNN for MNIST (≈99% accuracy)

cnn_mnist.py

```
from tensorflow import keras
from tensorflow.keras import layers
(x_tr, y_tr), (x_te, y_te) = keras.datasets.mnist.load_data()
x_tr = x_tr[..., None].astype("float32") / 255; x_te = x_te[..., None].astype("float32") / 255     # (N, 28, 28, 1)
model = keras.Sequential([
    layers.Input((28, 28, 1)),
    layers.Conv2D(32, 3, activation="relu", padding="same"), layers.MaxPooling2D(),      # 14×14×32
    layers.Conv2D(64, 3, activation="relu", padding="same"), layers.MaxPooling2D(),      # 7×7×64
    layers.Flatten(), layers.Dropout(0.4), layers.Dense(128, activation="relu"), layers.Dense(10, activation="softmax")])
model.compile("adam", "sparse_categorical_crossentropy", ["accuracy"])
model.fit(x_tr, y_tr, validation_split=0.1, epochs=10, batch_size=128)
model.evaluate(x_te, y_te)                        # ≈ 0.99
```

6.2 CIFAR-10 project (32×32 colour images, 10 classes)

cifar10_cnn.py

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
(x_tr, y_tr), (x_te, y_te) = keras.datasets.cifar10.load_data()
classes = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck"]
x_tr, x_te = x_tr.astype("float32") / 255, x_te.astype("float32") / 255

augment = keras.Sequential([layers.RandomFlip("horizontal"), layers.RandomRotation(0.1), layers.RandomZoom(0.1),
        layers.RandomTranslation(0.1, 0.1)])
def block(x, f):
    x = layers.Conv2D(f, 3, padding="same", use_bias=False)(x); x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
    x = layers.Conv2D(f, 3, padding="same", use_bias=False)(x); x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
    return layers.Dropout(0.25)(layers.MaxPooling2D()(x))
inp = keras.Input((32, 32, 3)); x = augment(inp)            # augmentation active only during training
for f in (32, 64, 128): x = block(x, f)
x = layers.GlobalAveragePooling2D()(x); x = layers.Dense(128, activation="relu")(x); x = layers.Dropout(0.5)(x)
out = layers.Dense(10, activation="softmax")(x); model = keras.Model(inp, out)

model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="sparse_categorical_crossentropy", metrics=["accuracy"])
cbs = [keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True), keras.callbacks.ReduceLROnPlateau(factor=0.5,
        patience=3)]
train_ds = tf.data.Dataset.from_tensor_slices((x_tr, y_tr)).shuffle(10000).batch(128).prefetch(tf.data.AUTOTUNE)
hist = model.fit(train_ds, validation_data=(x_te, y_te), epochs=60, callbacks=cbs)
model.evaluate(x_te, y_te)                       # typically ~85–90% for this size of model

import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
pred = model.predict(x_te).argmax(1); print(classification_report(y_te.ravel(), pred, target_names=classes))
```

(Use a separate validation split in real projects; the test set is used for validation above only to keep the example short.)

Loading your own images

own_images.py

```
train_ds = keras.utils.image_dataset_from_directory("data/train", image_size=(224, 224), batch_size=32, label_mode="int")
        # one folder per class
val_ds = keras.utils.image_dataset_from_directory("data/val", image_size=(224, 224), batch_size=32)
train_ds = train_ds.map(lambda x, y: (x / 255.0, y)).cache().prefetch(tf.data.AUTOTUNE)
```

6.3 Common mistakes

Mistake	Symptom	Fix
forgot to normalise pixels	slow / stuck training	/255 or Rescaling layer
wrong loss for label format	shape errors or bad accuracy	integer labels → sparse CE; one-hot → categorical CE
softmax + from_logits=True (or no softmax + from_logits=False)	poor results	match the output activation with the loss setting
augmentation on validation/test	noisy evaluation	augment only training data
too-large LR	accuracy ≈ 10% (random)	lower LR
channel order mismatch (NCHW vs NHWC)	shape errors	Keras uses (H, W, C) by default

6.4 Weekly Assessment 02 (with answers)

Q1. A 28×28×1 input passes Conv2D(16, 5×5, valid, stride 1). Output shape and params?

24×24×16; params = (5·5·1 + 1)·16 = 416.

Q2. Output size of 64×64 input with 3×3 kernel, padding 1, stride 2?

$\lfloor (64 - 3 + 2)/2 \rfloor + 1 = 32 \rightarrow 32 \times 32$.

Q3. How many parameters does a 2×2 MaxPooling layer have?

0.

Q4. Why do CNNs need far fewer parameters than dense networks on images?

Local connectivity and weight sharing — the same small kernel is reused at every position.

Q5. What does 'same' padding do?

Pads zeros so the output spatial size equals the input size (for stride 1).

Q6. Name two innovations of AlexNet.

ReLU activations and dropout (also GPU training, data augmentation).

Q7. Why does VGG use stacks of 3×3 convs instead of one 7×7?

Same receptive field (7×7 with three layers) with fewer parameters ($27C^2$ vs $49C^2$) and more non-linearities.

Q8. Where are most of VGG-16's parameters?

In the fully connected layers (~124M of ~138M).

Q9. What does Global Average Pooling output for a 7×7×512 tensor?

A 512-dimensional vector (mean of each feature map).

Q10. Translation equivariance vs invariance?

Equivariance: shifting the input shifts the feature map (convolution). Invariance: the output doesn't change with small shifts (pooling, GAP).

Q11. Params of Conv2D(128, 3×3) on 64 channels?

(3·3·64 + 1)·128 = 73,856.

Q12. Which layer type introduced in LeNet is still used today?

Convolution + pooling stacks followed by fully connected layers.

Q13. Your CNN has 99% train and 70% validation accuracy. Two fixes?

Data augmentation and dropout/weight decay (also more data, smaller model, early stopping, transfer learning).

Q14. What is a feature map?

The output of one filter applied across the input — a 2-D map of where that pattern occurs.

Q15. Loss for 10-class classification with integer labels in Keras?

sparse_categorical_crossentropy with a softmax output.

CHAPTER 7

Advanced CNN Architectures

GoogLeNet / Inception • ResNet & residual learning • bottleneck blocks • DenseNet • MobileNet & depthwise separable convolutions • SENet (attention) • EfficientNet compound scaling • Vision Transformers & ConvNeXt • choosing an architecture

7.1 GoogLeNet / Inception (Szegedy et al., 2014)

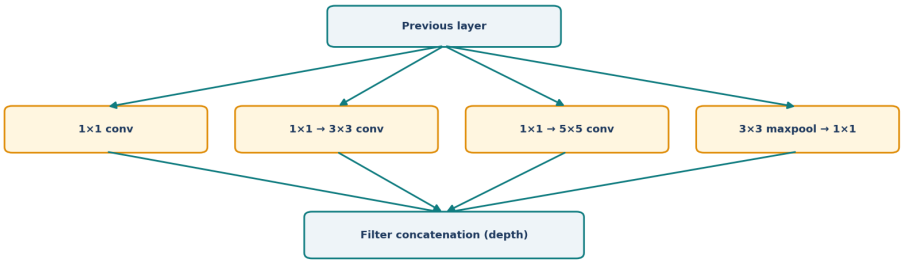


Figure: Inception module: parallel multi-scale branches concatenated along depth.

Instead of choosing one kernel size, the Inception module applies 1×1, 3×3, 5×5 convs and pooling **in parallel** and concatenates the results — the network learns which scale matters. 1×1 convs before the expensive 3×3/5×5 reduce channels (bottleneck). 22 layers but only ~6.8M parameters (GAP instead of big FC layers); auxiliary classifiers at intermediate layers helped gradients. Later versions (Inception v2–v4) added BatchNorm and factorised convolutions (5×5 → two 3×3; 3×3 → 1×3 + 3×1).

7.2 ResNet (He et al., 2015)

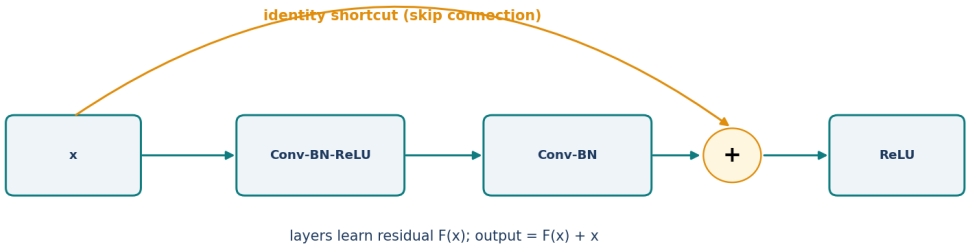


Figure: A residual block: the stacked layers learn F(x); the shortcut adds x back.

Problem — degradation: a 56-layer plain network had *higher training error* than a 20-layer one. This is not overfitting; deep plain networks are hard to optimise. **Solution**: let layers learn a residual F(x) = H(x) − x and add the input back:

$$\mathbf{y} = F(\mathbf{x}, \{W_i\}) + \mathbf{x} \qquad \Rightarrow \qquad \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial F}{\partial \mathbf{x}} + I$$

- If the best mapping is identity, the block only needs F = 0 — easy. Extra layers can never hurt much.
- The +I term gives gradients a direct highway to early layers → no vanishing gradient → 50, 101, 152 (even 1000+) layers trainable.

- When shapes differ, the shortcut uses a 1×1 conv with stride 2 (projection shortcut).
- **Bottleneck block** (ResNet-50+): 1×1 (reduce, e.g. 256→64) → 3×3 (64) → 1×1 (expand to 256) — cheaper than two 3×3 at 256 channels.
- ResNet-50: ~25.6M params, 3.57% top-5 error ensemble (ResNet-152) — won ImageNet 2015. Residual connections now appear in transformers, U-Nets, diffusion models.

residual_block.py

```
def residual_block(x, filters, stride=1):
    shortcut = x
    y = layers.Conv2D(filters, 3, strides=stride, padding="same", use_bias=False)(x); y = layers.BatchNormalization()(y); y
        = layers.ReLU()(y)
    y = layers.Conv2D(filters, 3, padding="same", use_bias=False)(y); y = layers.BatchNormalization()(y)
    if stride != 1 or shortcut.shape[-1] != filters:                    # projection shortcut
        shortcut = layers.BatchNormalization()(layers.Conv2D(filters, 1, strides=stride, use_bias=False)(shortcut))
    return layers.ReLU()(layers.Add()([y, shortcut]))
```

7.3 DenseNet (Huang et al., 2017)

Each layer receives the **concatenation** of all previous feature maps in its block: $x_l = H_l([x_0, x_1, \ldots, x_{l-1}])$. Strong feature reuse and gradient flow with few parameters (DenseNet-121 ≈ 8M). 'Growth rate' k = channels each layer adds; transition layers (1×1 conv + pooling) control size. Memory-hungry because of concatenations.

7.4 MobileNet (Howard et al., 2017) — efficient CNNs

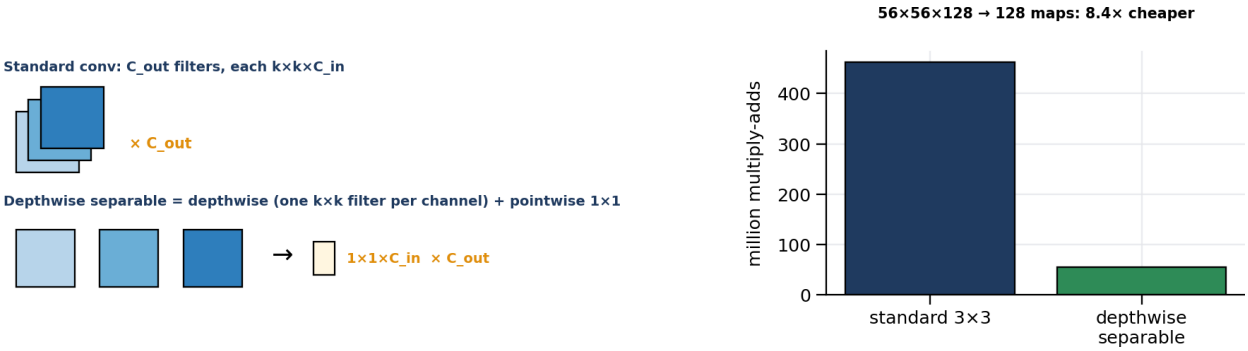


Figure: Depthwise separable convolution and its cost saving (computed for a 56×56×128 layer).

$$\frac{\text{cost}_{\text{separable}}}{\text{cost}_{\text{standard}}} = \frac{K^2 C_{in} + C_{in} C_{out}}{K^2 C_{in} C_{out}} = \frac{1}{C_{out}} + \frac{1}{K^2} \approx \frac{1}{9} \quad (K = 3)$$

MobileNetV1: depthwise separable convs + width/resolution multipliers. MobileNetV2: **inverted residuals** (expand with 1×1 → depthwise 3×3 → project with linear 1×1 bottleneck) with skip connections. MobileNetV3: neural architecture search + squeeze-excitation + hard-swish. Use for phones, edge devices, real-time apps (TFLite).

7.5 SENet — channel attention (Hu et al., 2017)

Squeeze-and-Excitation block: squeeze each channel to one number with global average pooling → two small FC layers (excitation) → sigmoid weights per channel → rescale the feature maps. The network learns *which channels matter* for each image. Won ImageNet 2017 (2.25% top-5); SE blocks are reused in MobileNetV3 and EfficientNet.

7.6 EfficientNet (Tan & Le, 2019)

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi}, \qquad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \quad \alpha, \beta, \gamma \geq 1$$

Scaling only depth, only width or only resolution saturates. **Compound scaling** grows all three together with fixed ratios found by grid search (α=1.2, β=1.1, γ=1.15); φ controls total compute (each +1 ≈ 2× FLOPs). Base network EfficientNet-B0 (MBConv blocks + SE + Swish, found by NAS) is scaled to B1…B7. Much better accuracy per parameter than ResNet. EfficientNetV2 adds Fused-MBConv and progressive learning for faster training.

7.7 Beyond CNNs: Vision Transformers and ConvNeXt

- **ViT (2020)**: split the image into 16×16 patches → linear embeddings + position embeddings → transformer encoder → [CLS] token for classification. Needs very large pre-training data (weaker inductive bias than CNNs) but scales extremely well. Variants: DeiT (data-efficient), Swin (shifted windows, hierarchical).
- **ConvNeXt (2022)**: a ResNet modernised with transformer-era design (large 7×7 depthwise kernels, LayerNorm, GELU, inverted bottlenecks) — matches Swin transformers, showing CNNs remain competitive.
- **Hybrid / foundation models**: CLIP image encoders, DINOv2, SAM backbones — usually used via transfer learning.

7.8 Comparison and choosing

Architecture	Params (approx.)	Key idea	Choose when
VGG-16	138M	deep stacks of 3×3	teaching, perceptual features
Inception v3	~24M	multi-scale modules	good accuracy/cost
ResNet-50	~25.6M	residual learning	default strong backbone, detection/segmentation
DenseNet-121	~8M	dense concatenation	small data, feature reuse
MobileNetV2/V3	~3.5M / ~5.4M	depthwise separable, inverted residuals	mobile / edge / real time
EfficientNet-B0…B7	5.3M…66M	compound scaling	best accuracy per FLOP
ViT-B/16	~86M	patches + transformer	large data / strong pre-training
ConvNeXt-T	~28M	modernised CNN	modern CNN backbone

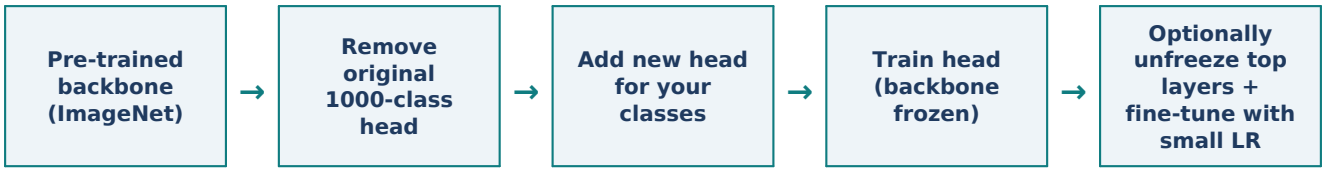
CHAPTER 8

Transfer Learning with Pre-trained Models

What and why • what transfers • feature extraction vs fine-tuning • decision matrix • Keras implementation • PyTorch implementation • preprocessing • BatchNorm caveat • discriminative learning rates • model zoos

8.1 Concept

Transfer learning reuses a model trained on a large source task (e.g. ImageNet, 1.2M images) as the starting point for your target task (e.g. 2,000 X-ray images). Early layers learn generic features (edges, textures) useful for almost any images; later layers are more task-specific. You get high accuracy with little data and compute.



8.2 Strategies

Your data	Similar to ImageNet	Different domain (medical, satellite, documents)
Small	**Feature extraction**: freeze backbone, train only the head	freeze early layers, fine-tune a few later layers; heavy augmentation
Large	**Fine-tune** most/all layers with a small LR	fine-tune all layers or train from scratch with pre-trained init

8.3 Keras implementation

transfer_keras.py

```
from tensorflow import keras
from tensorflow.keras import layers
IMG = (224, 224)
train_ds = keras.utils.image_dataset_from_directory("data/train", image_size=IMG, batch_size=32)
val_ds = keras.utils.image_dataset_from_directory("data/val", image_size=IMG, batch_size=32)
n_classes = len(train_ds.class_names)

base = keras.applications.EfficientNetB0(include_top=False, weights="imagenet", input_shape=IMG + (3,))
base.trainable = False                                                  # PHASE 1: feature extraction

inputs = keras.Input(IMG + (3,))
x = layers.RandomFlip("horizontal")(inputs); x = layers.RandomRotation(0.1)(x)
x = keras.applications.efficientnet.preprocess_input(x)                 # each model has its own preprocessing!
x = base(x, training=False)                                              # keep BatchNorm layers in inference mode
x = layers.GlobalAveragePooling2D()(x); x = layers.Dropout(0.3)(x)
outputs = layers.Dense(n_classes, activation="softmax")(x)
model = keras.Model(inputs, outputs)
model.compile(keras.optimizers.Adam(1e-3), "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10)

base.trainable = True                                                    # PHASE 2: fine-tuning
for layer in base.layers[:-40]: layer.trainable = False                 # unfreeze only the top ~40 layers
model.compile(keras.optimizers.Adam(1e-5), "sparse_categorical_crossentropy", ["accuracy"])   # recompile, LOW LR
model.fit(train_ds, validation_data=val_ds, epochs=10, callbacks=[keras.callbacks.EarlyStopping(patience=3,
        restore_best_weights=True)])
```

8.4 PyTorch implementation

transfer_pytorch.py

```
import torch, torch.nn as nn
from torchvision import models
weights = models.ResNet50_Weights.IMAGENET1K_V2
model = models.resnet50(weights=weights)
preprocess = weights.transforms()                       # correct resize/crop/normalise for this checkpoint
for p in model.parameters(): p.requires_grad = False   # freeze backbone
model.fc = nn.Linear(model.fc.in_features, n_classes)  # new head (trainable)
opt = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
# fine-tuning: unfreeze layer4 and use smaller LR for backbone (discriminative learning rates)
for p in model.layer4.parameters(): p.requires_grad = True
opt = torch.optim.AdamW([{"params": model.layer4.parameters(), "lr": 1e-5}, {"params": model.fc.parameters(), "lr":
        1e-4}])
```

8.5 Best practices

- Use the **same preprocessing** as the pre-trained model (input size, scaling, mean/std).
- Train the new head first; fine-tuning with a randomly initialised head destroys pre-trained features (large gradients).
- Fine-tune with a **10–100× smaller learning rate**; recompile after changing trainable flags (Keras).
- Keep BatchNorm layers in inference mode (training=False) when the dataset is small.
- Use augmentation and early stopping; monitor validation.
- Model zoos: keras.applications, torchvision.models, timm (hundreds of models), Hugging Face Hub (ViT, ConvNeXt, CLIP, DINOv2).
- Same idea everywhere: BERT for NLP, Whisper for speech, foundation models for everything — 'pre-train then adapt' is the core recipe of modern AI.

CHAPTER 9

Object Detection — R-CNN family, YOLO, SSD

Classification vs localisation vs detection • bounding boxes • IoU • NMS • anchors • precision/recall & mAP • R-CNN • Fast R-CNN • Faster R-CNN & RPN • FPN • YOLO (grid, output tensor, loss, versions) • SSD • RetinaNet & focal loss • DETR • hands-on • choosing a detector

9.1 The task

Task	Output
Classification	one label per image
Classification + localisation	one label + one box
Object detection	every object: class + confidence + box (x, y, w, h)
Instance segmentation	every object: class + pixel mask (Chapter 10)

Box formats: (x_min, y_min, x_max, y_max) — Pascal VOC; (x, y, w, h) top-left — COCO; (x_c, y_c, w, h) normalised to [0,1] — YOLO. Networks usually predict **offsets** relative to an anchor or grid cell rather than raw coordinates.

9.2 Core concepts: IoU, NMS, anchors

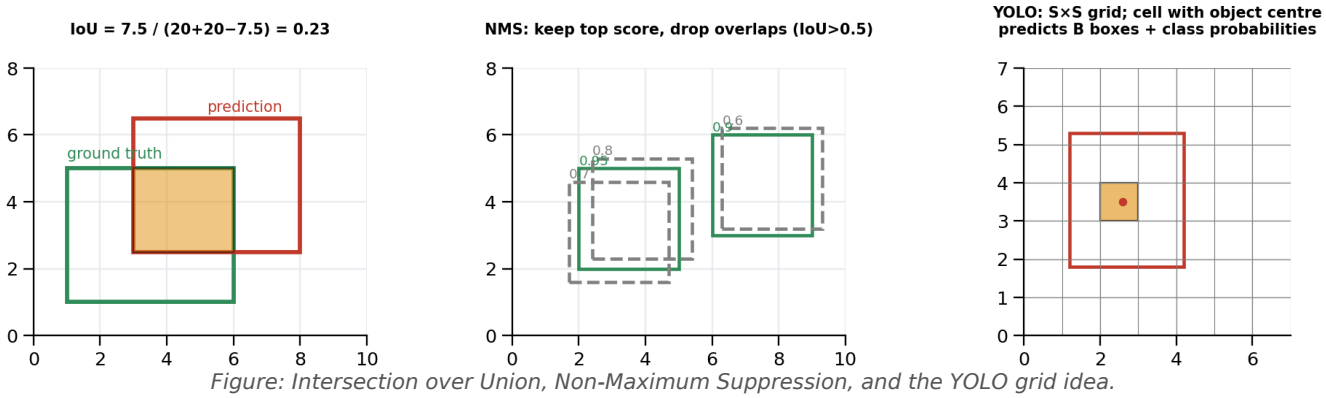


Figure: Intersection over Union, Non-Maximum Suppression, and the YOLO grid idea.

$$\text{IoU}(A,B) = \frac{|A \cap B|}{|A \cup B|} \in [0,1]$$

1. **NMS**: sort detections of a class by confidence.
2. Take the highest-scoring box, keep it.
3. Remove all remaining boxes with IoU > threshold (e.g. 0.5) with it.
4. Repeat until no boxes remain. (Soft-NMS decays scores instead of deleting.)

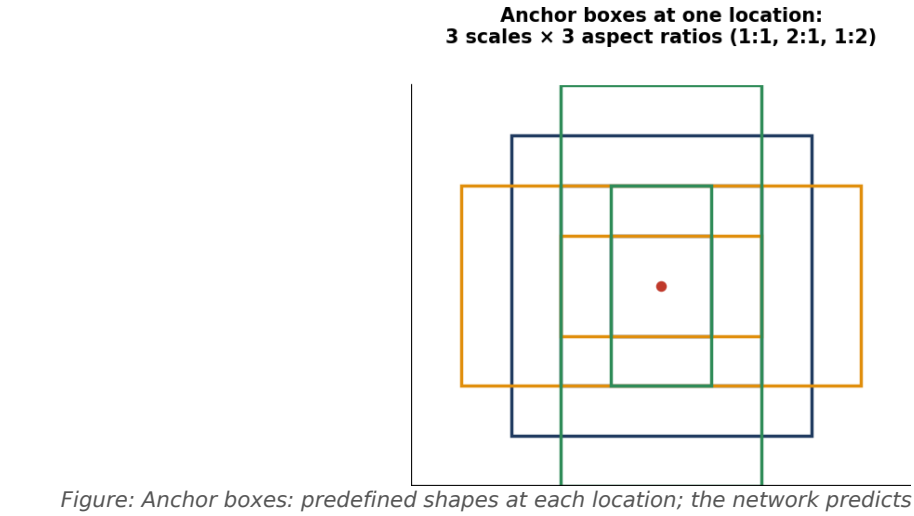


Figure: Anchor boxes: predefined shapes at each location; the network predicts offsets and objectness for each.

$$\hat{x} = x_a + w_a\, t_x, \quad \hat{y} = y_a + h_a\, t_y, \quad \hat{w} = w_a\, e^{t_w}, \quad \hat{h} = h_a\, e^{t_h}$$

box regression: the network predicts t = (t_x, t_y, t_w, t_h) relative to anchor a

9.3 Evaluation: precision, recall, AP, mAP

- A detection is a **true positive** if IoU with an unmatched ground-truth box ≥ threshold (0.5) and the class is right; otherwise false positive; missed ground truths are false negatives.
- Sort detections by confidence → compute precision/recall at each rank → **AP** = area under the precision–recall curve for one class.
- **mAP** = mean AP over classes. **mAP@0.5** (Pascal VOC) vs **mAP@[.5:.95]** (COCO: average over IoU 0.5, 0.55, …, 0.95 — rewards precise boxes).
- Speed: FPS or latency (ms) on stated hardware; always report both accuracy and speed.

$$AP = \int_0^1 p(r)\, dr \qquad mAP = \frac{1}{C} \sum_{c=1}^{C} AP_c$$

9.4 Two-stage detectors: the R-CNN family

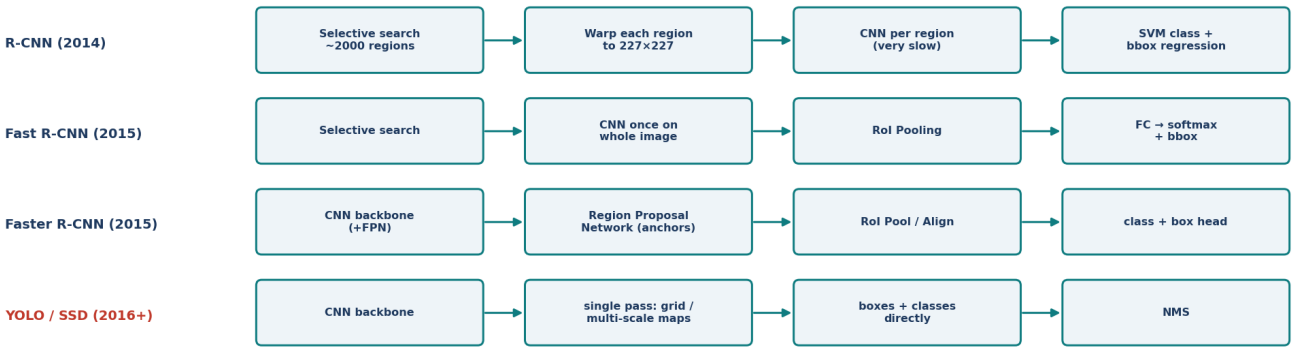


Figure: Evolution from R-CNN to single-stage detectors.

Model	How it works	Speed	Weakness fixed by the next
R-CNN (Girshick, 2014)	selective search proposes ~2000 regions → warp each → CNN features → SVM per class + box regression	~47 s/img	CNN run 2000× per image

Model	How it works	Speed	Weakness fixed by the next
Fast R-CNN (2015)	CNN once on whole image → project proposals onto feature map → **RoI Pooling** to fixed size → FC → softmax + box regressor (multi-task loss, end-to-end except proposals)	~2 s/img	selective search is slow (CPU)
Faster R-CNN (2015)	**Region Proposal Network** slides over the feature map; at each location k anchors (3 scales × 3 ratios) → objectness score + box refinement; top proposals → RoI Pool → head. Fully end-to-end, shared features	~5–7 FPS	two stages; small objects
+ FPN (2017)	Feature Pyramid Network: top-down pathway + lateral connections build strong multi-scale feature maps		standard neck today
Mask R-CNN (2017)	adds a mask branch + RoI Align (Chapter 10)	~5 FPS	

$$\mathcal{L} = \mathcal{L}_{cls}(p, u) + \lambda[u \geq 1]\,\mathcal{L}_{loc}(t^u, v)$$

multi-task loss: classification + smooth-L1 box regression only for non-background RoIs

9.5 YOLO — You Only Look Once

YOLO (Redmon et al., 2016) treats detection as a **single regression problem**: one CNN pass over the image predicts all boxes and classes simultaneously → real-time speed (45+ FPS in v1).

YOLOv1 in detail

- Divide the image into an **S×S grid** (S = 7). The cell containing an object's centre is responsible for it.
- Each cell predicts **B boxes** (B = 2), each with (x, y, w, h, confidence), where confidence = P(object) × IoU(pred, truth), plus **C class probabilities** per cell.
- Output tensor: **S × S × (B·5 + C)** = 7 × 7 × (2·5 + 20) = 7 × 7 × 30 for Pascal VOC.
- At test time: class-specific score = P(class | object) × confidence → threshold → NMS.

$$\mathcal{L} = \lambda_{coord}\sum \mathbb{1}^{obj}\left[(x-\hat{x})^2 + (y-\hat{y})^2 + (\sqrt{w}-\sqrt{\hat{w}})^2 + (\sqrt{h}-\sqrt{\hat{h}})^2\right] + \sum \mathbb{1}^{obj}(C-\hat{C})^2 + \lambda_{noobj}\sum \mathbb{1}^{noobj}(C-\hat{C})^2 + \sum \mathbb{1}^{obj}\sum_c (p(c)-\hat{p}(c))^2$$

λ_coord = 5, λ_noobj = 0.5; square roots make errors on small boxes count more

YOLO evolution

Version	Main improvements
v2 / YOLO9000 (2017)	anchor boxes (k-means on dataset box sizes), BatchNorm, higher resolution, multi-scale training, Darknet-19
v3 (2018)	Darknet-53 with residuals, predictions at 3 scales (FPN-like) → better small objects, independent logistic class scores
v4 (2020)	CSPDarknet, PANet neck, Mosaic augmentation, CIoU loss, many 'bag of freebies'
v5 (2020, Ultralytics)	PyTorch, easy training/export, auto-anchors
v6 / v7 (2022)	re-parameterised blocks, efficient layer aggregation
v8 (2023, Ultralytics)	anchor-free decoupled head; detection, segmentation, pose, classification in one framework
v9–v11+	programmable gradient information, NMS-free training (v10), further efficiency; versions keep evolving — check current releases

Limitations of early YOLO: struggles with many small objects in one cell and with unusual aspect ratios; lower localisation accuracy than two-stage detectors — mostly closed by later versions.

9.6 SSD — Single Shot MultiBox Detector (Liu et al., 2016)

Single-stage like YOLO, but predicts boxes from **multiple feature maps of different resolutions** (e.g. 38×38, 19×19, 10×10, 5×5, 3×3, 1×1 on a VGG backbone): early, high-resolution maps detect small objects; deep maps detect large ones. Each map location has several **default boxes** (anchors) of different aspect ratios; 3×3 conv heads output class scores + offsets. Uses hard negative mining (3:1 negatives:positives). SSD300 ran ~59 FPS with accuracy comparable to Faster R-CNN at the time.

9.7 RetinaNet and focal loss

$$FL(p_t) = -\alpha_t(1-p_t)^{\gamma}\log(p_t), \qquad \gamma = 2$$

down-weights the huge number of easy background examples so training focuses on hard ones

One-stage detectors evaluate ~100k candidate locations, almost all background — the easy negatives dominate the loss. Focal loss (Lin et al., 2017) fixed this and let a one-stage detector (RetinaNet = ResNet + FPN + focal loss) match two-stage accuracy.

9.8 Transformer detectors

DETR (2020): CNN backbone + transformer encoder–decoder with a fixed set of learned object queries → a set of predictions matched to ground truth with the Hungarian algorithm. No anchors, no NMS; slower to train. RT-DETR and DINO-DETR made it real-time and accurate; open-vocabulary detectors (OWL-ViT, Grounding DINO) detect objects described by text.

9.9 Comparison and choosing

Detector	Type	Accuracy	Speed	Use
Faster R-CNN + FPN	two-stage	high, good small objects	slower	accuracy-critical, offline
SSD	one-stage	medium	fast	mobile (SSD-MobileNet)
YOLO (v5/v8/…)	one-stage	high	real-time	most practical applications, edge, video
RetinaNet	one-stage	high	medium	dense/imbalanced scenes
DETR family	transformer	high	medium–fast	end-to-end, research

9.10 Hands-on

detection_handson.py

```
# pip install ultralytics
from ultralytics import YOLO
model = YOLO("yolov8n.pt")                                    # pre-trained on COCO (80 classes)
results = model("street.jpg", conf=0.25, iou=0.5)            # inference (NMS inside)
for b in results[0].boxes: print(model.names[int(b.cls)], float(b.conf), b.xyxy.tolist())
results[0].save("pred.jpg")

# train on a custom dataset in YOLO format (images/ + labels/*.txt with "class xc yc w h" normalised; data.yaml lists paths
        & names)
model = YOLO("yolov8n.pt"); model.train(data="data.yaml", epochs=50, imgsz=640, batch=16)
metrics = model.val(); print(metrics.box.map50, metrics.box.map)   # mAP@0.5, mAP@0.5:0.95
model.export(format="onnx")

# torchvision Faster R-CNN
import torchvision
det = torchvision.models.detection.fasterrcnn_resnet50_fpn(weights="DEFAULT").eval()
```

CHAPTER 10

Image Segmentation — U-Net, Mask R-CNN

Semantic vs instance vs panoptic • FCN • upsampling & transposed convolution • encoder–decoder • U-Net in depth • DeepLab & atrous convolution • Mask R-CNN & RoI Align • losses (CE, Dice, IoU, focal) • metrics (pixel acc., IoU, mIoU, Dice) • SAM • code

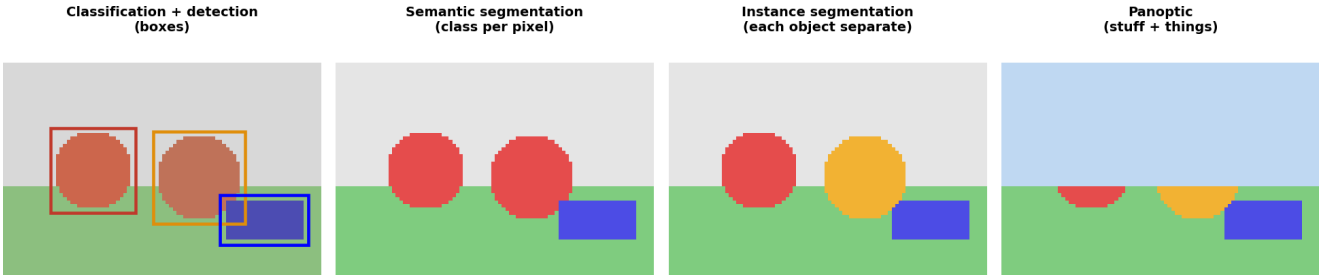


Figure: Detection vs semantic, instance and panoptic segmentation (synthetic illustration).

Type	Output	Example models
Semantic	class label for every pixel; objects of the same class merged	FCN, U-Net, DeepLab, SegFormer
Instance	separate mask per object (countable 'things')	Mask R-CNN, YOLOv8-seg
Panoptic	semantic for 'stuff' (sky, road) + instance for 'things'	Panoptic FPN, Mask2Former
Promptable	mask for any point/box/text prompt	Segment Anything (SAM)

10.1 Fully Convolutional Networks (FCN, Long et al., 2015)

Replace the FC layers of a classifier with 1×1 convolutions so the network outputs a coarse class **map** instead of one label, then **upsample** it to the input size. Skip connections from earlier layers (FCN-16s, FCN-8s) add finer detail. Works on any input size.

Upsampling options

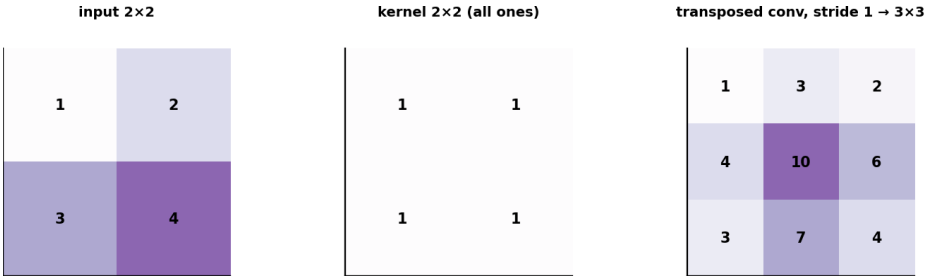


Figure: Transposed convolution: each input value 'stamps' a scaled kernel onto the output; overlaps add (computed).

Method	Learnable	Notes
Nearest / bilinear upsampling	no	UpSampling2D; often + conv afterwards

Method	Learnable	Notes
Transposed convolution (Conv2DTranspose)	yes	can cause checkerboard artefacts
Max-unpooling	no	uses stored max indices (SegNet)
Pixel shuffle	via conv	rearranges channels into space (super-resolution)

10.2 U-Net (Ronneberger et al., 2015)

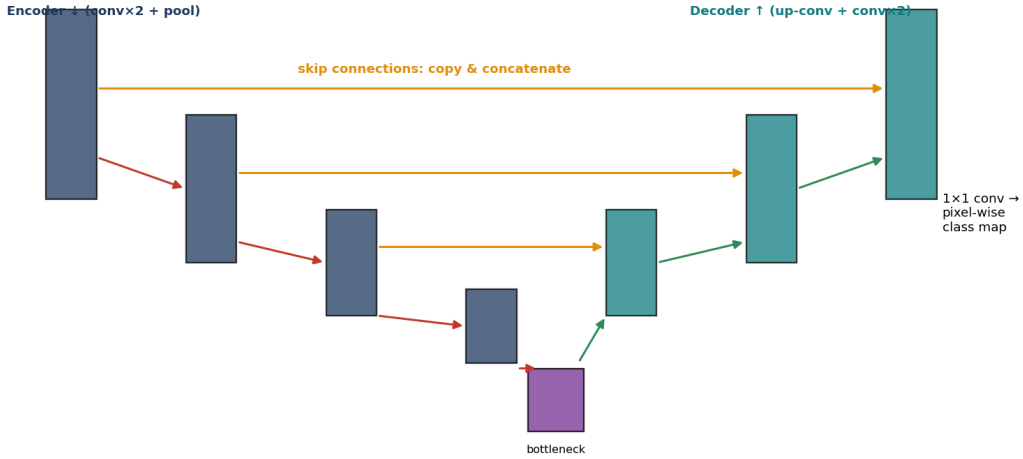


Figure: U-Net: contracting encoder, expanding decoder, and skip connections that concatenate encoder features into the decoder.

- **Encoder (contracting path)**: repeated [3×3 conv → ReLU] × 2 → 2×2 max-pool; channels double (64 → 128 → 256 → 512 → 1024). Captures *what* (context).
- **Decoder (expanding path)**: 2×2 up-convolution (halves channels) → **concatenate** with the encoder feature map of the same resolution → [3×3 conv → ReLU] × 2. Recovers *where* (precise localisation).
- **Skip connections** bring back fine spatial detail lost by pooling → sharp boundaries.
- Final 1×1 conv maps to the number of classes per pixel (sigmoid for binary masks, softmax for multi-class).
- Designed for biomedical images with very few training images, heavy elastic-deformation augmentation and a weighted loss emphasising borders between touching cells.
- Today U-Net variants are everywhere: medical imaging, satellite imagery, and the denoising backbone of Stable Diffusion.

unet_keras.py

```
from tensorflow.keras import layers, Model, Input
def conv_block(x, f):
    x = layers.Conv2D(f, 3, padding="same", activation="relu")(x); return layers.Conv2D(f, 3, padding="same",
        activation="relu")(x)
def unet(input_shape=(128, 128, 3), n_classes=1):
    inp = Input(input_shape); skips = []; x = inp
    for f in (64, 128, 256, 512):                    # encoder
        x = conv_block(x, f); skips.append(x); x = layers.MaxPooling2D()(x)
    x = conv_block(x, 1024)                          # bottleneck
    for f, s in zip((512, 256, 128, 64), reversed(skips)):   # decoder
        x = layers.Conv2DTranspose(f, 2, strides=2, padding="same")(x)
        x = layers.Concatenate()([x, s]); x = conv_block(x, f)
    out = layers.Conv2D(n_classes, 1, activation="sigmoid" if n_classes == 1 else "softmax")(x)
    return Model(inp, out)
model = unet(); model.compile("adam", loss=bce_dice_loss, metrics=[dice_coef])
```

10.3 DeepLab family

Uses **atrous (dilated) convolutions** to keep feature maps at higher resolution while enlarging the receptive field, and **ASPP** (Atrous Spatial Pyramid Pooling: parallel dilated convs with rates 6, 12, 18 + image pooling) for multi-scale context. DeepLabv3+ adds a light decoder for sharper boundaries. Strong for street scenes (Cityscapes).

10.4 Mask R-CNN (He et al., 2017)



- Extends Faster R-CNN with a third, parallel branch: a small FCN predicting a binary mask for each RoI and each class.
- **RoI Align** replaces RoI Pooling: no quantisation of coordinates; bilinear interpolation at sampling points → pixel-accurate alignment (big gain for masks).
- Loss = $L_{cls} + L_{box} + L_{mask}$; mask loss = per-pixel binary cross-entropy only on the mask of the ground-truth class (decouples mask and class prediction).
- Also does keypoint (pose) estimation with a keypoint head.

mask_rcnn.py

```
import torchvision, torch
model = torchvision.models.detection.maskrcnn_resnet50_fpn(weights="DEFAULT").eval()
with torch.no_grad(): out = model([img_tensor])[0]           # img_tensor: (3, H, W) in [0,1]
keep = out["scores"] > 0.7; masks = out["masks"][keep] > 0.5; labels = out["labels"][keep]   # masks: (N, 1, H,
       W)
```

10.5 Losses and metrics

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|} \qquad \mathcal{L}_{Dice} = 1 - \frac{2\sum p_i g_i + \varepsilon}{\sum p_i + \sum g_i + \varepsilon} \qquad \text{IoU} = \frac{|A \cap B|}{|A \cup B|} \qquad \text{mIoU} = \frac{1}{C}\sum_c \text{IoU}_c$$

Loss	When
Pixel-wise (binary/categorical) cross-entropy	default
Weighted CE / focal loss	class imbalance (small objects)
Dice / IoU (Jaccard) loss	foreground is small (medical lesions); directly optimises overlap
BCE + Dice	common robust combination
Boundary / Lovász losses	sharper edges / IoU optimisation

seg_losses.py

```
import tensorflow as tf
def dice_coef(y_true, y_pred, eps=1e-6):
    y_true = tf.reshape(y_true, [-1]); y_pred = tf.reshape(y_pred, [-1])
    inter = tf.reduce_sum(y_true * y_pred); return (2 * inter + eps) / (tf.reduce_sum(y_true) + tf.reduce_sum(y_pred) + eps)
def bce_dice_loss(y_true, y_pred): return tf.keras.losses.binary_crossentropy(y_true, y_pred) + (1 - dice_coef(y_true,
       y_pred))
```

10.6 Segment Anything (SAM)

Meta's foundation model (2023) trained on 1B+ masks: a ViT image encoder + prompt encoder (points, boxes) + lightweight mask decoder produce masks for any object zero-shot. Commonly used for fast annotation and combined with detectors (Grounding DINO + SAM = text-prompted segmentation).

CHAPTER 11

CNN Visualisation Techniques

Why interpret CNNs • visualising filters • feature maps • occlusion sensitivity • saliency maps • Grad-CAM (math + code) • Guided Grad-CAM, Integrated Gradients • embedding visualisation (t-SNE) • activation maximisation & DeepDream • debugging with visualisations

11.1 Why visualise?

- Trust and debugging: is the model looking at the object or at a watermark / background shortcut (e.g. 'snow' for wolves)?
- Error analysis: why did it misclassify?
- Compliance/medical: explain decisions to clinicians and auditors.
- Understanding: what each layer learns.

11.2 Visualising filters and feature maps

First-layer filters (e.g. 3×3×3 or 7×7×3) can be plotted directly — they look like edge and colour detectors (as in the hand-made filters of Chapter 4). Deeper filters are not interpretable as images; instead visualise their **activations** (feature maps) for a given input.

feature_maps.py

```
from tensorflow import keras
import matplotlib.pyplot as plt
w, b = model.get_layer("conv2d").get_weights()                # (3, 3, C_in, C_out)
layer_outputs = [l.output for l in model.layers if "conv" in l.name]
activation_model = keras.Model(model.inputs, layer_outputs)
acts = activation_model.predict(img[None])                     # list of (1, H, W, C) arrays
for i in range(16): plt.subplot(4, 4, i + 1); plt.imshow(acts[0][0, :, :, i], cmap="viridis");
        plt.axis("off")
```

11.3 Occlusion sensitivity

Slide a grey/zero patch over the image and record how much the predicted class probability drops. Large drops mark regions the model relies on. Model-agnostic but slow (one forward pass per patch position).

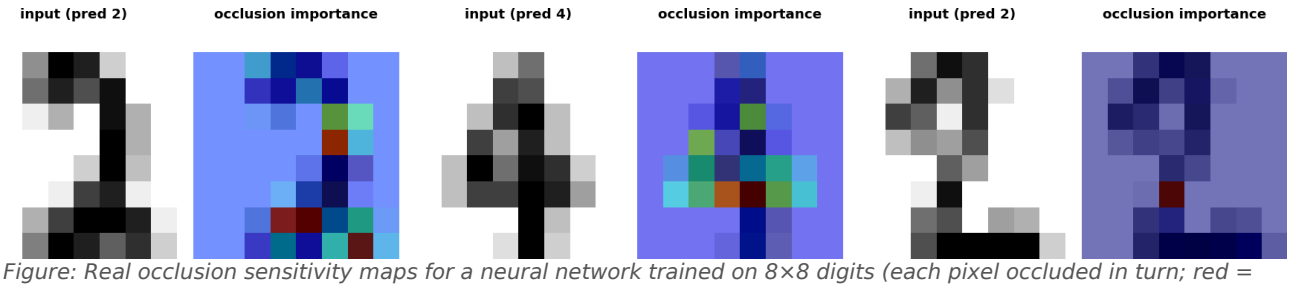


Figure: Real occlusion sensitivity maps for a neural network trained on 8×8 digits (each pixel occluded in turn; red = most important).

11.4 Saliency maps (vanilla gradients)

$$M(i,j) = \max_c \left| \frac{\partial y^c}{\partial x_{i,j,c}} \right|$$

how much a tiny change of each pixel changes the class score

One backward pass; noisy. **SmoothGrad** averages saliency over noisy copies of the input; **Integrated Gradients** integrates gradients along a path from a baseline (black image) to the input and satisfies useful axioms (completeness).

11.5 Grad-CAM (Selvaraju et al., 2017)

$$\alpha_k^c = \frac{1}{Z}\sum_i\sum_j \frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right)$$

1. Take the last convolutional layer's feature maps A^k (e.g. 7×7×2048).
2. Compute gradients of the class score y^c (before softmax) w.r.t. those maps.
3. Global-average-pool the gradients → importance weight α_k per channel.
4. Weighted sum of feature maps, ReLU (keep positive evidence) → coarse heat-map.
5. Upsample to image size and overlay. Class-discriminative; works for any CNN without retraining.

grad_cam.py

```
import tensorflow as tf, numpy as np
def grad_cam(model, img, last_conv="top_conv", class_idx=None):
    grad_model = tf.keras.Model(model.inputs, [model.get_layer(last_conv).output, model.output])
    with tf.GradientTape() as tape:
        conv_out, preds = grad_model(img[None])
        class_idx = class_idx if class_idx is not None else tf.argmax(preds[0])
        score = preds[:, class_idx]
    grads = tape.gradient(score, conv_out)                          # (1, h, w, K)
    weights = tf.reduce_mean(grads, axis=(0, 1, 2))                 # α_k
    cam = tf.nn.relu(tf.reduce_sum(conv_out[0] * weights, axis=-1))
    cam = cam / (tf.reduce_max(cam) + 1e-8)
    return tf.image.resize(cam[..., None], img.shape[:2]).numpy().squeeze()   # overlay with plt.imshow(cam, alpha=0.4,
        cmap="jet")
```

Variants: Grad-CAM++ (better for multiple instances), Score-CAM (gradient-free), Guided Grad-CAM (combines with guided backprop for fine detail). Libraries: tf-keras-vis, pytorch-grad-cam, Captum (PyTorch), SHAP.

11.6 Other techniques

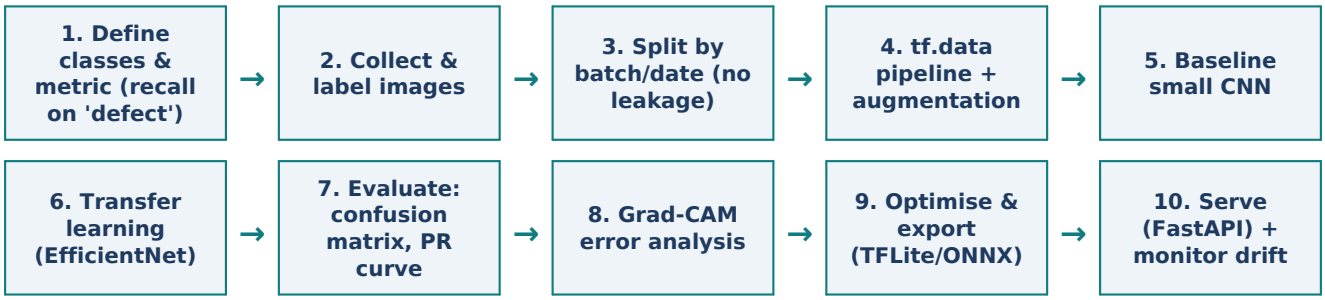
Technique	What it shows
t-SNE / UMAP of penultimate-layer embeddings	whether classes form separate clusters; mislabelled samples
Activation maximisation	synthesise an input that maximally activates a neuron/class (gradient ascent on the image)
DeepDream	amplify whatever a layer detects → psychedelic patterns
Deconvnet (ZFNet)	project activations back to pixel space
Network dissection	match units to human concepts (textures, objects)

CHAPTER 12

Hands-on: End-to-End Computer Vision Project

Problem definition • data collection & labelling • splitting • pipeline & augmentation • baseline → transfer learning • training • evaluation & error analysis • Grad-CAM checks • optimisation & export • FastAPI deployment • monitoring

12.1 Project: defect detection on manufactured parts (classification)



12.2 Project structure

structure

```
cv_project/
├── data/                  raw/  train/<class>/  val/<class>/  test/<class>/
├── src/
│   ├── data.py            # loading, augmentation, tf.data pipeline
│   ├── model.py           # build_model() — transfer-learning architecture
│   ├── train.py           # training loop, callbacks, MLflow/TensorBoard logging
│   ├── evaluate.py        # metrics, confusion matrix, Grad-CAM samples
│   └── export.py          # SavedModel / TFLite / ONNX
├── api/main.py            # FastAPI inference service
├── tests/                 # pytest: shapes, preprocessing, API
├── Dockerfile  requirements.txt  README.md
```

12.3 Key code

data / model / train / export

```
# src/data.py
import tensorflow as tf
IMG, BS = (224, 224), 32
def load(split, shuffle):
    ds = tf.keras.utils.image_dataset_from_directory(f"data/{split}", image_size=IMG, batch_size=BS, shuffle=shuffle,
        label_mode="int")
    return ds.cache().prefetch(tf.data.AUTOTUNE), ds.class_names

# src/model.py
from tensorflow import keras
from tensorflow.keras import layers
def build_model(n_classes):
    aug = keras.Sequential([layers.RandomFlip("horizontal_and_vertical"), layers.RandomRotation(0.15),
        layers.RandomContrast(0.2)])
    base = keras.applications.EfficientNetB0(include_top=False, weights="imagenet"); base.trainable = False
    inp = keras.Input(IMG + (3,)); x = aug(inp); x = keras.applications.efficientnet.preprocess_input(x)
    x = base(x, training=False); x = layers.GlobalAveragePooling2D()(x); x = layers.Dropout(0.3)(x)
    return keras.Model(inp, layers.Dense(n_classes, activation="softmax")(x)), base

# src/train.py
train_ds, classes = load("train", True); val_ds, _ = load("val", False)
model, base = build_model(len(classes))
counts = ...  # images per class → class_weight = {i: total / (len(classes) * n_i)}
model.compile(keras.optimizers.Adam(1e-3), "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=15, class_weight=class_weight,
          callbacks=[keras.callbacks.EarlyStopping(patience=4, restore_best_weights=True),
        keras.callbacks.TensorBoard("logs")])
base.trainable = True; [setattr(l, "trainable", False) for l in base.layers[:-30]]
model.compile(keras.optimizers.Adam(1e-5), "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10); model.save("defect_model.keras")

# src/export.py — smaller, faster model for edge devices
conv = tf.lite.TFLiteConverter.from_keras_model(model); conv.optimizations = [tf.lite.Optimize.DEFAULT]   # dynamic-range
        quantisation
open("defect.tflite", "wb").write(conv.convert())
```

api/main.py

```
# api/main.py
from fastapi import FastAPI, UploadFile, File
import numpy as np, io, tensorflow as tf
from PIL import Image
app = FastAPI(); model = tf.keras.models.load_model("defect_model.keras"); CLASSES = ["defect", "ok"]
@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    img = Image.open(io.BytesIO(await file.read())).convert("RGB").resize((224, 224))
    p = model.predict(np.asarray(img, dtype="float32")[None], verbose=0)[0]       # preprocessing is inside the model
    return {"label": CLASSES[int(p.argmax())], "confidence": float(p.max()), "probs": dict(zip(CLASSES, map(float, p)))}
```

12.4 Evaluation & error analysis checklist

- Report per-class precision/recall, confusion matrix, PR curve; choose the decision threshold for the business cost (missing a defect is expensive → high recall).
- Look at misclassified images grouped by cause (lighting, blur, rare defect type, label error).
- Run Grad-CAM on correct and wrong predictions: is the model focusing on the defect or on background/fixtures?
- Test robustness: brightness, rotation, new camera; check performance by production line/date.
- Measure latency and model size for the target hardware; quantise or switch to MobileNet if needed.
- In production: log inputs/predictions, monitor confidence distribution and image statistics (drift), collect hard cases for relabelling and retraining.

Part III — Sequence Models

RNNs, vanishing gradients, LSTM, GRU, bidirectional networks, sequence-to-sequence, attention and time-series forecasting

CHAPTER 13

Recurrent Neural Networks (RNN) — Theory

Sequential data • why feed-forward nets fail • the recurrent cell • unrolling & weight sharing • equations & shapes • RNN architectures (one-to-many …) • training with BPTT • parameter count • Keras implementation • applications & limitations

13.1 Sequential data

Text, speech, sensor readings, stock prices, logs, DNA, video frames: the **order** matters, lengths vary, and an element's meaning depends on context ('not good' ≠ 'good'). A dense network takes a fixed-size input and treats positions independently; it cannot share what it learned at position 3 with position 30 and cannot handle variable length naturally.

13.2 The recurrent neuron

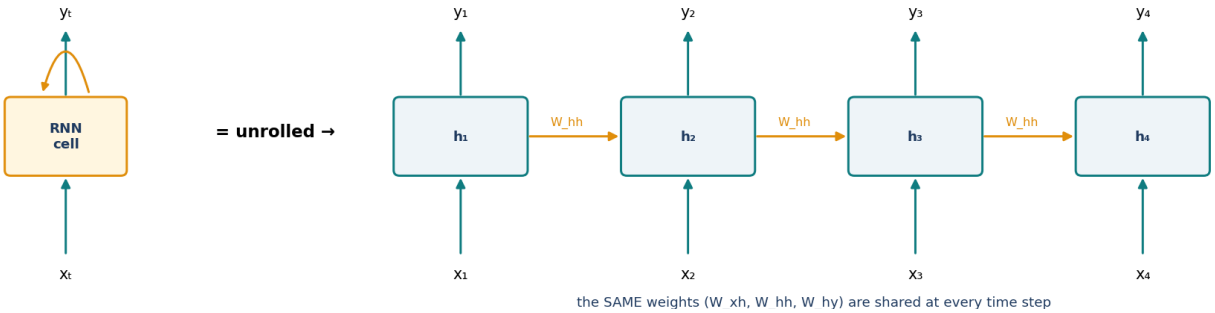


Figure: An RNN cell with a feedback loop, and the same network unrolled through time.

$$\mathbf{h}_t = \tanh\left(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h\right), \qquad \hat{\mathbf{y}}_t = g\left(W_{hy}\mathbf{h}_t + \mathbf{b}_y\right), \qquad \mathbf{h}_0 = \mathbf{0}$$

- **Hidden state $\mathbf{h}_t$** = the network's memory: a summary of everything seen up to time t.
- The **same weights** are used at every time step (weight sharing over time) → handles any length, few parameters.
- Shapes: $x_t \in \mathbb{R}^d$, $h_t \in \mathbb{R}^h$; W_{xh}: h×d, W_{hh}: h×h. Keras input tensor: (batch, timesteps, features).
- **Parameters** of SimpleRNN(h) on d features = h·d + h·h + h. Example: SimpleRNN(64) on 10 features → 64·10 + 64·64 + 64 = **4,800**.

Tiny numeric example

Step	Input	Computation (scalar RNN, W_x = 0.5, W_h = 0.8)	h_t
t=1	x=1.0	tanh(0.5·1.0 + 0.8·0.000)	0.462
t=2	x=0.5	tanh(0.5·0.5 + 0.8·0.462)	0.551
t=3	x=-1.0	tanh(0.5·-1.0 + 0.8·0.551)	-0.059

The final h_3 depends on all three inputs — information is carried forward through h.

13.3 RNN architectures

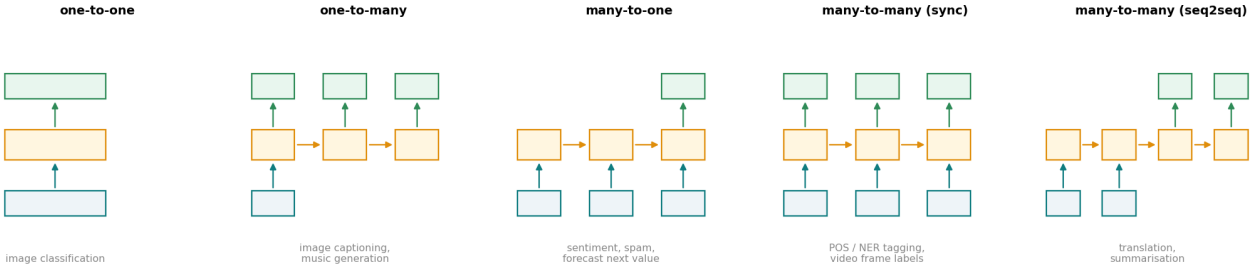


Figure: Input/output configurations of recurrent networks.

13.4 Training: Backpropagation Through Time (BPTT)

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t, \qquad \frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \left(\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial \mathbf{h}_k}{\partial W_{hh}}$$

Unroll the network over T steps (a deep feed-forward net with shared weights), run ordinary backprop, and **sum** the gradients of the shared weights over all steps. **Truncated BPTT** limits backprop to the last k steps for long sequences (memory/speed). The product of Jacobians in the formula is the source of the vanishing/exploding gradient problem (next chapter).

13.5 Keras implementation

simple_rnn.py

```
from tensorflow import keras
from tensorflow.keras import layers
# many-to-one: sentiment classification of padded integer sequences
model = keras.Sequential([
    layers.Input(shape=(200,)),                        # 200 token ids
    layers.Embedding(input_dim=20000, output_dim=64, mask_zero=True),   # (batch, 200, 64)
    layers.SimpleRNN(64),                              # returns last h_T → (batch, 64)
    layers.Dense(1, activation="sigmoid")])
# many-to-many (tagging): layers.SimpleRNN(64, return_sequences=True) → (batch, 200, 64) then TimeDistributed(Dense(n_tags,
        "softmax"))
# stacked RNN: every recurrent layer except the last needs return_sequences=True
# return_state=True also returns the final hidden state (used in seq2seq encoders)
```

Argument	Meaning
units	size of hidden state h
return_sequences	False: only last h (many-to-one); True: all h_t (many-to-many, stacking)
return_state	also return final state(s)
dropout / recurrent_dropout	dropout on inputs / on recurrent connections
stateful	carry state across batches (long streams)
go_backwards	process reversed sequence

13.6 Applications and limitations

Applications: language modelling, sentiment, NER/POS tagging, speech recognition, machine translation, music generation, time-series forecasting, anomaly detection in logs. Limitations: sequential computation (no parallelism over time → slow), difficulty learning long-range dependencies (vanishing gradients), fixed-size memory. Solved partly by LSTM/GRU, then by attention and transformers.

CHAPTER 14

The Vanishing (and Exploding) Gradient Problem in RNNs

Why gradients shrink or blow up • the math • effect on learning long dependencies • visualisation • exploding gradients & gradient clipping • all remedies

14.1 The math

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \prod_{j=k+1}^{t} \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) W_{hh}$$

Each factor contains $\tanh' \leq 1$ and W_{hh}. If the largest singular value of W_{hh} times the tanh derivative is **< 1**, the norm of this product decays roughly like γ^{t-k} → gradients from distant time steps **vanish**. If it is **> 1** they grow exponentially → **explode**.

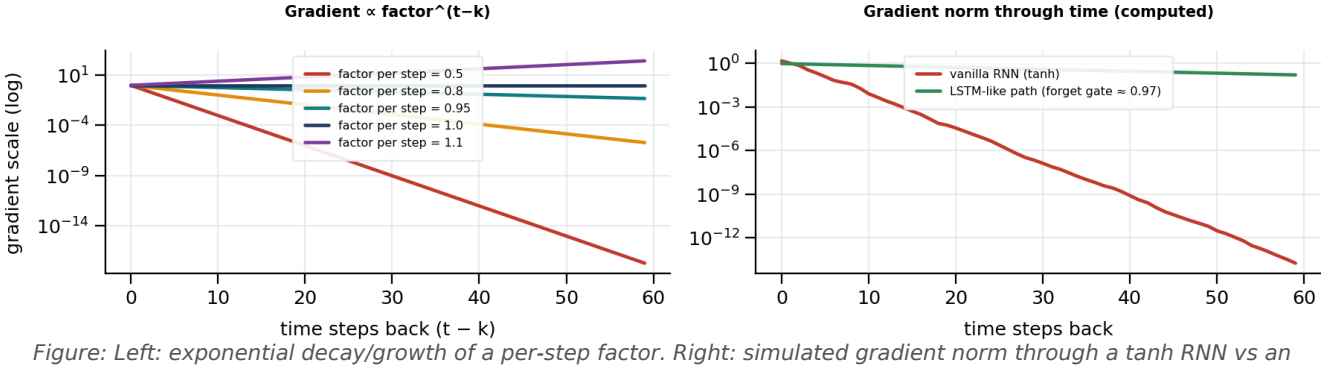


Figure: Left: exponential decay/growth of a per-step factor. Right: simulated gradient norm through a tanh RNN vs an LSTM-like additive path (computed).

14.2 Consequences

- **Vanishing**: the network cannot learn dependencies more than ~10–20 steps apart. Example: 'The *keys* that the manager left on the table near the window *are* missing' — the verb agreement depends on a word far back. Training appears to work but only short-term patterns are learned.
- **Exploding**: loss suddenly spikes or becomes NaN; weights jump wildly.

14.3 Remedies

Remedy	Fixes	How
Gradient clipping	exploding	rescale gradient if ‖g‖ > threshold: g ← g·τ/‖g‖ (Keras: optimizer(clipnorm=1.0); PyTorch: clip_grad_norm_)
Gated cells: LSTM / GRU	vanishing	additive cell-state updates with gates create a 'gradient highway' (Chapter 15)
Careful initialisation	both	orthogonal W_hh (eigenvalues on unit circle), identity init + ReLU (IRNN)
Non-saturating activations	vanishing	ReLU in simple RNNs (risk of exploding)

Remedy	Fixes	How
Layer normalisation	both	stabilises hidden-state statistics
Truncated BPTT / shorter sequences	exploding, cost	limit unroll length
Skip connections over time / attention	vanishing	direct paths between distant steps (attention, transformers)

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1,\ \frac{\tau}{\|\mathbf{g}\|}\right)$$

gradient clipping by norm (τ = threshold, e.g. 1.0)

CHAPTER 15

LSTM & GRU: Implementation and Applications; Bidirectional RNN and LSTM

LSTM intuition • gate equations • step-by-step data flow • why it solves vanishing gradients • parameter count • GRU • LSTM vs GRU • stacked & bidirectional RNNs • Keras implementations • sentiment analysis • sequence tagging • text generation • applications

15.1 LSTM — Long Short-Term Memory (Hochreiter & Schmidhuber, 1997)

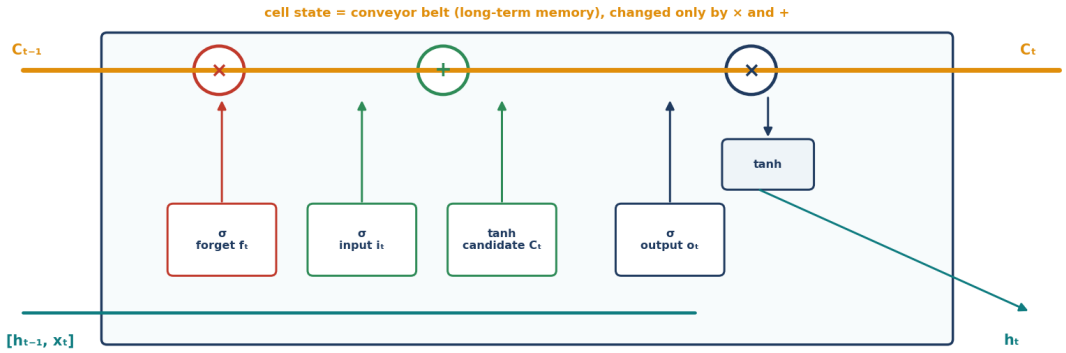


Figure: LSTM cell: a separate cell state C_t flows along the top; gates decide what to forget, add and output.

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \qquad \mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \qquad \tilde{\mathbf{C}}_t = \tanh(W_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C)$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \qquad \mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

Component	Role	Intuition (language model reading a story)
Forget gate f_t ($\sigma \to 0..1$)	how much of each memory cell to keep	new subject appears → forget the old subject's gender
Input gate i_t	how much new information to write	decide to store the new subject
Candidate C_t (tanh)	the new information itself	the new subject's features
Cell state C_t	long-term memory; updated additively	the running story context
Output gate o_t	what part of memory to expose as h_t	output what's needed to predict the next word (e.g. singular/plural)
Hidden state h_t	short-term output / working memory	used for predictions and next step

Why LSTMs fix vanishing gradients

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \text{diag}(\mathbf{f}_t) \text{ (+ small indirect terms)}$$

The cell state is updated by **addition**, not repeated matrix multiplication + squashing. When the forget gate is near 1, gradients flow back through many steps almost unchanged (the 'constant error carousel'). The network *learns* when to keep and when to reset memory. Tip: initialise the forget-gate bias to 1 (Keras unit_forget_bias=True by default).

Parameter count

$$\#\text{params}_{LSTM} = 4(h \cdot (h + d) + h), \qquad \#\text{params}_{GRU} \approx 3(h \cdot (h + d) + h)$$

Example: LSTM(64) on 100-dim embeddings → 4·(64·164 + 64) = **42,240** (four gate/candidate weight sets). GRU(64) in Keras (reset_after=True adds an extra bias) → 3·(64·164 + 2·64) = **31,872**.

15.2 GRU — Gated Recurrent Unit (Cho et al., 2014)

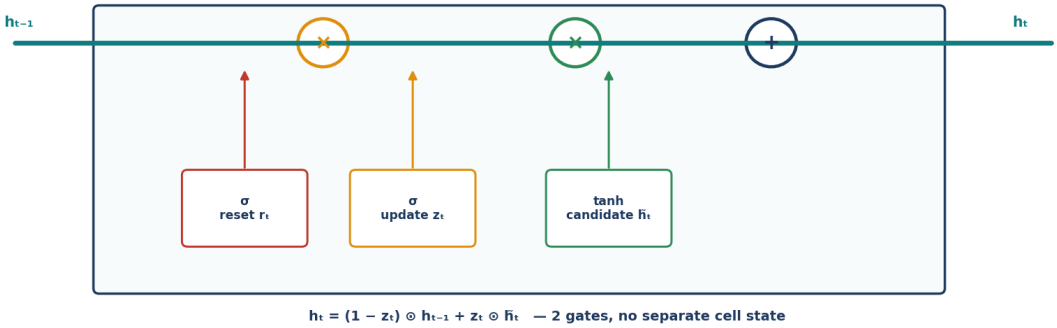


Figure: GRU: an update gate and a reset gate, no separate cell state.

$$\mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \tilde{\mathbf{h}}_t = \tanh(W[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

	LSTM	GRU
Gates	3 (forget, input, output)	2 (update, reset)
Memory	cell state + hidden state	hidden state only
Parameters	4 weight sets	3 weight sets (~25% fewer)
Speed	slower	faster
Accuracy	often slightly better on long/complex sequences	comparable on many tasks, better on small data
Default choice	LSTM when in doubt	GRU for speed / smaller data

15.3 Stacked and Bidirectional RNNs

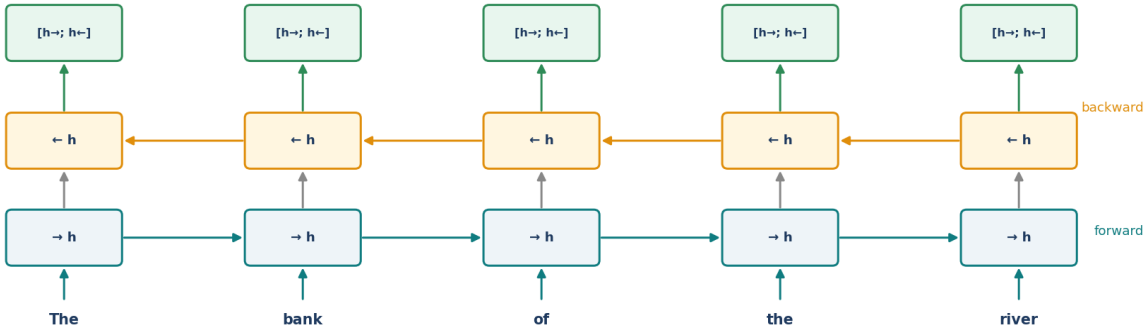


Figure: Bidirectional RNN: one layer reads left→right, another right→left; outputs are concatenated per position.

- **Bidirectional**: each position gets context from both the past and the future: $h_t = [h\rightarrow_t; h\leftarrow_t]$ (2× units). Ideal for **classification and tagging** where the whole sequence is available (NER: 'bank' meaning depends on later words). Not usable for real-time streaming or next-step forecasting/generation (the future is unknown). BERT is the transformer counterpart of this idea.
- **Stacked (deep) RNNs**: several recurrent layers on top of each other (lower layers return_sequences=True); 2–3 layers usually enough; add dropout between layers.
- Merge modes for Bidirectional: concat (default), sum, mul, ave.

15.4 Implementations

lstm_keras.py

```
from tensorflow import keras
from tensorflow.keras import layers
# (a) Sentiment analysis on IMDB with a stacked Bi-LSTM
(x_tr, y_tr), (x_te, y_te) = keras.datasets.imdb.load_data(num_words=20000)
x_tr = keras.utils.pad_sequences(x_tr, maxlen=200); x_te = keras.utils.pad_sequences(x_te, maxlen=200)
model = keras.Sequential([
    layers.Input((200,)),
    layers.Embedding(20000, 128, mask_zero=True),
    layers.Bidirectional(layers.LSTM(64, return_sequences=True, dropout=0.2)),
    layers.Bidirectional(layers.LSTM(32)),
    layers.Dense(64, activation="relu"), layers.Dropout(0.5),
    layers.Dense(1, activation="sigmoid")])
model.compile(keras.optimizers.Adam(1e-3, clipnorm=1.0), "binary_crossentropy", ["accuracy"])   # clipnorm → no exploding
        gradients
model.fit(x_tr, y_tr, validation_split=0.2, epochs=5, batch_size=64)
model.evaluate(x_te, y_te)                    # ≈ 0.86–0.89

# (b) Sequence tagging (NER/POS): one label per token
tagger = keras.Sequential([layers.Input((None,)), layers.Embedding(vocab, 100, mask_zero=True),
    layers.Bidirectional(layers.LSTM(64, return_sequences=True)), layers.TimeDistributed(layers.Dense(n_tags,
        activation="softmax"))])

# (c) Character-level text generation (many-to-one next-char prediction, then sample repeatedly)
gen = keras.Sequential([layers.Input((40, n_chars)), layers.LSTM(256), layers.Dense(n_chars, activation="softmax")])
```

bilstm_pytorch.py

```
import torch, torch.nn as nn
class BiLSTMClassifier(nn.Module):
    def __init__(self, vocab, emb=128, hid=64, n_cls=2):
        super().__init__(); self.emb = nn.Embedding(vocab, emb, padding_idx=0)
        self.lstm = nn.LSTM(emb, hid, num_layers=2, batch_first=True, bidirectional=True, dropout=0.2)
        self.fc = nn.Linear(2 * hid, n_cls)
    def forward(self, x):
        out, (h_n, c_n) = self.lstm(self.emb(x))            # h_n: (layers*2, batch, hid)
        h = torch.cat([h_n[-2], h_n[-1]], dim=1)            # last layer forward + backward states
        return self.fc(h)
```

15.5 Applications

Domain	Use
NLP	sentiment, NER, POS, language models, machine translation (pre-transformer), speech recognition
Time series	demand/energy/stock forecasting, sensor anomaly detection, predictive maintenance
Audio	speech recognition, keyword spotting, music generation
Others	handwriting recognition, video activity recognition (CNN + LSTM), clickstream modelling

CHAPTER 16

Sequence-to-Sequence Models

Encoder–decoder architecture • context vector • teacher forcing • training vs inference • decoding: greedy, beam search, sampling • exposure bias • the bottleneck problem • Keras implementation • applications

16.1 Architecture

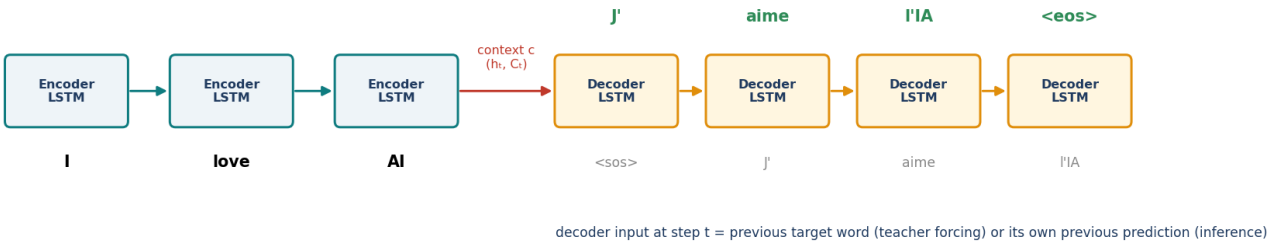


Figure: Encoder–decoder: the encoder summarises the input; the decoder generates the output token by token.

- Input and output sequences can have **different lengths** (translation, summarisation, chatbots, speech-to-text, code generation).
- **Encoder** (LSTM/GRU) reads the source sequence; its final states (h_T, C_T) form the **context vector**.
- **Decoder** (LSTM/GRU) initialised with the context generates the target one token at a time, starting from <sos> until it emits <eos>.
- Sutskever et al. (2014) also reversed the source sentence to shorten dependencies — an early sign of the bottleneck problem.

$$P(y_1, \ldots, y_{T'} \mid x_1, \ldots, x_T) = \prod_{t=1}^{T'} P(y_t \mid y_{<t}, \mathbf{c})$$

16.2 Training with teacher forcing

During training the decoder input at step t is the **ground-truth** previous token, not its own prediction. This makes training fast and stable (all steps computed in parallel with known inputs). Downside: **exposure bias** — at inference the model sees its own (possibly wrong) outputs, which it never practised on; mitigated by scheduled sampling.

16.3 Inference and decoding strategies

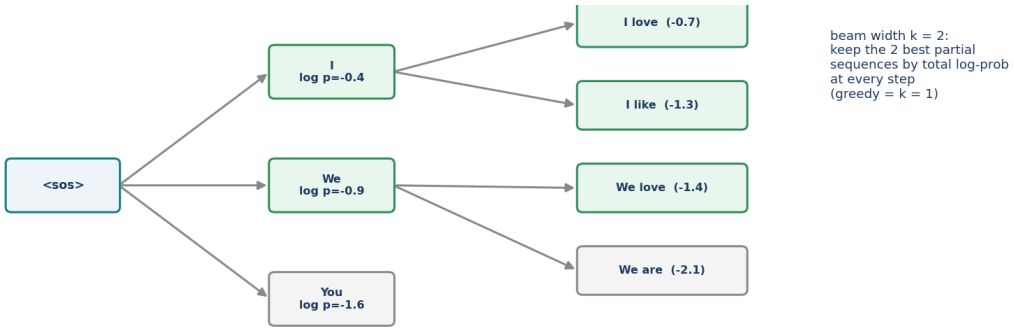


Figure: Beam search keeps the k best partial sequences at each step.

Strategy	How	Notes
Greedy	pick argmax token each step	fast; can miss better sequences
Beam search (k = 3–10)	keep top-k partial hypotheses by total log-probability	standard for translation/summarisation; length normalisation needed
Sampling (temperature, top-k, top-p)	sample from the distribution	diverse, creative text (chat, stories)

16.4 Keras implementation (character/word-level translation)

seq2seq_keras.py

```
from tensorflow import keras
from tensorflow.keras import layers
latent = 256
# ---- training model ----
enc_in = keras.Input((None,)); enc_emb = layers.Embedding(src_vocab, 128, mask_zero=True)(enc_in)
_, h, c = layers.LSTM(latent, return_state=True)(enc_emb)                    # context = [h, c]
dec_in = keras.Input((None,)); dec_emb_layer = layers.Embedding(tgt_vocab, 128, mask_zero=True); dec_emb =
        dec_emb_layer(dec_in)
dec_lstm = layers.LSTM(latent, return_sequences=True, return_state=True)
dec_out, _, _ = dec_lstm(dec_emb, initial_state=[h, c])
dec_dense = layers.Dense(tgt_vocab, activation="softmax"); out = dec_dense(dec_out)
model = keras.Model([enc_in, dec_in], out)
model.compile("adam", "sparse_categorical_crossentropy", ["accuracy"])
model.fit([src_seqs, tgt_in_seqs], tgt_out_seqs, batch_size=64, epochs=30, validation_split=0.1)
# tgt_in = "<sos> J' aime l'IA" ; tgt_out = "J' aime l'IA <eos>"  (shifted by one → teacher forcing)

# ---- inference models ----
encoder = keras.Model(enc_in, [h, c])
s_h, s_c = keras.Input((latent,)), keras.Input((latent,)); tok = keras.Input((None,))
o, h2, c2 = dec_lstm(dec_emb_layer(tok), initial_state=[s_h, s_c])
decoder = keras.Model([tok, s_h, s_c], [dec_dense(o), h2, c2])
def translate(src, max_len=50):
    states = encoder.predict(src[None], verbose=0); cur = np.array([[SOS]]); out = []
    for _ in range(max_len):
        p, hh, cc = decoder.predict([cur] + list(states), verbose=0); nxt = int(p[0, -1].argmax())   # greedy
        if nxt == EOS: break
        out.append(nxt); cur = np.array([[nxt]]); states = [hh, cc]
    return out
```

16.5 The bottleneck problem

The whole input must be squeezed into one fixed-size vector. Translation quality drops sharply for long sentences (Cho et al., 2014 observed BLEU falling as length grows). Solution: let the decoder look back at **all** encoder states at every step — **attention**.

CHAPTER 17

Attention Mechanism — Theory and Implementation

Intuition • Bahdanau (additive) attention • Luong (multiplicative) attention • alignment • attention in seq2seq step by step • self-attention • scaled dot-product attention • multi-head attention • masks • implementations (NumPy, Keras, PyTorch) • transformers • why attention replaced RNNs

17.1 Intuition

When translating word t, a human looks at the relevant source words, not the whole sentence equally. Attention computes, at every decoder step, a set of weights over the encoder states and a **context vector** = weighted sum — a soft, differentiable lookup.

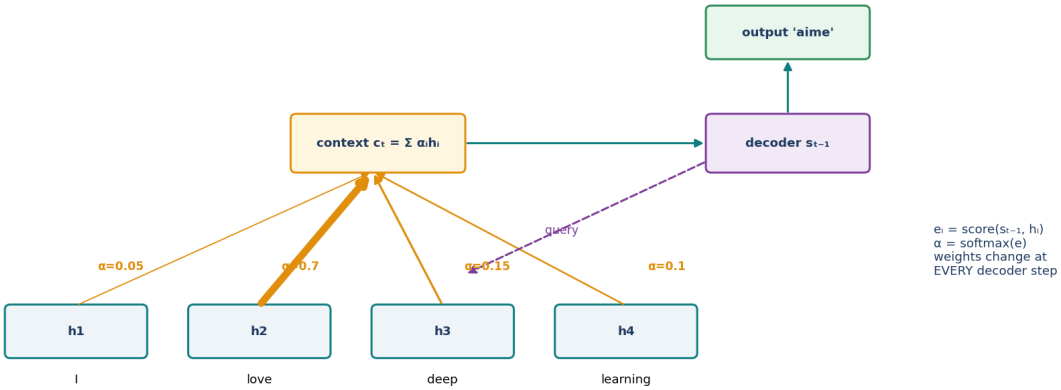


Figure: Bahdanau attention: the decoder state queries all encoder states; weights α form a new context for each output word.

17.2 Bahdanau (additive) attention, 2014

$$e_{t,i} = \mathbf{v}_a^T \tanh\left(W_a \mathbf{s}_{t-1} + U_a \mathbf{h}_i\right), \qquad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}, \qquad \mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i$$

1. Encoder (often bidirectional) produces annotations $h_1 \dots h_T$.
2. For decoder step t, **score** every h_i against the previous decoder state s_{t-1} with a small feed-forward network.
3. **Softmax** the scores → attention weights $\alpha_{t,i}$ (sum to 1).
4. **Context** c_t = weighted sum of encoder states.
5. Decoder uses [c_t; previous output] to update s_t and predict y_t. Repeat with new weights for the next word.

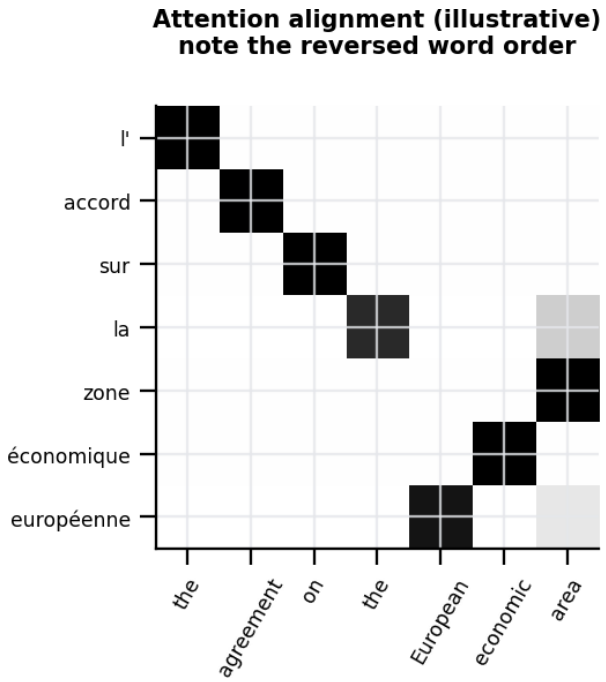


Figure: Attention weights form a soft alignment between source and target words (illustrative).

17.3 Luong (multiplicative) attention, 2015

Score function	Formula
dot	$s_t^T h_i$
general	$s_t^T W h_i$
concat (≈ Bahdanau)	$v^T \tanh(W[s_t; h_i])$

Luong uses the *current* decoder state s_t, cheaper multiplicative scores, and also proposed **local attention** (attend to a window). Dot-product attention is the ancestor of transformer attention.

17.4 Self-attention and scaled dot-product attention

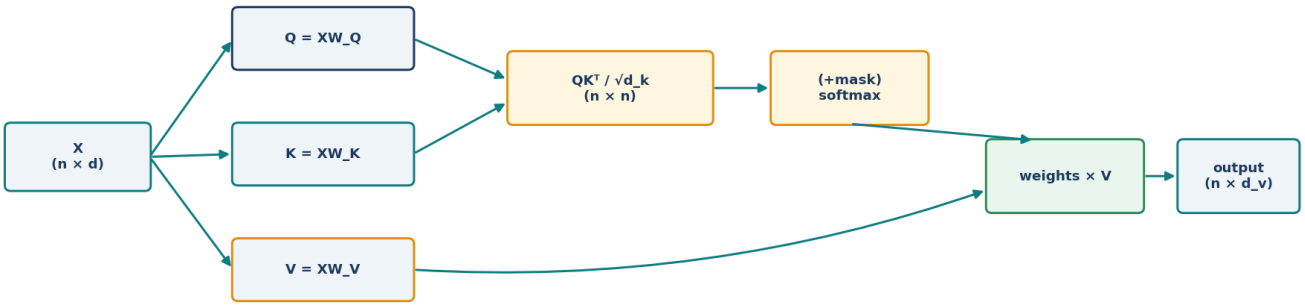


Figure: Scaled dot-product attention: queries, keys and values are projections of the same sequence.

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \qquad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- **Self-attention**: queries, keys and values all come from the same sequence → every token builds a context-aware representation by attending to every other token (no recurrence).
- **Query** = what I'm looking for; **Key** = what I contain; **Value** = what I pass on.

- **Scaling by $\sqrt{d_k}$**: dot products of d-dimensional random vectors have variance ∝ d; large values saturate softmax → tiny gradients.
- **Mask M**: $-\infty$ at forbidden positions — padding tokens, or future tokens in a decoder (causal mask).
- Cost $O(n^2 \cdot d)$ in sequence length n.

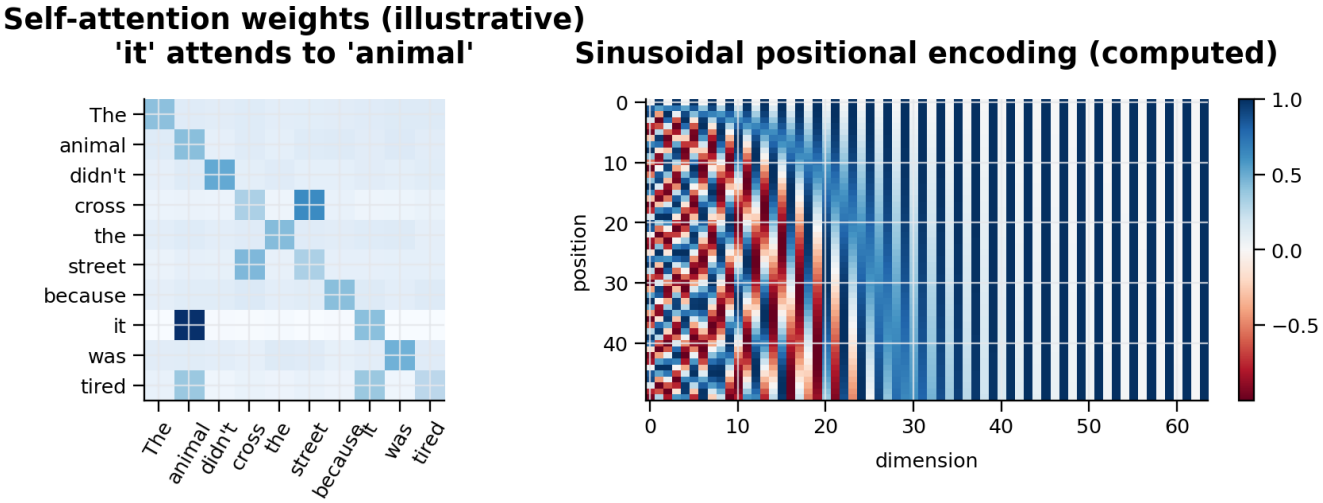


Figure: Self-attention weights for a sentence (illustrative) and sinusoidal positional encodings (computed).

Multi-head attention

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)W^O, \qquad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

h parallel attention heads (e.g. 8 heads of 64 dims for $d_{model} = 512$) attend to different relationships (syntax, coreference, position) at about the same cost as one big head.

17.5 Implementations

attention_numpy.py

```
import numpy as np
def softmax(x, axis=-1): e = np.exp(x - x.max(axis, keepdims=True)); return e / e.sum(axis, keepdims=True)
def scaled_dot_product_attention(Q, K, V, mask=None):
    scores = Q @ K.T / np.sqrt(Q.shape[-1])                  # (n_q, n_k)
    if mask is not None: scores = np.where(mask, scores, -1e9)
    w = softmax(scores); return w @ V, w
X = np.random.randn(4, 8)                                    # 4 tokens, d = 8
Wq, Wk, Wv = (np.random.randn(8, 8) for _ in range(3))
out, weights = scaled_dot_product_attention(X @ Wq, X @ Wk, X @ Wv)
print(weights.round(2), weights.sum(1))                      # each row sums to 1
causal = np.tril(np.ones((4, 4), bool))                      # decoder mask
```

attention_keras.py

```
from tensorflow import keras
from tensorflow.keras import layers
import tensorflow as tf
class BahdanauAttention(layers.Layer):
    def __init__(self, units):
        super().__init__(); self.W = layers.Dense(units); self.U = layers.Dense(units); self.v = layers.Dense(1)
    def call(self, query, values):                      # query: (B, H) decoder state; values: (B, T, H) encoder outputs
        score = self.v(tf.nn.tanh(self.W(query[:, None, :]) + self.U(values)))   # (B, T, 1)
        weights = tf.nn.softmax(score, axis=1)
        context = tf.reduce_sum(weights * values, axis=1)                         # (B, H)
        return context, weights

# Built-in layers
attn = layers.Attention()            # Luong dot-product: attn([query_seq, value_seq])
add_attn = layers.AdditiveAttention() # Bahdanau-style
mha = layers.MultiHeadAttention(num_heads=8, key_dim=64)
y = mha(query=x, value=x, key=x, use_causal_mask=False)   # self-attention

# Attention-pooling text classifier: Bi-LSTM + attention
inp = keras.Input((200,)); e = layers.Embedding(20000, 128)(inp)
h = layers.Bidirectional(layers.LSTM(64, return_sequences=True))(e)      # (B, 200, 128)
s = layers.Dense(1, activation="tanh")(h); a = layers.Softmax(axis=1)(s) # attention weight per token
ctx = layers.Lambda(lambda t: tf.reduce_sum(t[0] * t[1], axis=1))([h, a])
model = keras.Model(inp, layers.Dense(1, activation="sigmoid")(ctx))      # weights a show which words mattered
```

attention_pytorch.py

```
import torch, torch.nn.functional as F
q = k = v = torch.randn(2, 8, 10, 64)                    # (batch, heads, seq, d)
out = F.scaled_dot_product_attention(q, k, v, is_causal=True)   # fused, memory-efficient (FlashAttention when available)
mha = torch.nn.MultiheadAttention(embed_dim=512, num_heads=8, batch_first=True)
```

17.6 From attention to Transformers

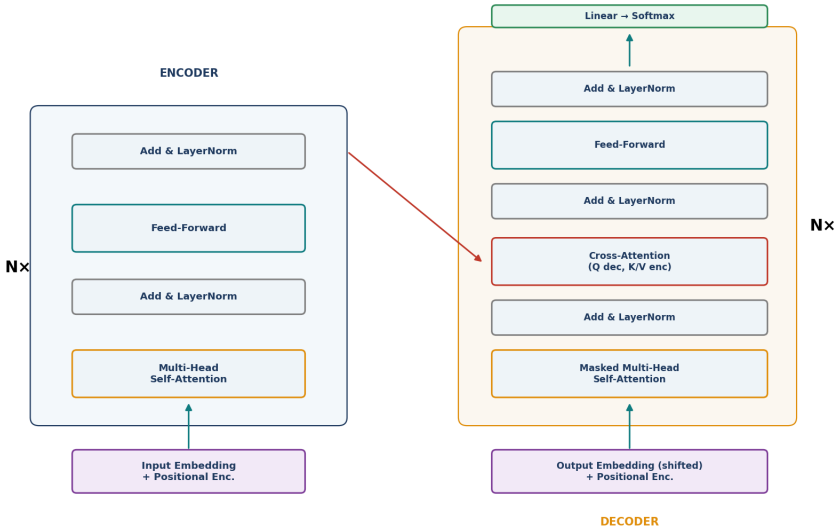


Figure: The Transformer (Vaswani et al., 2017): attention + feed-forward blocks, no recurrence.

RNN + attention	Transformer
sequential: step t waits for t−1 → slow on GPUs	all positions in parallel → fast training on huge data
long paths between distant words (O(n))	any two positions connected in one step (O(1))
fixed hidden state + attention	every token keeps its own representation
order built in	needs positional encodings
O(n) per layer memory	$O(n^2)$ attention memory (mitigated by FlashAttention, sparse attention)

Transformers (encoder: BERT; decoder: GPT; encoder–decoder: T5) are the basis of today's LLMs, ViT for images, Whisper for speech, and diffusion transformers for image/video generation.

CHAPTER 18

Hands-on: Time Series Forecasting with LSTM

Time-series components • stationarity • framing as supervised learning (sliding windows) • scaling without leakage • univariate & multivariate • single-step & multi-step forecasting • baselines • LSTM / GRU / CNN-LSTM models • evaluation metrics • walk-forward validation • pitfalls

18.1 Understanding the series

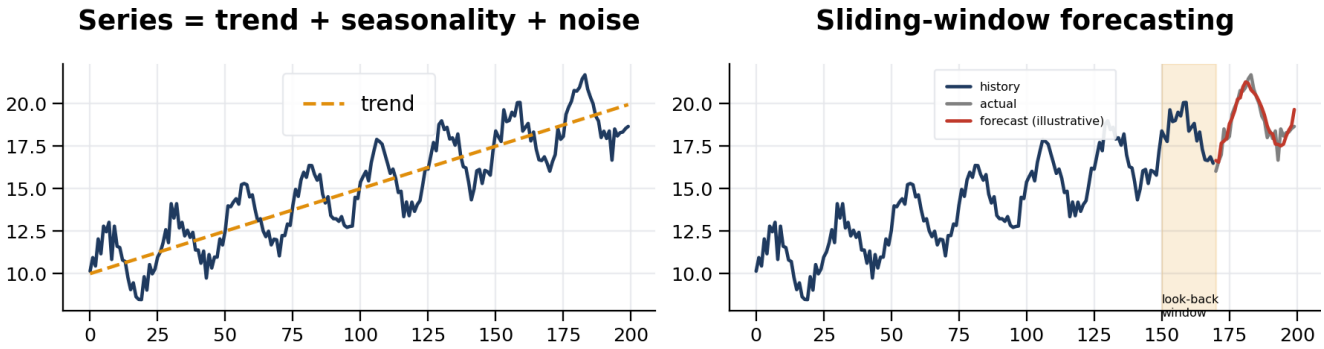


Figure: Trend + seasonality + noise, and the sliding-window forecasting set-up.

Concept	Meaning
Trend	long-term increase/decrease
Seasonality	repeating patterns (daily, weekly, yearly)
Cyclic	irregular long cycles (economy)
Noise	random variation
Stationarity	constant mean/variance over time; differencing or log transforms help
Autocorrelation	correlation with lagged values (ACF/PACF plots)

18.2 Framing as supervised learning

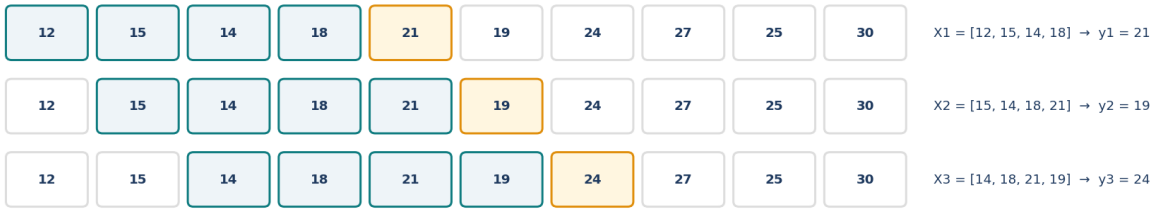


Figure: Sliding windows turn a series into (X = last n values, y = next value) pairs.

windows.py

```
import numpy as np, pandas as pd
from sklearn.preprocessing import MinMaxScaler
df = pd.read_csv("energy.csv", parse_dates=["timestamp"], index_col="timestamp")
series = df["load"].values.reshape(-1, 1)

n = len(series); train_end, val_end = int(n * 0.7), int(n * 0.85)              # CHRONOLOGICAL split — never shuffle
scaler = MinMaxScaler().fit(series[:train_end])                                  # fit on TRAIN only (no leakage)
scaled = scaler.transform(series)

def make_windows(data, look_back=48, horizon=1):
    X, y = [], []
    for i in range(len(data) - look_back - horizon + 1):
        X.append(data[i:i + look_back]); y.append(data[i + look_back:i + look_back + horizon, 0])
    return np.array(X), np.array(y)                    # X: (samples, look_back, features), y: (samples, horizon)
X_tr, y_tr = make_windows(scaled[:train_end]); X_va, y_va = make_windows(scaled[train_end - 48:val_end]); X_te, y_te =
        make_windows(scaled[val_end - 48:])
```

18.3 Models

lstm_forecast.py

```
from tensorflow import keras
from tensorflow.keras import layers
look_back, n_feat, horizon = 48, 1, 1
model = keras.Sequential([
    layers.Input((look_back, n_feat)),
    layers.LSTM(64, return_sequences=True), layers.Dropout(0.2),
    layers.LSTM(32), layers.Dense(16, activation="relu"),
    layers.Dense(horizon)])                                  # linear output for regression
model.compile(keras.optimizers.Adam(1e-3), loss="mse", metrics=["mae"])
model.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=100, batch_size=64, shuffle=False,
          callbacks=[keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True)])

pred = scaler.inverse_transform(model.predict(X_te)); true = scaler.inverse_transform(y_te)   # back to original units
mae = np.mean(np.abs(pred - true)); rmse = np.sqrt(np.mean((pred - true) ** 2)); mape = np.mean(np.abs((true - pred) /
        true)) * 100

# Variants
gru = keras.Sequential([layers.Input((look_back, n_feat)), layers.GRU(64), layers.Dense(horizon)])
cnn_lstm = keras.Sequential([layers.Input((look_back, n_feat)), layers.Conv1D(32, 3, activation="relu"),
        layers.MaxPooling1D(2), layers.LSTM(32), layers.Dense(horizon)])
bilstm = keras.Sequential([layers.Input((look_back, n_feat)), layers.Bidirectional(layers.LSTM(32)),
        layers.Dense(horizon)])   # OK: the window is fully in the past
```

18.4 Multivariate and multi-step forecasting

Approach	How
Multivariate inputs	features = [load, temperature, hour_sin, hour_cos, is_holiday, lag_24 …] → X shape (samples, look_back, n_features)
Recursive multi-step	predict 1 step, append prediction, repeat (errors accumulate)
Direct multi-output	Dense(horizon) predicts all future steps at once (used above with horizon > 1)
Seq2seq	encoder LSTM reads history, decoder LSTM generates the horizon (can use known future covariates)

18.5 Baselines and evaluation

$$\text{MAE} = \frac{1}{n}\sum|y-\hat{y}| \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum(y-\hat{y})^2} \quad \text{MAPE} = \frac{100}{n}\sum\left|\frac{y-\hat{y}}{y}\right| \quad \text{MASE} = \frac{\text{MAE}}{\text{MAE}_{\text{naive}}}$$

- Always compare with **naive** ($\hat{y}_{t+1} = y_t$), **seasonal naive** (value one season ago), moving average, ARIMA/SARIMA, Prophet, and gradient boosting on lag features — LSTMs often do *not* beat strong baselines on small univariate series.
- **Walk-forward (rolling-origin) validation**: train on [0, t], test on (t, t+h], move t forward; sklearn TimeSeriesSplit.
- Plot predictions vs actuals and residuals; check errors by hour/day/season.
- Pitfalls: shuffling, scaling on full data, using future information in features, evaluating in scaled units, forgetting to inverse-transform, too short look-back for the seasonality.

Interview tip: Modern alternatives: Temporal Convolutional Networks, N-BEATS / N-HiTS, Temporal Fusion Transformer, PatchTST, and pre-trained time-series foundation models (e.g. TimesFM, Chronos). Mention them in interviews as the 'next step' after LSTM.

Part IV — Generative Models

Generative vs discriminative models, GANs, advanced GAN variants, autoencoders and VAEs, and how they compare with diffusion models and LLMs

CHAPTER 19

Generative Models Overview & GAN Fundamentals

Discriminative vs generative • taxonomy of generative models • GAN idea • minimax objective • optimal discriminator & JS divergence • training algorithm • non-saturating loss • DCGAN guidelines • Keras implementation • training problems • evaluation (IS, FID)

19.1 Discriminative vs generative

	Discriminative	Generative
Learns	P(y | x) — decision boundary	P(x) or P(x | y) — the data distribution
Can	classify, regress	sample new data, estimate density, fill in missing parts
Examples	logistic regression, CNN classifier, BERT classifier	Naive Bayes, GMM, VAE, GAN, diffusion, GPT

Taxonomy

Family	Likelihood	Idea	Examples
Autoregressive	explicit, exact	factorise $P(x) = \Pi\, P(x_t \mid x_{<t})$	PixelCNN, WaveNet, GPT
Variational autoencoders	explicit, lower bound (ELBO)	latent variable model with encoder/decoder	VAE, VQ-VAE
Normalising flows	explicit, exact	invertible transforms with tractable Jacobians	RealNVP, Glow
GANs	implicit	adversarial game between generator and discriminator	DCGAN, StyleGAN
Diffusion / score-based	lower bound	learn to reverse a noising process	DDPM, Stable Diffusion
Energy-based	unnormalised	learn an energy function	EBMs

19.2 The GAN idea (Goodfellow et al., 2014)

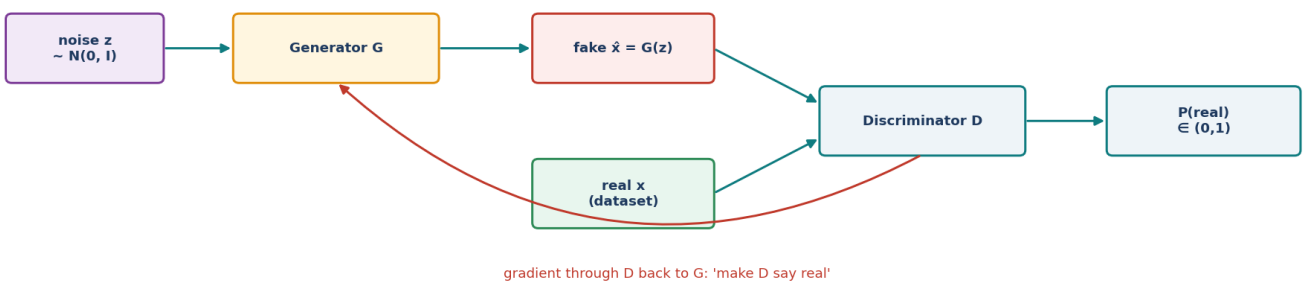


Figure: Generator vs discriminator.

Two networks play a game. The **generator G** maps random noise z to fake samples G(z) — a counterfeiter. The **discriminator D** outputs the probability that its input is real — the police. D is trained to tell real from fake; G is trained to fool D. As both improve, G's samples become indistinguishable from real data.

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$$

What the game optimises

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \qquad \Rightarrow \qquad V(D^*, G) = -\log 4 + 2 \cdot JS(p_{data} \,\|\, p_g)$$

For a fixed G the best discriminator is the ratio above; plugging it in shows that G minimises the **Jensen–Shannon divergence** between real and generated distributions. Global optimum: $p_g = p_{data}$ and D(x) = ½ everywhere.

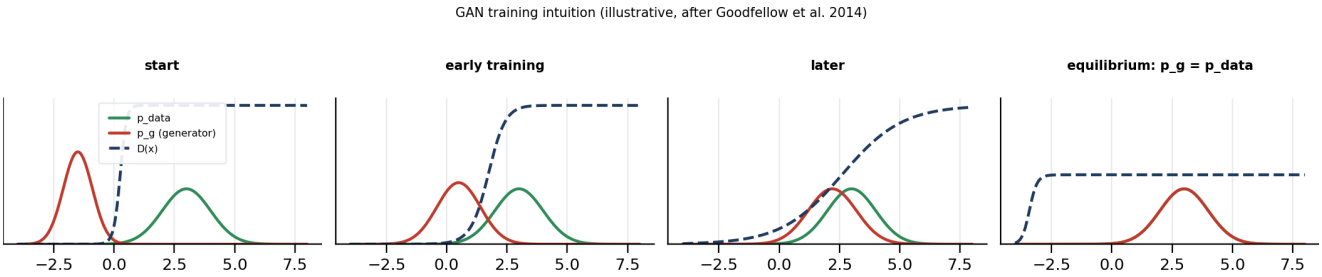


Figure: As training progresses the generator distribution moves onto the data distribution and D becomes 0.5 everywhere (illustrative).

19.3 Training algorithm

1. Sample a minibatch of real data x and noise z; generate fakes G(z).
2. **Update D** (k steps, often k = 1): maximise log D(x) + log(1 − D(G(z))) ⇔ binary cross-entropy with labels real = 1, fake = 0.
3. Sample new noise; **update G** (D frozen): in practice maximise log D(G(z)) — the **non-saturating loss** — i.e. BCE with fake samples labelled as real. Early in training D easily rejects fakes, so log(1 − D(G(z))) has vanishing gradient; the non-saturating version gives strong gradients.
4. Repeat; monitor both losses and, most importantly, generated samples.

19.4 DCGAN architecture guidelines (Radford et al., 2015)

- Replace pooling with strided convolutions (D) and transposed convolutions (G).
- BatchNorm in both networks (not in G's output or D's input layer).
- No fully connected hidden layers in deep architectures.
- G: ReLU in hidden layers, **tanh** output (scale images to [−1, 1]). D: **LeakyReLU** (0.2).
- Adam with lr = 2e-4, β_1 = 0.5.

dcgan_mnist.py

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
(x, _), _ = keras.datasets.mnist.load_data(); x = (x.astype("float32") - 127.5) / 127.5; x = x[..., None]   # [-1, 1]
ds = tf.data.Dataset.from_tensor_slices(x).shuffle(60000).batch(128, drop_remainder=True)
LATENT = 100
G = keras.Sequential([layers.Input((LATENT,)), layers.Dense(7 * 7 * 128, use_bias=False), layers.BatchNormalization(),
        layers.ReLU(), layers.Reshape((7, 7, 128)),
    layers.Conv2DTranspose(64, 4, strides=2, padding="same", use_bias=False), layers.BatchNormalization(), layers.ReLU(),
        # 14×14
    layers.Conv2DTranspose(1, 4, strides=2, padding="same", activation="tanh")])
        # 28×28×1
D = keras.Sequential([layers.Input((28, 28, 1)), layers.Conv2D(64, 4, strides=2, padding="same"), layers.LeakyReLU(0.2),
    layers.Conv2D(128, 4, strides=2, padding="same"), layers.BatchNormalization(), layers.LeakyReLU(0.2), layers.Flatten(),
        layers.Dense(1)])   # logits
bce = keras.losses.BinaryCrossentropy(from_logits=True)
g_opt = keras.optimizers.Adam(2e-4, beta_1=0.5); d_opt = keras.optimizers.Adam(2e-4, beta_1=0.5)

@tf.function
def train_step(real):
    z = tf.random.normal((tf.shape(real)[0], LATENT))
    with tf.GradientTape() as gt, tf.GradientTape() as dt:
        fake = G(z, training=True)
        r_logit, f_logit = D(real, training=True), D(fake, training=True)
        d_loss = bce(tf.ones_like(r_logit) * 0.9, r_logit) + bce(tf.zeros_like(f_logit), f_logit)   # one-sided label
        smoothing
        g_loss = bce(tf.ones_like(f_logit), f_logit)                                               # non-saturating G loss
    d_opt.apply_gradients(zip(dt.gradient(d_loss, D.trainable_variables), D.trainable_variables))
    g_opt.apply_gradients(zip(gt.gradient(g_loss, G.trainable_variables), G.trainable_variables))
    return d_loss, g_loss
for epoch in range(50):
    for real in ds: d_loss, g_loss = train_step(real)
    print(epoch, float(d_loss), float(g_loss))      # save a grid of G(fixed_z) every epoch to watch progress
```

19.5 Why GANs are hard to train

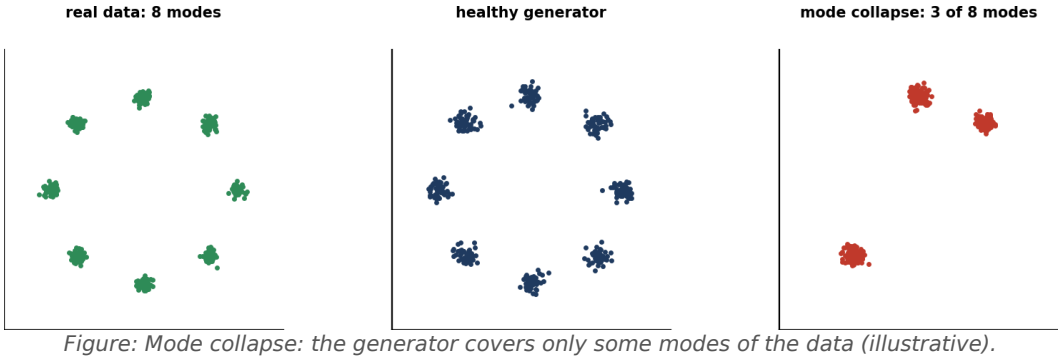


Figure: Mode collapse: the generator covers only some modes of the data (illustrative).

Problem	Symptom	Remedies
Mode collapse	G produces few kinds of samples (same digit, same face)	minibatch discrimination, unrolled GAN, WGAN, diversity losses, PacGAN
Non-convergence / oscillation	losses oscillate; quality goes up and down	TTUR (different LRs), spectral normalisation, gradient penalty, balanced architectures
Vanishing gradients for G	D too strong, D(G(z)) ≈ 0	non-saturating loss, Wasserstein loss, label smoothing, noise on D inputs
Losses don't indicate quality	g_loss tells little	watch samples; use FID
Hyper-parameter sensitivity	small changes break training	follow DCGAN/StyleGAN recipes

19.6 Evaluating generative models

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right)$$

Fréchet Inception Distance between Inception-v3 feature statistics of real and generated images; lower is better

- **Inception Score (IS)**: high when each image is confidently classified (quality) and classes are diverse; ignores real data.
- **FID**: standard metric, compares to real data; sensitive to both quality and diversity.
- **Precision/recall for generative models**: separate fidelity and coverage.
- **Human evaluation** and task-based metrics (CLIP score for text-to-image).

CHAPTER 20

Advanced GAN Variants & Variational Autoencoders (Conceptual)

Conditional GAN • WGAN & WGAN-GP • LSGAN • Pix2Pix • CycleGAN • StyleGAN 1–3 • ProGAN • SRGAN/ESRGAN • BigGAN • autoencoders (undercomplete, denoising, sparse) • VAE: latent variables, ELBO, KL, reparameterisation • β-VAE • CVAE • VQ-VAE • AE vs VAE vs GAN

20.1 Advanced GAN variants

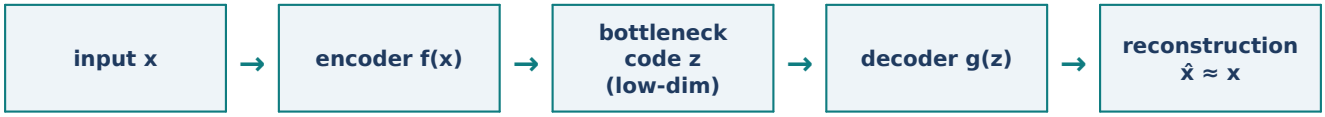
Variant	Key idea	Use
Conditional GAN (cGAN, 2014)	feed a condition y (class label, text, image) to both G and D: G(z, y), D(x, y)	generate a chosen class; basis of image-to-image models
WGAN (2017)	replace JS divergence with the **Wasserstein-1 (earth mover's) distance**; D becomes a 'critic' outputting a score (no sigmoid); critic must be 1-Lipschitz (weight clipping)	smooth, meaningful loss that correlates with quality; less mode collapse
WGAN-GP	enforce Lipschitz with a **gradient penalty** $\lambda(\|\nabla D(\hat{x})\| - 1)^2$ on interpolates	stable default for many GANs
LSGAN	least-squares loss instead of log loss	more stable gradients
Spectral-norm GAN (SNGAN)	normalise D's weight matrices by their largest singular value	simple, effective stabiliser
ProGAN (2017)	progressively grow G and D from 4×4 to 1024×1024	first high-res faces
StyleGAN 1/2/3 (2018–21)	mapping network z → w; style injected per layer via AdaIN (v1) / weight demodulation (v2); per-pixel noise for stochastic detail; style mixing; v3 alias-free	photorealistic faces, controllable attributes (thispersondoesnotexist)
Pix2Pix (2017)	cGAN for **paired** image-to-image translation; U-Net generator + PatchGAN discriminator (judges N×N patches) + L1 loss	sketch→photo, map↔aerial, day→night
CycleGAN (2017)	**unpaired** translation with two generators G: X→Y, F: Y→X and cycle-consistency loss $\|F(G(x)) - x\|_1$	horse↔zebra, summer↔winter, photo↔Monet
SRGAN / ESRGAN	perceptual (VGG feature) loss + adversarial loss for super-resolution	upscaling photos/video
BigGAN (2018)	large-scale class-conditional GAN, orthogonal regularisation, truncation trick	high-fidelity ImageNet samples
GigaGAN / GAN-based distillation	scaled GANs; GANs used to speed up diffusion samplers	fast text-to-image

$$W(p_r, p_g) = \sup_{\|f\|_L \le 1} \mathbb{E}_{x\sim p_r}[f(x)] - \mathbb{E}_{x\sim p_g}[f(x)] \qquad \mathcal{L}_{GP} = \lambda\, \mathbb{E}_{\hat{x}}\left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2\right]$$

$$\mathcal{L}_{cyc} = \mathbb{E}_x[\|F(G(x)) - x\|_1] + \mathbb{E}_y[\|G(F(y)) - y\|_1]$$

CycleGAN cycle-consistency loss

20.2 Autoencoders



$$\mathcal{L}_{AE} = \|x - g(f(x))\|^2$$

Type	Idea	Use
Undercomplete	bottleneck smaller than input	dimensionality reduction (non-linear PCA)
Denoising	corrupt input, reconstruct clean	denoising, robust features
Sparse	sparsity penalty on activations	feature learning
Convolutional	conv encoder, transposed-conv decoder	images
Anomaly detection	train on normal data; high reconstruction error = anomaly	fraud, machine faults

A plain autoencoder's latent space has gaps and no structure, so sampling random z gives garbage — it is not a good generator. The VAE fixes this.

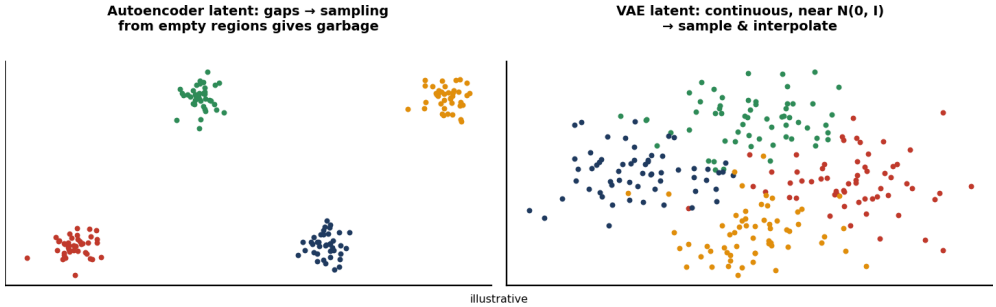


Figure: Autoencoder vs VAE latent spaces (illustrative).

20.3 Variational Autoencoder (Kingma & Welling, 2013)

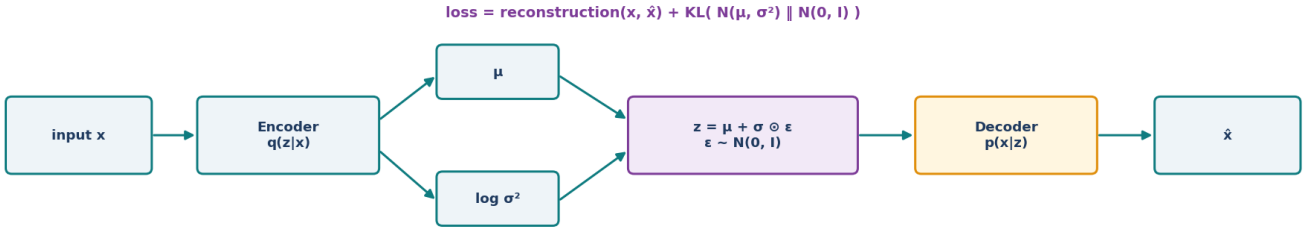


Figure: VAE: the encoder outputs a distribution; sampling uses the reparameterisation trick.

- The encoder outputs the parameters of a distribution $q_\varphi(z|x) = N(\mu, \sigma^2)$ instead of a single point.
- A sample z is decoded by $p_\theta(x|z)$.
- A KL term pulls every q(z|x) toward the prior N(0, I) → the latent space becomes continuous and dense: nearby z decode to similar outputs; you can **sample** new data (z ~ N(0, I)) and **interpolate** smoothly.

$$\log p_\theta(x) \geq \mathcal{L}_{ELBO} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \,\|\, \mathcal{N}(0, I))$$

Evidence Lower BOund: reconstruction term − KL term

$$D_{KL} = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right) \qquad z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Reparameterisation trick: sampling is not differentiable, so write z as a deterministic function of μ, σ and external noise ε; gradients flow through μ and σ. Loss to minimise = reconstruction loss (BCE/MSE) + KL.

vae_keras.py

```
class Sampling(layers.Layer):
    def call(self, inputs):
        mu, log_var = inputs; eps = tf.random.normal(tf.shape(mu)); return mu + tf.exp(0.5 * log_var) * eps
enc_in = keras.Input((28, 28, 1)); h = layers.Flatten()(enc_in); h = layers.Dense(256, activation="relu")(h)
mu = layers.Dense(2)(h); log_var = layers.Dense(2)(h); z = Sampling()([mu, log_var])
encoder = keras.Model(enc_in, [mu, log_var, z])
dec_in = keras.Input((2,)); d = layers.Dense(256, activation="relu")(dec_in); d = layers.Dense(784, activation="sigmoid")(d)
decoder = keras.Model(dec_in, layers.Reshape((28, 28, 1))(d))
def vae_loss(x, x_hat, mu, log_var):
    rec = tf.reduce_mean(tf.reduce_sum(keras.losses.binary_crossentropy(x, x_hat), axis=(1, 2)))
    kl = -0.5 * tf.reduce_mean(tf.reduce_sum(1 + log_var - tf.square(mu) - tf.exp(log_var), axis=1))
    return rec + kl
# after training: decoder(tf.random.normal((16, 2))) → 16 new digits; grid over z1, z2 shows smooth morphing
```

VAE variant	Idea
β-VAE	weight KL by β > 1 → more disentangled latent factors (one dimension = one attribute)
Conditional VAE	condition encoder and decoder on a label → generate a chosen class
VQ-VAE	discrete latent codebook (vector quantisation); used for audio/image tokens (DALL-E 1, many audio codecs)
Latent diffusion VAE	Stable Diffusion's VAE compresses 512×512×3 images to 64×64×4 latents

20.4 Autoencoder vs VAE vs GAN

	Autoencoder	VAE	GAN
Goal	compress / reconstruct	probabilistic generation	realistic generation
Latent space	unstructured	smooth, N(0, I)	noise input, no encoder
Training	stable (MSE)	stable (ELBO)	unstable (adversarial)
Sample quality	n/a	often blurry	sharp
Likelihood	no	lower bound	no
Mode coverage	—	good	risk of collapse
Use	denoising, anomaly detection, features	smooth latent spaces, representation learning, compression in diffusion	photorealism, super-resolution, image translation

CHAPTER 21

Hands-on Demo & Comparison with Modern GenAI + Weekly Assessment 03

Demo plan (VAE vs DCGAN on MNIST) • diffusion models in one page • autoregressive transformers & LLMs • comparison table • where GANs/VAEs live inside modern systems • assessment questions

21.1 Hands-on demo plan

1. Train the DCGAN (Chapter 19) and the VAE (Chapter 20) on MNIST for the same number of epochs.
2. Generate 64 samples from each: compare sharpness (GAN) vs smoothness/blur (VAE).
3. Latent interpolation: for the VAE decode points on a line between two encodings; for the GAN interpolate between two z vectors.
4. Compute FID for both (pytorch-fid / torchmetrics) on 10k samples.
5. Optional: run a pre-trained diffusion model (diffusers) and a GPT-style model to see today's quality.

modern_genai_demo.py

```
# Modern generative AI in a few lines (Hugging Face)
from diffusers import StableDiffusionPipeline
import torch
pipe = StableDiffusionPipeline.from_pretrained("stabilityai/stable-diffusion-2-1", torch_dtype=torch.float16).to("cuda")
img = pipe("a watercolor painting of a robot reading a book", num_inference_steps=30, guidance_scale=7.5).images[0]

from transformers import pipeline
gen = pipeline("text-generation", model="gpt2"); print(gen("Deep learning is", max_new_tokens=30)[0]["generated_text"])
```

21.2 Diffusion models in one page

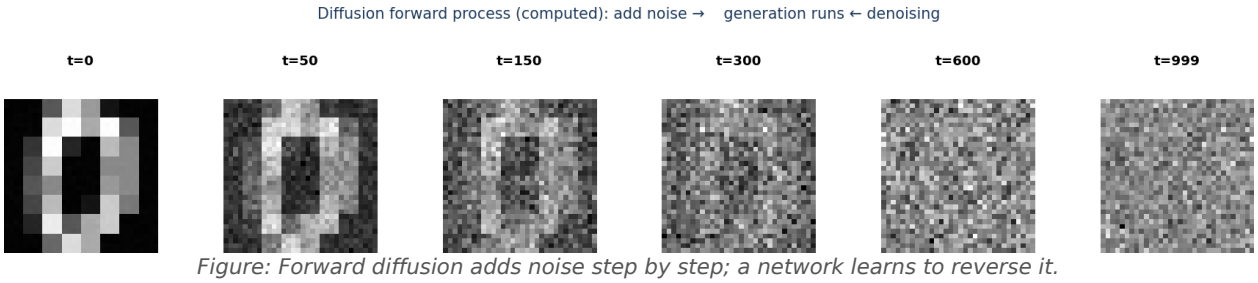


Figure: Forward diffusion adds noise step by step; a network learns to reverse it.

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \mathcal{L} = \mathbb{E}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right]$$

Training: take a real image, add a random amount of Gaussian noise (step t), train a U-Net / transformer to predict the noise. Generation: start from pure noise and denoise in 20–1000 steps. Text conditioning via cross-attention + classifier-free guidance. Stable training (simple regression loss), excellent quality and diversity; slower sampling (reduced by DDIM solvers and distillation). Stable Diffusion runs in the latent space of a VAE.

21.3 Autoregressive transformers (LLMs)

GPT-style decoders model $P(x) = \Pi\, P(x_t \mid x_{<t})$ with masked self-attention, trained on trillions of tokens, then instruction-tuned and aligned (RLHF/DPO). The same recipe generates code, images as tokens, audio codec tokens and multimodal outputs.

21.4 Comparison

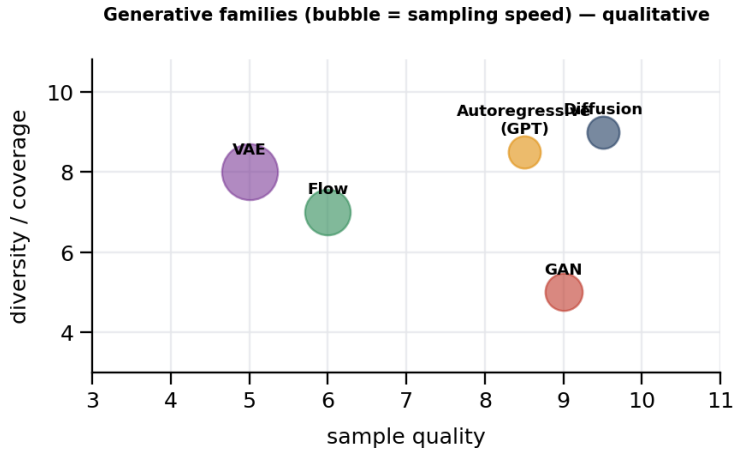


Figure: Qualitative positioning of generative families.

	VAE	GAN	Diffusion	Autoregressive transformer
Training	stable	unstable	stable	stable
Quality	blurry	sharp	state of the art	state of the art (text)
Diversity	good	mode-collapse risk	excellent	excellent
Sampling speed	1 pass	1 pass (fast)	many steps	token by token
Likelihood	bound	none	bound	exact
Controllability	latent arithmetic	latent/style (StyleGAN)	text prompts, ControlNet	prompts
Typical today	latent compression inside diffusion, VQ tokenisers	super-resolution, real-time, distillation of diffusion	images, video, audio	LLMs, code, multimodal

Interview tip: GANs and VAEs are not obsolete: Stable Diffusion = VAE + diffusion U-Net; adversarial losses train the VAE decoders and audio codecs; GAN-style discriminators distil diffusion into 1–4-step generators; VQ-VAEs tokenise images and audio for transformers.

21.5 Weekly Assessment 03 (with answers)

Q1. What does the discriminator output in a standard GAN?

The probability (or logit) that its input is a real sample.

Q2. Write the GAN minimax objective.

min_G max_D E[log D(x)] + E[log(1 − D(G(z)))].

Q3. Why is the non-saturating generator loss used?

log(1 − D(G(z))) has vanishing gradients when D confidently rejects fakes early in training; maximising log D(G(z)) gives strong gradients.

Q4. What is mode collapse?

The generator produces only a few kinds of outputs that fool D, ignoring other modes of the data distribution.

Q5. What divergence does the original GAN minimise at the optimal D?

Jensen–Shannon divergence.

Q6. What does WGAN change?

Uses the Wasserstein distance with a 1-Lipschitz critic (weight clipping or gradient penalty), giving smoother gradients and a loss that tracks quality.

Q7. Pix2Pix vs CycleGAN?

Pix2Pix needs paired images (cGAN + L1); CycleGAN works with unpaired domains using two generators and cycle-consistency loss.

Q8. What is special about StyleGAN?

A mapping network to an intermediate latent w and per-layer style modulation (AdaIN/demodulation) plus noise inputs, enabling controllable, photorealistic images and style mixing.

Q9. Why can't a plain autoencoder generate new data well?

Its latent space is irregular with gaps; random latent points decode to meaningless outputs.

Q10. What are the two terms of the VAE loss?

Reconstruction loss and KL divergence between q(z|x) and the prior N(0, I).

Q11. What is the reparameterisation trick?

$z = \mu + \sigma \cdot \varepsilon$ with $\varepsilon \sim N(0, I)$, making sampling differentiable with respect to μ and σ.

Q12. Why are VAE samples often blurry?

Pixel-wise reconstruction losses (MSE/BCE) average over plausible outputs, and the KL regulariser limits latent detail.

Q13. Which metric compares Inception features of real and generated images?

FID (Fréchet Inception Distance), lower is better.

Q14. Name one way GANs are used in modern GenAI.

Adversarial losses in VAE decoders/audio codecs, super-resolution (ESRGAN), or distilling diffusion models into few-step generators.

Q15. Diffusion vs GAN in one line?

Diffusion trains stably by learning to denoise and gives better diversity but samples slowly; GANs sample in one pass but train unstably.

Q16. In DCGAN, which activation is used at the generator output and why?

tanh, because images are scaled to [−1, 1].

CHAPTER 22

Interview Questions & Answers

Fundamentals • training • CNN • architectures & transfer learning • detection & segmentation • RNN/LSTM • seq2seq & attention • generative models • practical scenarios

22.1 Fundamentals & training

Q1. Why do neural networks need non-linear activations?

Without them a stack of layers is equivalent to a single linear layer and can only learn linear functions.

Q2. Explain backpropagation.

After a forward pass computes the loss, the chain rule is applied backwards layer by layer to compute the gradient of the loss with respect to every weight; an optimiser then updates the weights.

Q3. Why ReLU over sigmoid in hidden layers?

ReLU doesn't saturate for positive inputs (gradient 1), avoiding vanishing gradients; it is cheap and gives sparse activations. Its issue is dying ReLU, addressed by Leaky ReLU/GELU.

Q4. Epoch vs batch vs iteration?

Epoch = one pass over the training set; batch = subset used for one update; iteration = one update step.

Q5. What does the learning rate control, and what happens if it is too high or low?

The step size of updates; too high diverges/oscillates, too low trains slowly or gets stuck.

Q6. SGD vs Adam?

SGD uses one global step size (with momentum it often generalises well); Adam adapts per-parameter step sizes using first and second moment estimates and converges faster with little tuning.

Q7. Why not initialise all weights to zero?

All neurons in a layer would compute the same output and receive the same gradient — symmetry is never broken.

Q8. Xavier vs He initialisation?

Xavier (variance 2/(n_in + n_out)) suits tanh/sigmoid; He (variance 2/n_in) compensates for ReLU zeroing half the activations.

Q9. What does batch normalisation do?

Normalises each feature over the mini-batch and re-scales with learned γ, β, stabilising and speeding up training and allowing higher learning rates; uses running statistics at inference.

Q10. How does dropout prevent overfitting?

Randomly dropping units forces redundant, robust features and approximates averaging many thinned networks; it is disabled at inference.

Q11. How do you detect overfitting in a neural network?

Training loss keeps falling while validation loss rises; fix with more data, augmentation, regularisation, dropout, smaller model or early stopping.

Q12. Softmax + cross-entropy gradient?

$\partial L/\partial z = \hat{y} - y$, which is simple and does not saturate.

22.2 CNNs

Q13. Why CNNs for images?

Local connectivity and weight sharing give far fewer parameters, exploit spatial structure and provide translation equivariance; pooling adds some invariance.

Q14. Output size formula?

$O = \lfloor (W - K + 2P)/S \rfloor + 1$.

Q15. Parameters of a conv layer?

$(K \cdot K \cdot C_{in} + 1) \cdot C_{out}$.

Q16. Max vs average pooling?

Max keeps the strongest activation (detects presence of a feature); average smooths. Global average pooling replaces large dense layers.

Q17. What is a receptive field?

The region of the input that influences a given output unit; it grows with depth, kernel size, stride and pooling.

Q18. Why 1×1 convolutions?

To change channel depth cheaply, add non-linearity and mix channels (bottlenecks).

Q19. What problem does ResNet solve?

The degradation problem of very deep plain networks; skip connections let layers learn residuals and give gradients an identity path.

Q20. Depthwise separable convolution?

A per-channel spatial conv followed by a 1×1 pointwise conv — about K^2 times cheaper than a standard conv; used in MobileNet.

Q21. What is EfficientNet's compound scaling?

Scaling depth, width and resolution together with fixed ratios instead of one dimension at a time.

Q22. Feature extraction vs fine-tuning?

Feature extraction freezes the pre-trained backbone and trains a new head; fine-tuning also unfreezes some layers and trains them with a small learning rate.

22.3 Detection, segmentation & visualisation

Q23. IoU and NMS?

IoU = overlap/union of two boxes; NMS keeps the highest-scoring box and removes boxes overlapping it above an IoU threshold.

Q24. One-stage vs two-stage detectors?

Two-stage (Faster R-CNN) proposes regions then classifies them — accurate but slower; one-stage (YOLO, SSD) predict boxes and classes in one pass — real-time.

Q25. What is the RPN?

A small network in Faster R-CNN that slides over the feature map and, for each anchor, predicts objectness and box offsets to propose regions.

Q26. YOLOv1 output tensor for VOC?

$S \times S \times (B \cdot 5 + C) = 7 \times 7 \times 30$.

Q27. What is mAP@[.5:.95]?

Mean average precision averaged over IoU thresholds 0.5 to 0.95 in steps of 0.05 (COCO metric).

Q28. Why focal loss?

It down-weights easy background examples so one-stage detectors are not overwhelmed by class imbalance.

Q29. Semantic vs instance segmentation?

Semantic labels each pixel by class; instance separates individual objects.

Q30. Why skip connections in U-Net?

To pass high-resolution encoder features to the decoder, restoring spatial detail lost by downsampling.

Q31. RoI Align vs RoI Pooling?

RoI Align avoids coordinate rounding and uses bilinear interpolation, giving pixel-accurate features essential for masks.

Q32. Dice loss?

$1 - 2|A \cap B|/(|A| + |B|)$; directly optimises overlap, good for small foreground objects.

Q33. How does Grad-CAM work?

It weights the last conv layer's feature maps by the average gradient of the class score and applies ReLU to produce a class-specific heat-map.

22.4 RNN, LSTM, seq2seq & attention

Q34. Why RNNs for sequences?

They process inputs step by step with a hidden state that carries information forward and share weights across time, handling variable-length sequences.

Q35. What is BPTT?

Backpropagation through the unrolled network over time, summing gradients of the shared weights across steps.

Q36. Why do vanilla RNNs suffer vanishing gradients?

The gradient involves a product of many Jacobians containing W_hh and $\tanh' \leq 1$, which shrinks exponentially over long time spans.

Q37. How do you handle exploding gradients?

Gradient clipping by norm, lower learning rates, careful initialisation.

Q38. Explain the LSTM gates.

Forget gate removes information from the cell state, input gate writes new candidate information, output gate decides what part of the cell state becomes the hidden state.

Q39. Why does LSTM mitigate vanishing gradients?

Its cell state is updated additively; when the forget gate is near 1, gradients flow across many steps almost unchanged.

Q40. LSTM vs GRU?

GRU has two gates and no separate cell state — fewer parameters and faster; LSTM has three gates and is sometimes better on long, complex sequences.

Q41. LSTM parameter count?

$4 \cdot (h \cdot (h + d) + h)$ for hidden size h and input size d.

Q42. When can't you use a bidirectional RNN?

When future inputs are unavailable: real-time streaming, next-step forecasting, autoregressive generation.

Q43. What is teacher forcing?

Feeding the ground-truth previous token to the decoder during training; causes exposure bias at inference.

Q44. Greedy vs beam search?

Greedy picks the most likely token each step; beam search keeps the k best partial sequences and finds higher-probability outputs.

Q45. What problem does attention solve in seq2seq?

The fixed-length context bottleneck: the decoder can look at all encoder states with learned weights at every step.

Q46. Bahdanau vs Luong attention?

Bahdanau uses an additive feed-forward score with the previous decoder state; Luong uses multiplicative (dot/general) scores with the current state.

Q47. Why scale by √d_k?

To keep dot-product magnitudes from growing with dimension, preventing softmax saturation and tiny gradients.

Q48. Why did transformers replace RNNs?

Parallel computation over all positions, constant path length between any two tokens, and better scaling with data and compute.

Q49. How do you forecast time series with LSTM without leakage?

Chronological splits, scaler fit on training data only, sliding windows built within each split, walk-forward validation and comparison with naive baselines.

22.5 Generative models

Q50. GAN in one sentence?

A generator learns to produce samples that a discriminator cannot distinguish from real data, through an adversarial minimax game.

Q51. Common GAN failure modes?

Mode collapse, non-convergence/oscillation, vanishing generator gradients.

Q52. What is the VAE reparameterisation trick?

$z = \mu + \sigma \cdot \varepsilon$ with $\varepsilon \sim N(0, I)$, making sampling differentiable.

Q53. Autoencoder vs VAE?

An autoencoder learns a deterministic code for reconstruction; a VAE learns a regularised probabilistic latent space from which new samples can be drawn.

Q54. Why diffusion models dominate image generation?

Stable training via a denoising regression objective, excellent diversity and quality, and easy conditioning on text.

22.6 Practical scenarios

Q55. You have 800 labelled medical images. Approach?

Transfer learning from an ImageNet (or medical) pre-trained CNN, freeze then fine-tune top layers with a small LR, heavy augmentation, stratified CV, class weights, Grad-CAM checks with clinicians.

Q56. Validation accuracy stuck at 10% on a 10-class problem.

The model isn't learning: check label/loss mismatch, data scaling, learning rate (too high), output activation vs from_logits, and data pipeline (labels shuffled separately from images).

Q57. Loss becomes NaN during training.

Lower the learning rate, add gradient clipping, check for log(0)/division by zero, check input normalisation and bad data.

Q58. Need real-time detection on a phone.

A small one-stage detector (YOLO-nano/SSD-MobileNet), quantised (INT8 TFLite) and possibly pruned.

Q59. Model performs well on test but fails in the factory.

Distribution shift (camera, lighting, new defects): collect production samples, augment for those conditions, monitor drift, fine-tune.

CHAPTER 23

Cheat Sheet

Formulas • shapes & parameter counts • architecture timeline • which model when

23.1 Formulas

Neuron	$a = f(\mathbf{w}^T\mathbf{x} + b)$
Sigmoid / softmax	$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \frac{e^{z_k}}{\sum e^{z_j}}$
Cross-entropy	$-\sum y_k \log \hat{y}_k$
Gradient descent	$\theta \leftarrow \theta - \eta \nabla \mathcal{L}$
Backprop	$\delta^{[l]} = (W^{[l+1]T}\delta^{[l+1]}) \odot f'(z^{[l]})$
Adam	$\theta \leftarrow \theta - \eta \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$
He init	$\text{Var}(w) = 2/n_{in}$
BatchNorm	$\gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta$
Conv output	$O = \lfloor (W - K + 2P)/S \rfloor + 1$
Conv params	$(K^2 C_{in} + 1) C_{out}$
ResNet	$y = F(x) + x$
IoU	$\lvert A \cap B \rvert / \lvert A \cup B \rvert$
Dice	$2\lvert A \cap B \rvert / (\lvert A \rvert + \lvert B \rvert)$
Focal loss	$-\alpha(1-p_t)^\gamma \log p_t$
RNN	$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b)$
LSTM cell	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \; h_t = o_t \odot \tanh C_t$
GRU	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
LSTM params	$4(h(h+d) + h)$
Bahdanau	$\alpha_{t,i} = \text{softmax}(v^T \tanh(W s_{t-1} + U h_i))$
Attention	$\text{softmax}(QK^T/\sqrt{d_k})V$

GAN	$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$
VAE	$E[\log p(x\|z)] - D_{KL}(q(z\|x)\|N(0, I))$
Diffusion	$\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$

23.2 Which model when?

Problem	Go-to model
Tabular data	gradient boosting first; MLP if very large data / embeddings
Image classification (small data)	transfer learning: EfficientNet / ResNet / ConvNeXt
Image classification (edge)	MobileNetV3 / EfficientNet-Lite, quantised
Object detection (real time)	YOLO family
Detection (max accuracy)	Faster R-CNN + FPN / DETR variants
Semantic segmentation	U-Net (medical), DeepLabv3+ / SegFormer (scenes)
Instance segmentation	Mask R-CNN, YOLOv8-seg, SAM-assisted
Text classification / NER	fine-tuned transformer (BERT); Bi-LSTM as lighter baseline
Time-series forecasting	baselines + gradient boosting; LSTM/GRU/TCN; TFT / foundation models
Translation / summarisation	transformer seq2seq (T5, BART) or LLMs
Image generation	diffusion; GAN for real-time / super-resolution
Anomaly detection	autoencoder reconstruction error, Isolation Forest on embeddings

Watch out: Code in this guide is reference-quality and was not executed in your environment; the figures marked 'computed' were produced with NumPy/scikit-learn/SciPy. Library APIs (Keras 3, torchvision weights, Ultralytics, diffusers) evolve — check the documentation of your installed versions.